ARC White-Box Estimation Challenge 2026 — Phase 2 Participant Handbook

WhestBench: predict per-neuron mean activations of a random 1024x16 ReLU MLP under a 2^{41}-FLOP budget, metered by flopscope. Organised by the Alignment Research Center with AIcrowd.

Status on 21 September 2026: Phase 2 is LIVE. Submissions close 17 October 2026 23:59 UTC (26 days). Team freeze 2 October 2026 (11 days). Write-up deadline 24 October 2026.

Sources: AIcrowd challenge page and leaderboard, the complete whest-starterkit repository (README, CHANGELOG, all 40+ docs pages, all four examples, local_engine, pyproject, .whestignore, as of commit 5eb9aa1 / 16 Sep 2026), the Hugging Face dataset card, arXiv:2605.05179, ARC's blog posts, and the challenge forum (Phase 2 launch post, prize guidelines, participant write-ups). Compiled offline so you never need to re-open the website.

How to use: Part A is what you must know today. Part B is the rules that can zero your score or disqualify you. Part C is the tooling. Parts D-F are scoring, API and mathematics. Part G is what the community has found. Part H is a concrete 26-day plan. Part I is troubleshooting and links.

Table of Contents

Part A — Phase 2 at a glance 7

A1. Countdown and calendar . . . 7

A2. The task in one paragraph . . . 7

A3. Numbers to memorise . . . 8

A4. Prizes and how winners are decided (Overview page, transcribed 21 Sep 2026) . . . 8

Part B — Rules that can zero you or disqualify you 10

B1. Allowed code (exactly three things) . . . 10

B2. Prohibited (no price makes them allowed; disqualifiable; review continues after grading) . . . 10

B3. Permitted and encouraged . . . 10

B4. Enforcement . . . 10

B5. The four caps and what breaching each does . . . 11

Part C — Tooling: install, iterate, evaluate, submit 12

C1. Install (once) . . . 12

C2. The five-stage ladder . . . 12

C3. whest run flags you will actually use . . . 13

C4. Reading the score report (whest run --format json) . . . 13

C5. Dataset facts (verified on the Hugging Face Hub, 21 Sep 2026) . . . 14

C6. Packaging and .whestignore . . . 15

Part D — Scoring and the compute meter 17

D1. The exact formula . . . 17

D2. What the formula implies for strategy . . . 17

D3. flopscope billing . . . 17

D4. Practical meter rules . . . 18

Part E — Estimator API and reference code 19

E1. Contract . . . 19

E2. setup() and state on the grader . . . 19

E3. Failure semantics . . . 20
E4. Bundled examples (verbatim logic) . . . 20
Part F — Mathematics and algorithm directions 22
F1. Rectified-Gaussian moments . . . 22
F2. Why the bundled closures stall, and what the frontier does . . . 22
F3. Measured dead ends (do not repeat) . . . 23
F4. Organizer-listed open directions . . . 23
Part G — Phase 2 discussion digest (all 28 forum threads read in full, 21 Sep 2026) 24
G1. Official announcements and organizer clarifications (mohanty / aicrowd_team) . . . 24
G2. Leaderboard state and what the score clusters are . . . 25
G3. Write-ups: methods, numbers, what failed . . . 25
G3.1 504aldo — factorized K=3 cumulant propagation, MIT code (github.com/504aldo/whest-p2-cumulant-k3) . . . 25
G3.2 yujigon (#330001) — zero fitted constants, rank 64/877, adjusted 1.066e-7 . . . 27
G3.3 venture_agent_ai (#329951) — exact bivariate ReLU covariance + per-layer calibration, 1.32e-7 . . . 27
G3.4 baghyeonsig (#328274) — 0.75/0.25 blend of gain-covariance and whitened antithetic MC, 3.04e-7 . . . 27
G3.5 nkosi_ndwandwe (#18219) — the compute budget is not a lever . . . 27
G3.6 Phase 1 material that still transfers (thylinao, keenanpepper, jamesrahenry, Puffi, uzziel_perez, bin_yong_bong) 28
G4. Grader and infrastructure facts learned the hard way . . . 28
G5. Statistics of the Phase 2 MLPs (keenanpepper, #18199, #18141) . . . 29
G6. Public resources you can build on right now . . . 29
G7. How people work (AI usage, process) . . . 29
G8. Actionable insights distilled for the remaining 26 days . . . 30
Part H — A 26-day plan (21 Sep - 17 Oct 2026) 31
H1. Week 1 (21-27 Sep): pipeline + covariance-level score . . . 31
H2. Week 2 (28 Sep - 4 Oct): third cumulant — the only path below 1e-7 . . . 31
H3. Week 3 (5-11 Oct): optimise and harden . . . 31
H4. Week 4 (12-17 Oct): freeze and designate . . . 32
H5. Daily loop and submission discipline . . . 32
Part J — Prize tracks: we compete for all of them 33

J1. Score prizes (leaderboard / private re-run) . . . 33

J2. Algorithmic contribution prize (20,000 USD, ARC's discretion) — SPECIAL TRACK . . . 33

J2.1 Write-up backbone (as of 21 Sep 2026) — what we already have . . . 34

J3. Community contribution prizes (500 - 5,000 USD each, individual awards) . . . 34

Part K — Living log: experiments and contribution candidates (updated 21 Sep 2026) 35

K1. Graded submissions . . . 35

K2. Local / Kaggle measurements (one local MLP, seed 7, steady-state second call) . . . 36

K3. Contribution candidates found (to write up / report) . . . 37

K3b. Research pilot A (21 Sep): are the Hadamard products of source legs compressible? NO . . . 37

K3c. Research pilot B (21 Sep): where does the old-source content matter? EVERYWHERE . . . 38

K3d. Research pilot C (21 Sep): can the CONTRACTED old-source query be approximated cheaply? NO . 38

K3e. Research pilot D (21 Sep): the aggregate response / pooled-core family. DEAD . . . 39

K3f. Kaggle knob sweep (21 Sep): the author's constants are optimal . . . 40

K3g. Research pilot E (21 Sep): is next-layer old content predictable from the previous slices? NO . . . 40

K3h. Research pilot F (21 Sep): birth-column screening of sources - a real but small cost lever . . . 40

K3i. Research pilot G (21 Sep): ARC's own reference chains at 1024 x 16 - the K=3 reference is NOT the leaders' method . . . 41

K3j. Research pilot H (21 Sep): the shared-basis ranks are at their optimum, and a pooled core cannot beat the stacks . . . 43

K3k. Research pilot I (21 Sep): 'a different closure with compensating errors' - NO (Kaggle, 10 baked-truth MLPs) . . . 43

K3l. Round-3 consultation (21 Sep, GPT-5.6 Sol high + Astra 6 Max) and its two falsifiers - both negative . 44

K3m. Own line (21 Sep): adjoint / linear-response contraction of the slices - the sensitivity IS low-rank, but the response is carried by near-range paths . . . 45

K3n. Round 4 (22 Sep, GPT-5.6 Sol GitHub audit): the angular/spherical input (LIVE) and the query-space row skeleton (dead) . . . 45

K3o. Audit of the negative results (22 Sep): were the failures real or ours? . . . 46

K3p. Precision (22 Sep): float32 rounding is not part of the error . . . 47

K3q. Astra 6 GitHub audit (22 Sep): no new method; a correction of the source algebra and one diagnostic (done) . . . 48

K3r. Map of method families (22 Sep): where each one dies against 2e-8 at <= 0.16 B . . . 48

K3s. Round 5 (22 Sep, GPT-5.6 Sol 'Beyond sum-of-sources'): 14 alternative families - assessment and the two that were measurable . . . 49

K3u. Round 5b (22 Sep, Astra 6 'Beyond sum-of-sources'): six alternatives with executed cost gates - all closed . . . 49

K3v. Residual-structure study V33 (22 Sep): the chain's residual is unpredictable from any cheap per-neuron feature . . . 50

K3t. One-step closure floors and the copula oracle (22 Sep, Kaggle GPU, 1024 x 16, 2e8 samples per layer) 50

K3w. The K3 -> K4 accuracy gain grows with width (22 Sep, ARC reference chains, 2e9-sample GPU truths) 51

K3x. The kappa4 teacher (22-23 Sep): V29's lambda is a fitted COMPENSATOR, not an approximation of the true law . . . 51

K3y. Open-source and community sweep (23 Sep): GitHub, Hugging Face, the AIcrowd forum and the literature . . . 52

1. The kappa4 ceiling is now measured - by the author of the chain, in public . . . 52
2. Correction to K3x . . . 52
3. What exists in public code and data . . . 53

K3z. Three own ideas after the public sweep (23 Sep): extrapolation dead, per-family Strassen and a joint knob search open . . . 53

K3aa. Round 6 (23 Sep): Fable 5 as a working subagent, unbiased column sampling, and the asymmetric contraction rank . . . 54

K3ab. Error budget of the chain (23 Sep): the closure is exact, the error is made by three inputs and carried by the mean . . . 55

K3ac. The corrected kappa4 oracle (23 Sep): the exact kappa4 DIAGONAL is worth -30% - the leaders' raw level . . . 56

K3ad. The complete error map (23 Sep): three numbers per neuron, and the variance is the biggest lever . 57

1. Channel oracles: the true value of ONE input of the mean closure E[relu(z_i)] at every layer . . . 57
2. Where the variance error comes from . . . 57
3. Where the kappa4 diagonal comes from . . . 58
4. Reading . . . 58

K3ae. Round 7 (Fable 5 subagent, 23 Sep): the kappa3 slice error lives in the all-distinct part of the post-ReLU tensor . . . 58

K3af. Round 7 answers (Astra, GPT 5.6 Terra, Grok, Copilot) and the teacher-forced closure (23 Sep) . . 59

K3ag. Round 8 (24 Sep): Fable 5 synthesis of eleven round-7 answers; the 100-MLP knob results; the kernel's missing (1,1,1,1) term . . . 59

K3ah. Moonshot A (24 Sep): the metered cost ledger, the join rebuild, and the free gains shipped . . . 60

K3ai. Round 9 (GPT 5.6 Sol): source-free DHMP killed by its own one-step gate . . . 61

K3aj. The horizon oracle (24 Sep): the kappa3 memory is long, flat and nearly orthogonal across ages . . 61

K3ak. The chain's own per-channel error against the teacher (24 Sep, the Round 11 stage-1 bar) 62

K4. Next experiments queued 62

Part I — Checklists, troubleshooting, links 64

I1. Pre-submission checklist (from the kit, condensed) 64

I2. Troubleshooting table 64

I3. Links 65

Part A — Phase 2 at a glance

A1. Countdown and calendar

Date (UTC)	Event	Days from 21 Sep
22-23 Aug 2026	Phase 2 opened 22 Aug 00:00 UTC (round id 1429; forum post 23 Aug; dataset v2-phase2 baked 13 Aug). First week: failed submissions did not count against the daily cap.	-30
21 Sep 2026	Today (this handbook). Challenge page: 69.2k views, 992 participants, 22,008 submissions.	0
2 Oct 2026 23:59	**Registration / team freeze.** Teams must be final; the single designated final submission belongs to the team.	+11
17 Oct 2026 23:59	**Submissions close.**	+26
after 17 Oct	Submission selection closes (date to be published near the end of the phase): nominate up to TWO submissions. Private re-evaluation on a fresh suite 17 Oct - 7 Nov.	-
15 Nov 2026	Winner announcement.	+55
24 Oct 2026 23:59	Technical write-up deadline (algorithmic-contribution prize; one PDF tied to one graded submission ID).	+33

Submission quota: **10 per team per UTC day**, counter resets 00:00 UTC, unused slots do not roll over, failed submissions count. Over the remaining 26 days that is at most 260 graded attempts; in practice budget 2-4 meaningful submissions per day and rehearse locally (Stage 4) before every one.

A2. The task in one paragraph

predict(mlp, budget) receives 16 float32 weight matrices of shape (1024, 1024) (He init N(0, 2/1024), no biases) and must return a (16, 1024) array whose row i is E[ReLU-output of layer i] when the input $x \sim N(0, I_{1024})$ and each layer computes y = ReLU(W.T @ x). Ground truth is a 1e9-sample Monte Carlo mean. You are scored only on the last row (final layer). Every arithmetic op must go through flopscope.numpy (fnp) so it is metered; you have 2^{41} = 2,199,023,255,552 FLOPs per MLP (~65,000 forward passes). Score = final_layer_mse x max(0.1, FLOPs_used / 2^{41}), averaged over 100 MLPs (50 public + 50 holdout). Lower is better.

A3. Numbers to memorise

Quantity	Value
Shape / output	width 1024, depth 16 -> (16, 1024); read mlp.width / mlp.depth (validate probes width=4, depth=2)
Budget B	2**41 = 2,199,023,255,552 FLOPs per MLP; one forward pass ~33.6M FLOPs; (1024,1024)@(1024,1024) float32 = 2,146,435,072 -> B buys ~1,024 of them
Score	adjusted_final_layer_score = mean_m[final_layer_mse_m x max(0.1, F_m / B)]; floor 0.1 at 10% utilisation
Caps per MLP	FLOP budget; residual wall time 0.4 s; wall clock 120 s. Any breach: predictions zeroed AND multiplier forced to 1.0 (~0.91 for that MLP = 10x worse than a valid zeros answer)
setup()	5 s hard (import + setup) per run; runs 5-15 times per submission; breach = SETUP_TIMEOUT, whole submission fails
Memory / CPU	8 GB for your process (of a 64 GB instance; RLIMIT_AS); 4 GiB per array; your code runs on ONE physical core (2 vCPUs), the other 7 cores (14 vCPUs) run the flopscope backend + harness; CPU-only, no network
Grader watchdog	no MLP finished for 8 min OR 40 min total -> WATCHDOG_TIMEOUT for unfinished MLPs
Ground-truth floor	avg final-layer variance 0.0748 / 1e9 = 7.5e-11 MSE (irrelevant in practice); organizers' reference costs ~3.36e16 FLOPs per MLP = 15,000x your budget; MC-equivalent samples in budget: 65,374
Baselines (mini, raw final MSE)	zeros 0.9095 | random 0.6856 | mean-prop 2.216e-4 (8.66e7 FLOPs) | cov-prop 4.050e-6 (5.17e10 FLOPs = 2.35% B) | MC sampler floor ~1.2e-5 raw at 10% B
Leaderboard 20-21 Sep	#1 2.1e-9 adjusted (raw 1.64e-8, util 0.13); top-20 2.1e-9..4.8e-9, util 0.13-0.23; cov-prop baseline 3.93e-7; Overview counters: 2,653 graded Phase 2 submissions, 711 under a tenth of budget, 142 failed on some MLPs; 992 participants, 22,008 submissions overall
Environment	Python >= 3.10, uv; kit pins whestbench>=0.16.1,<0.17 + flopscope>=0.12.1,<0.13; grader runs whestbench 0.16.0 + flopscope[server]==0.12.0 (identical prices for everything the kit uses)
Dataset	hf://aicrowd/arc-whestbench-public-2026@v2-phase2 ; mini = 100 MLPs (~6.3-6.5 GB, default), full = 1000 (~63-65 GB); prepared Arrow fast path available since 30 Aug 2026 ; CC-BY-4.0
Contacts	arc-whestbench@aicrowd.com ; forum discourse.aicrowd.com/c/white-box-estimation-challenge-2026 ; GitHub Issues

A4. Prizes and how winners are decided (Overview page, transcribed 21 Sep 2026)

Prize	Phase 1	Phase 2 (current)
1st place	$25,000	**$50,000**
2nd place	$10,000	**$20,000**
3rd place	$5,000	**$10,000**
Algorithmic contribution (best technical contribution to mechanistic estimation; judged on score, algorithmic ideas and write-up quality)	$10,000	**$20,000**
Subtotal	$50,000	$100,000
Community contribution (discretionary, per contributor: explanations, algorithmic ideas, bug reports, community help)	$500 - $5,000	$500 - $5,000

- All amounts USD ARV. Sponsor may increase or add prizes (announced on the Competition Site). Tied ranks after disambiguation share the combined amounts.
- **Final ranking**: after 17 Oct each team nominates **up to two** valid Phase 2 submissions (selection deadline published near the end of the phase; with no nomination the two highest-ranked public submissions are used). They are re-run on a freshly generated private suite with private seeds; prize ranking comes exclusively from that re-run. Statistically indistinguishable leaders: more MLPs are generated. Private re-evaluation window 17 Oct - 7 Nov; **winner announcement 15 Nov 2026**.
- **Open-source obligation**: place-prize winners must release the prize-determining code and required artifacts under an OSI-approved licence within 30 days of notification and keep it public for 3 years (Rules section 6).

- **Teams**: up to five eligible individuals, frozen 2 Oct 2026 23:59 UTC. Daily cap 10 submissions per team.
- **Autoresearch policy**: LLM-assisted and agentic development is welcome; you remain responsible for compliance, attribution, reproducibility and write-up disclosures. Boundary = rule evasion: no automated registration or mass uploads, no tampering with flopscope, no reading private grader materials, no submitting work you cannot verify.
- Eligibility risks: non-compliant code invalidated even after grading; participant-chosen RNG seeds may cost prize eligibility; rules must be re-accepted at each phase (our account: rules_accepted = true, participation_terms_accepted = true as of 21 Sep).

Part B — Rules that can zero you or disqualify you

B1. Allowed code (exactly three things)

- **The grader's CPython interpreter**: bytecode, builtins, GC. Nothing you compile, link or launch.
- **The flopscope client API**: `import flopscope as flops`, `import flopscope.numpy as fnp` (plus whestbench contract types BaseEstimator, MLP, SetupContext). The only array path; every op billed analytically.
- **The pure-Python standard library** for control flow, bookkeeping, loading shipped files. Still prohibited even though stdlib: ctypes, threading, multiprocessing, subprocess, asyncio, concurrent.futures. A pure-Python loop over 1024 neurons is legal but unmetered: it burns residual time and doing real arithmetic that way is a violation.

B2. Prohibited (no price makes them allowed; disqualifiable; review continues after grading)

- Vendored numpy / scipy / BLAS (import numpy does not exist in the sandbox; no requirements.txt step; scipy/torch absent).
- Compiled kernels of any kind (.so/.dylib/.pyd/extension modules); ctypes / cffi / FFI.
- asyncio / threads / subprocess / multiprocessing, including 'only to overlap I/O'.
- Computing while a flopscope op is in flight (callbacks, hooks, lazy objects that run uncounted).
- Touching the flopscope client, transport or accounting (patching, wrapping, editing counts).
- **Value packing**: packing several independent values into one machine element (e.g. booleans into a wider int for bitwise ops) to reduce billed work — banned even if every op is metered and even though int8/int16/int32 share the 1.0 rate. Older flopscope docs calling it 'in-bounds' do not apply. Ordinary quantisation / lower-precision dtypes are not by themselves this technique, but must still satisfy the Rules.
- Meaningful numerical work in residual time, whether or not it fits under 400 ms.
- Reads outside SetupContext.submission_dir; any network call; time-based or participant-chosen RNG seeds.

B3. Permitted and encouraged

- **Data files**: weights, lookup tables, calibration constants, projection bases, precomputed artifacts, as pickle-free .npz (np.savez offline, or flopscope.Module.save()). Loaded in setup() with fnp.load(str(path)) / Weights.from_file(str(path)) at 0 FLOPs. torch.save / joblib / pickle will NOT load. A file that is really a program (bytecode, serialised kernel, instruction table you dispatch through) can be ruled not-data; ask first.
- Offline computation of anything (training on PyTorch, scipy routines) as long as only the resulting numbers ship.
- Any estimation method: white-box, black-box sampling, hybrid.
- Asking arc-whestbench@aicrowd.com before building on an uncertain technique, reporting suspected mispricing (send op, shapes, dtypes, ctx.summary() rows, expected vs observed flops_used), or requesting a residual-cap exception for legitimate plumbing.

B4. Enforcement

Automated checks on every submission -> agent-assisted validation of flagged ones -> human review of anything unresolved. A leaderboard position or a clean local run is NOT an eligibility ruling. In Phase 1 a real billing bug (fnp.tensordot with contracted rank > 52 billing multiplies only) was found, disclosed by the finder, and the affected entry invalidated.

B5. The four caps and what breaching each does

Cap	Value	Measured by	Consequence
FLOP budget	2**41 per MLP	flopscope, analytically (BudgetExhaustedError before the op runs); post-hoc backstop combined_budget_exhausted	that MLP zeroed, multiplier 1.0
Residual wall time	0.4 s per MLP	whestbench after predict() returns: wall - backend - overhead	that MLP zeroed, multiplier 1.0
Wall clock	120 s per predict()	flopscope TimeExhaustedError	that MLP zeroed, multiplier 1.0
setup()	5 s per run (import + setup), 5-15 runs per submission	grader	SETUP_TIMEOUT: WHOLE submission fails, slot spent
Memory	8 GB process; 4 GiB single array	RLIMIT_AS (Linux) / sandbox	worker dies and is replaced (re-running setup) or per-MLP 'result array too large'
Watchdog	8 min no progress / 40 min total	grader	unfinished MLPs = WATCHDOG_TIMEOUT (failed)

Residual-time killers found by participants: every float(), int(), bool() on a flopscope value, every data-dependent if/assert, and every Python-side scalar coefficient forces the client to synchronise with the backend (~0.02 ms per op plus the whole pending graph). The shipped cov-prop example tripped 400 ms locally for this reason; removing 166 sync points per MLP fixed it at identical FLOPs. Keep everything as flopscope objects until the final return. Also: a print() inside predict() under the subprocess runner writes into the IPC pipe and HANGS the run.

Part C — Tooling: install, iterate, evaluate, submit

C1. Install (once)

```
# needs Python >= 3.10 and uv (https://docs.astral.sh/uv/getting-started/installation/)
git clone https://github.com/AIcrowd/whest-starterkit.git
cd whest-starterkit
uv sync                          # installs whestbench 0.16.1 + flopscope 0.12.1 (pinned pair)
uv run whest doctor              # 6 checks: Python>=3.10, uv, install mode, BLAS pool, >=1 GiB disk, CWD writable (
uv run whest smoke-test          # built-in estimator end-to-end; proves the install
uv run whest version
# prefetch the 7 GB mini split into the HF cache (one-time, ~6.3 GB via the prepared/ Arrow path; HF_XET_HIGH_PERFOR
uv run whest dataset download aicrowd/arc-whestbench-public-2026 --revision v2-phase2 --split mini
uv run whest init my-sub         # optional: scaffold estimator.py + .whestignore in a fresh folder
```

Repo layout: estimator.py (your entry point; all-zeros template with a --baseline comparison loop), local_engine.py (single-file harness: build_mlp, monte_carlo_layer_means, compare_against_monte_carlo), examples/01-04, docs/ (tutorial, concepts, how-to, reference, troubleshooting, advanced), tests/ (drift gates that pin every FLOP figure in the docs), .whestignore, CHANGELOG.md (read before tuning; every rule/parameter/FLOP change is announced there), Makefile (install/test/lint). Related repos: AIcrowd/whestbench (harness + whest CLI + authoritative CLI reference), AIcrowd/flopscope (meter + cost-model reference), AIcrowd/whestbench-explorer (hosted visualiser).

C2. The five-stage ladder

Stage	Command	What it proves / notes
1 Iterate	uv run python estimator.py [--baseline mean_propagation|covariance_propagation|random|shipped_weights] [--width --depth --seed]	One fixed MLP (build_mlp seed 0); prints MC convergence table n=10..100,000 with sampling_flops, estimator_flops, all_layers_mse, final_layer_mse (both vs the MC estimate at that n, NOT vs truth; read the bottom row). Uses the real 2**41 cap. Does not check dtype.
2 Validate	uv run whest validate --estimator estimator.py [--seed N] [--class Name]	Probe MLP width=4 depth=2, budget=100 (not enforced): class resolved, setup() within 5 s, shape (depth, width), finite. READ THE CHECKS TABLE: a FAIL on the setup cap still prints Status: success and exits 0; --json prints no rows.
3 Run local	uv run whest run --estimator estimator.py --dataset hf://aicrowd/arc-whestbench-public-2026@v2-phase2 --split mini --runner local	Real scoring pipeline, 100 mini MLPs, baked 1e9 truth; in-process so breakpoint()/pdb work. Without --dataset: 10 fresh MLPs at 1024x16, truth from 200,000 local MC draws (~6 s/MLP), redrawn each run unless --seed.
4 Subprocess	... --runner subprocess	Same transport as the grader: catches imports that only worked in your shell, stdout writes (stdout is the IPC pipe: a print() hangs the run), values that don't survive the JSON round trip, and runaway memory on Linux (8 GB RLIMIT_AS; on macOS the cap is silently NOT applied). Does NOT reset state between MLPs (one worker serves the suite). Score must equal Stage 3 exactly.
5 Package/submit	uv run whest package --estimator estimator.py --output submission.tar.gz ; uv run whest validate-package submission.tar.gz ; uv run whest login ; uv run whest submit --estimator estimator.py --watch	Single file ships only that file (+manifest.json). Folder mode (--estimator .) ships everything non-ignored and needs estimator.py at the root. --dry-run previews without uploading. Browser upload of a whest-built tarball also works.

C3. whest run flags you will actually use

Flag	Default / when to use
--dataset hf://...@v2-phase2 [--split mini|full] [--revision TAG] [--streaming]	Baked truth. mini is the default split. --streaming iterates from HF without caching (debug only). Always pin @v2-phase2.
--n-mlps N	10 without --dataset; whole split with it (100/1000). Use 3 while iterating, 20+ to reduce noise.
--n-samples N	200,000 (dataset-less only). --n-samples 5000 gives a ~9x faster sanity run (12.9 s -> 1.4 s per MLP); ignored with --dataset.
--seed N	Deterministic MLPs and ctx.seed; with --dataset controls ctx.seed only.
--flop-budget N	2199023255552. Raise to 1e13 to check an idea is not budget-starved before optimising; lower to 272000000000 / 68000000000 only to replay Phase 1 / warm-up (together with --lambda-flops-per-second 1e11 --no-residual-wall-time-limit --wall-time-limit 60 and the old dataset tag).
--wall-time-limit S / --residual-wall-time-limit S / --setup-timeout S	120 / 0.4 / 5.0 = graded values already. Pass only to be STRICTER (e.g. --setup-timeout 2.5, --residual-wall-time-limit 0.2) to leave headroom for a slower grader.
--runner local|subprocess	local (default, debuggable) vs isolated worker (grader transport). 'server' is a legacy alias.
--profile	Per-namespace FLOP and time breakdown; read residual_wall_time_s per MLP and the operations ranking.
--show-diagnostic-plots	Inline convergence and per-layer error plots.
--max-threads 1	Pin BLAS threads: pessimistic wall-clock rehearsal (grader gives you 2 vCPUs).
--format rich|plain|json (--json alias), --detail raw|full	json for machine-readable reports (run_config shows the limits used); plain when using pdb.set_trace (or use breakpoint(), which the CLI handles); full adds per-MLP raw arrays.
--debug, --fail-fast, --class Name	Tracebacks in the Estimator Errors panel; stop at first failure; select the estimator class if several exist.
WHEST_SKIP_HARDWARE_FALLBACK_PROBES=1	Env var: skip slow hardware metadata probes locally.

C4. Reading the score report (whest run --format json)

- results.adjusted_final_layer_score (ranked), final_layer_mse, all_layers_mse, **per_layer_mse** (list of 16; a rising tail means error compounds with depth — your most useful diagnostic), best/worst_mlp_adjusted_final_layer_score, mean_score_multiplier [0.1, 1.0], mean_compute_utilization (unclamped), mean_effective_compute (= mean flops_used), n_failed_mlps, failure_breakdown {budget_exhausted, time_exhausted, residual_wall_time_exhausted, combined_budget_exhausted, error}.
- per_mlp[i]: mlp_index, mlp_name (stable slug, e.g. 'megan-chang'), flops_used, effective_compute, adjusted_final_layer_score, the four failure flags, wall_time_s = flopscope_backend_time_s + flopscope_overhead_time_s + residual_wall_time_s, final_layer_mse, all_layers_mse, per_layer_mse, breakdowns.estimator.by_namespace[...].operations, traceback (null on a clean run), error / error_code (PREDICT_ERROR under subprocess, exception class under local; branch on flags, not on this string).
- run_config: seed, dataset {path, sha256 of metadata.json, n_mlps, seed_protocol 3.0/4.0}, n_mlps, width, depth, flop_budget, setup_timeout_s, wall_time_limit_s, residual_wall_time_limit_s, lambda_flops_per_second (0.0). breakdowns.sampling.time_source = 'bake' for dataset runs.
- High flopscope_overhead_time_s = many tiny ops (batch them); high residual_wall_time_s = your Python is the bottleneck (vectorise, remove syncs).

```
# rank where your FLOPs went (from a --format json report saved as report.json)
uv run python -c "import json; ops = json.load(open('report.json'))['results']['breakdowns']['estimator']['by_namesp

# profile_me.py: one predict() under the real budget with a per-op summary
import flopscope as flops, local_engine
from estimator import Estimator
B_M = 2_199_023_255_552
mlp = local_engine.build_mlp(width=1024, depth=16, seed=0)
with flops.BudgetContext(flop_budget=B_M) as budget:
    result = Estimator().predict(mlp, B_M)
    print(budget.summary(by_namespace=True)); print(budget.flops_used, budget.residual_wall_time_s)
```

C5. Dataset facts (verified on the Hugging Face Hub, 21 Sep 2026)

Repo: **aicrowd/arc-whestbench-public-2026** (CC-BY-4.0, ~20,400 downloads/month, last modified 30 Aug 2026). Git refs: tags v1-warmup, v1-phase1, **v2-phase2**; branch main currently = Phase 2 content (set 21 Aug 2026). main advances each phase, so always pin **revision='v2-phase2'** / **hf://...@v2-phase2**. Two HF configs: **default** (split mini, data/mini-*.parquet) and **full** (split full, data/full-*.parquet); loading one never pulls the other. mini is NOT a subset of full (disjoint seeds; verify via mlp_seed).

Revision	Round	Shape	MLPs	Fwd-pass FLOPs	Budget	MC samples affordable	Created
v2-phase2	Phase 2 (current)	1024x16	1,100	33,554,432	2^41	65,536	2026-08-13 15:11 UTC
v1-phase1	Phase 1	256x32	1,100	4,194,304	2.72e11	~64,850	2026-06-16
v1-warmup	Warm-up	256x8	1,100	1,048,576	6.8e10	~64,850	2026-05-26

What is on the Hub at v2-phase2	Size
data/mini-0000x-of-00007.parquet (7 shards)	6.54 GiB
data/full-000xx-of-00067.parquet (63 shards listed)	65.2 GiB
prepared/mini/ (14 Arrow shards + dataset_info.json + state.json; save_to_disk format)	6.26 GiB
prepared/full/ (135 Arrow shards)	62.6 GiB
README.md (dataset card), metadata.json (schema 3.0, seed protocol 3.0, n_samples 1e9, width 1024, depth 16, default_split mini, hardware_fingerprints, prepared_splits)	19 KB / 8.6 KB

Correction to the starter-kit doc: metadata.json at v2-phase2 DOES declare prepared_splits (commits of 29-30 Aug 2026 added prepared/mini and prepared/full). whestbench.load_dataset / whest run therefore download only prepared/<split>/ and memory-map it, skipping the parquet-to-arrow conversion. Plain datasets.load_dataset(...) still takes the parquet path. whest dataset download without --split fetches everything (~140 GB with both formats); always pass --split mini.

Schema (8 columns, one row per MLP)

Column	Type	Meaning
mlp_id	int32	0-based absolute index within the dataset
mlp_name	string	deterministic slug derived from mlp_seed (e.g. 'danielle-johnson'); carries no extra information
mlp_seed	int64	the MLP's INPUT seed (seed protocol 3.0). MLP.seed seen by your estimator is derived locally from it (deterministic). Under legacy protocol 2.0 it was the derived seed itself.
weights	float32[16,1024,1024]	i.i.d. N(0, 2/width) He init, NO biases, NO separate linear output layer: every matrix is followed by ReLU. Layer l: h_l = max(0, h_{l-1} @ W_l) with h_0 = x (row vector on the LEFT; equivalently max(0, W_l^T h_{l-1}) in column form).
all_layer_means	float32[16,1024]	GROUND TRUTH: [l, j] = mean of neuron j's post-ReLU output at layer l over N = 1e9 inputs. This is what you predict.
final_means	float32[1024]	last row of all_layer_means, materialised separately because the ranked metric only looks at it
avg_variance	float64	(1/width) sum_j Var[h_16(x)_j] over the same draws. Diagnostic only; useful to normalise your MSE (variance-normalised MSE as in the paper) or as input to variance-aware estimators. Not used by the score.
sampling_budget_breakdown	string (JSON)	FLOP accounting of the bake for this row: flop_budget (= flops_used; unlimited bake), flops_used, flops_remaining 0, wall_time_s, backend/overhead/residual times, time_source, by_namespace {operations {flop_cost, calls, ...}}, provenance {chunk_size, n_chunks, ...}. Unrelated to your evaluation budget.

Ground truth provenance

- N = 1,000,000,000 fresh x ~ N(0, I) per MLP; one pass through all 16 layers per sample, so all layers share the same draws. Sums accumulated in float64, divided by N, downcast to float32. avg_variance = E[h^2] - E[h]^2 over the same draws.

- Per-neuron std of the published mean = sqrt(Var[h_j]/N); ground-truth MSE = avg_variance / N (mini average 0.0748 -> 7.5e-11).
- Baked on torch 2.4.1+cu124 across a fleet of NVIDIA RTX 3090 (cc 8.6), Python 3.11.10, NumPy 2.4.6, whestbench 0.16.0, flopscope 0.12.0, deterministic config (use_deterministic_algorithms, cudnn.deterministic, cudnn.benchmark off, CUBLAS_WORKSPACE_CONFIG=:4096:8). 1,100 partial bakes merged with whest dataset merge. Bit-exact re-bake requires the same torch version, GPU architecture and the per-row provenance.chunk_size (float64 accumulation order); otherwise means are statistically equivalent (differences ~ sqrt(2 avg_variance/N) ~ 1e-5).
- Forward pass = 2 x depth x width^2 FLOPs exactly, in float32, and stable across flopscope releases; a real sampler pays another 0.4-1% for RNG, casts and the running mean.

Loading APIs

```
from datasets import load_dataset                                    # pure HF path (no whestbench needed)
mini = load_dataset("aicrowd/arc-whestbench-public-2026", revision="v2-phase2", split="mini").with_format("numpy")
full = load_dataset("aicrowd/arc-whestbench-public-2026", "full", revision="v2-phase2", split="full")
row = mini[0]; row["weights"].shape          # (16, 1024, 1024) float32 ; without .with_format('numpy') -> python li

import whestbench                                                     # convenience wrapper: schema validation + prov
ds = whestbench.load_dataset("aicrowd/arc-whestbench-public-2026", revision="v2-phase2", split="mini")
for mlp in whestbench.iter_mlps(ds):                                  # whestbench.MLP with .weights .seed .name .wid
    ...
meta = whestbench.metadata(ds); meta["n_samples"], meta["created_at_utc"]

# re-bake from published seeds (weights exact; means bit-exact only with matching --chunk-size from provenance)
import json; json.dump([int(r["mlp_seed"]) for r in ds], open("mini-seeds.json", "w"))
whest dataset bake --n-mlps 100 --n-samples 1000000000 --width 1024 --depth 16 --split mini --mlp-seeds mini-seeds.j
```

- Grader suite: 100 MLPs (50 public drive the visible leaderboard + 50 holdout), identical for every submission and NOT the public mini/full rows (different seeds). The final prize re-run uses a fresh private seed, so **never fit constants on public leaderboard feedback**; fit on mini/full or your own bakes and validate on the other.
- Use avg_variance to report variance-normalised MSE (MSE / avg_variance) in your write-up: it is the metric the paper uses and makes MLPs comparable.
- Baking your own smaller datasets for fast iteration: whest dataset bake --output DIR --n-mlps N --n-samples N --width 1024 --depth 16 [--mlp-seeds seeds.json] [--torch --device cuda --mlps-per-batch --chunk-size --compile]; shard with --slice K/N or --mlp-range A-B and merge with whest dataset merge; ~80 s and 670 MB for 10 MLPs at 100k samples on CPU; --device does nothing without --torch; whest dataset info DIR inspects a bake.
- Citation: @misc{whestbench2026, title={WhestBench 2026: ARC White-Box Estimation Challenge}, author={Alignment Research Center and AIcrowd}, year={2026}, howpublished={aicrowd.com/challenges/arc-white-box-estimation-challenge-2026}}.

C6. Packaging and .whestignore

- Single-file mode ships ONLY that file (renamed estimator.py) + manifest.json (entrypoint, whestbench/flopscope/numpy/Python versions, per-file SHA-256, timestamp). It refuses if the file imports a sibling module (PACKAGING_VALIDATION_ERROR); data files beside it are silently dropped -> folder mode.
- Folder mode: every non-ignored file, listed with sizes and confirmed [y/N] (--yes / -y in scripts). Requires estimator.py at the root (also fails if a .gitignore/.whestignore pattern matches it). Credential files (.env, *.pem, *.key, *.p12, *.pfx, id_rsa*, .netrc, .pypirc, .npmrc, .aws, .ssh, .gnupg, credentials*) never ship. Everything else does — the leaderboard is public.
- .gitignore and .whestignore are read as ONE pattern list with no '!' negation: a .gitignore rule matching weights.npz drops it silently. The kit's .whestignore excludes .git, .github, .venv, caches, local_engine.py, examples/, tests/, docs/, assets/, scripts/, Makefile, README.md, pyproject.toml, uv.lock (otherwise 64 files > 50-file cap).
- Caps: 50 MiB total (CLI says ~52 MB), 50 files. One (1024,1024) float32 = 4 MiB; ship rank-k factors / projection bases, not full matrices per layer.

- Never hand-roll the tarball (manifest / SHA mismatch -> IMPORT_FAILED before grading). whest validate-package checks a built archive.
- After upload: sandbox unpack, setup() once per worker (5-15x), predict() per MLP in parallel, leaderboard updates; local Stage 4 score should match within ~1-2%.

Part D — Scoring and the compute meter

D1. The exact formula

```
adjusted_final_layer_score = (1/M) SUM_m  final_layer_mse_m * max(0.1, F_m / B)        (M = 100 on the grader)
final_layer_mse_m          = (1/1024) SUM_i (pred_m[15, i] - truth_m[15, i])^2        (last row only)
all_layers_mse, per_layer_mse                                                         diagnostics, no multiplier
F_m = flopscope's analytical FLOP count inside predict()   (Phase 1 added lambda*R_m with lambda = 1e11; Phase 2: la
Failure of any cap:  pred_m := 0, multiplier := 1.0  ->  ~0.9095 for that MLP
```

D2. What the formula implies for strategy

- Below 10% of B the multiplier is pinned at 0.1: the board is a pure final-layer-MSE ranking (score = MSE/10). All bundled examples sit here.
- For a plain sampler MSE ~ 1/k and cost ~ k, so above the floor MSE x util is constant: a pure Monte Carlo estimator bottoms out at ~1.2e-6 adjusted no matter how much budget it spends (measured 1.19e-6 at 6,500 passes, 1.16e-6 at 10,000; a forum test at 0.20-0.45 B stayed at 1.337e-6). More samples cannot win.
- Spending above 10% is only worth it if each extra FLOP buys MORE than proportional accuracy. The whole top-20 sits at 13-23% utilisation with raw MSE ~2e-8: their extra compute (third-cumulant propagation) is super-linear in accuracy. A team with 13% worse MSE but 6.5% utilisation beat a 40%-utilisation sampler by 3.6x.
- One failed MLP out of 100 adds ~0.009 to the mean — a million times the leaders' score. **Zero failures is mandatory.** Guard NaN/Inf, shapes, residual time and memory before chasing accuracy.
- Worked example: truth [0.42,0.38,0.51], pred [0.40,0.35,0.55] -> MSE 0.000967; at 30% budget adjusted = 0.000290; at <=10% adjusted = 0.0000967.

D3. flopscope billing

```
charged = flop_cost * weight * dtype_rate * complex_factor
  weight {0,1,4,16};  dtype_rate 1.0 (<=32-bit), 2.0 (float64/int64), 4.0 (float128);  NumPy promotion decides the b
```

Tier	Ops and cost at float32
FREE	fnp.zeros, fnp.empty, views (.T, x[0], x[:,3]), fnp.diag/diagonal, no-copy fnp.asarray on a flopscope array, default_rng(seed) construction, fnp.load
1 / element	+ - * / comparisons, maximum (ReLU), sqrt, abs; fills ones/full/eye (eye: diagonal only), fnp.array and copying asarray, .copy(), .astype() (destination dtype rate), reshape/ravel (even where numpy views), stack, concatenate, tile, repeat, outer (n x m), fill_diagonal (n)
Reductions	sum/max/min/prod/all/any/argmax/cumsum N-1; mean/median/average N; var 4N; std 4N+1; nan* add an N scan (except nanmax/nanmin)
4 / element	gathers (take, fancy int indexing), 3-arg where, ix_; sorts/argsort/unique/searchsorted ~4 N ceil(log2 N)
16 / element	exp, log, trig, x ** y including x ** 2 (write x * x)
Matmul	@ / matmul / einsum: M N (2K-1). (1024,1024)@(1024,1024) = 2,146,435,072. Symmetric-tagged einsum sandwich 'ij,ia,jb->ab' = 3,220,700,672 per layer vs 4,292,870,144 untagged (-25%)
Samplers	standard_normal 16/elem float32, 32/elem float64 (default); uniform 6/elem (always float64)
Composites	flops.stats.norm.pdf ~54/elem, .cdf ~96/elem (both promote to float64: cast back); hand-rolled pdf ~20, A&S cdf ~48
Symmetry	flops.as_symmetric(cov, symmetry=(0,1)) = 7n-1 per element pass (7,340,031 at n=1024^2 float32); any write into a tagged buffer voids the tag SILENTLY; flops.symmetrize(mode='canonical-copy') also exists
Egress / manual	fnp.save/savez inside predict ~4/elem x rate; ctx.deduct('name', flop_cost=..., subscripts=None, shapes=(), dtypes=(fnp.float32,))

D4. Practical meter rules

- Seed all state at float32 (fnp.zeros(..., dtype=fnp.float32)); mlp.weights already are (re-casting bills 1,048,576/layer for nothing). Measured savings: mean-prop 153,913,344 -> 86,639,616 (43.7%), cov-prop 103,407,495,614 -> 51,709,240,799 (49.99%), MSE unchanged to 5 s.f.
- Cast flops.stats.norm.* results back with .astype(fnp.float32) (2,048 FLOPs) or the float64 promotion re-infects the loop.
- Avoid chained w.T @ cov @ w (symmetry lost, SymmetryLossWarning); use the single einsum over a tagged cov and re-tag after fill_diagonal.
- Mean-prop FLOP walkthrough (86,639,616 total): matmul 67,076,096 (77.42%), multiply 16,891,904 (19.5%), norm.cdf 1,572,864, norm.pdf 884,736, astype 65,536, add 49,152, maximum 32,768, sqrt/divide/subtract/stack 16,384 each, ones 1,024, zeros 0. Cov-prop: 16 einsums 51,531,210,752 (99.66%), 17 as_symmetric 124,780,527, everything else negligible.
- flopscope arrays are immutable on the grader: no arr[i]=v, no += ; use fnp.where / concatenate / arr = arr + x; fill_diagonal(M, v) as a statement is the one in-place op.
- Only numeric dtypes (kind in 'biufc'); object/str/datetime raise UnsupportedDtypeError (flopscope.errors). Reduction axis must be int/tuple. flops.accounting.* cost helpers price at float32.
- Memory is not metered but capped: 65,000 x 1024 float32 = ~270 MB per activation tensor; a full (1024,1024) covariance is 4 MiB (16 MiB float64); count what you hold simultaneously against 8 GB and 4 GiB/array.
- Meter history (why old numbers differ): 0.9 four-factor billing (dtype 2x, transcendentals 16x, data movement billed); 0.10 symmetry tags voided on write, out= priced; 0.11 >52-letter einsum bills full FMA, ix_ billed, non-numeric dtypes raise; 0.12 ~40 repricings (nan* scans, boolean-answer ops 2x, NEP 50 weak promotion for scalars); 0.12.1 full/full_like with array fill lose inferred symmetry.

Part E — Estimator API and reference code

E1. Contract

```
from __future__ import annotations
import flopscope as flops
import flopscope.numpy as fnp
from whestbench import BaseEstimator, SetupContext
from whestbench.domain import MLP          # or: from whestbench import MLP

class Estimator(BaseEstimator):            # exactly one BaseEstimator subclass in the entrypoint (or pass --class)
    def __init__(self) -> None:            # cheap; no I/O, no compute
        self._weights = None
    def setup(self, ctx: SetupContext) -> None:     # optional; per worker; off-budget; 5 s cap; idempotent
        self._setup_rng = fnp.random.default_rng(ctx.seed)
        if ctx.submission_dir is not None:          # guard None; never touch ctx.scratch_dir (None everywhere, rais
            path = Path(ctx.submission_dir) / "weights.npz"
            if path.exists(): self._weights = Weights.from_file(str(path))   # str, not Path; 0 FLOPs
    def predict(self, mlp: MLP, budget: int) -> fnp.ndarray:
        rng = fnp.random.default_rng(mlp.seed)      # per-MLP grader seed; sub-streams: master.bit_generator.spawn(k
        ...
        return fnp.stack(rows, axis=0)              # (mlp.depth, mlp.width), finite; coerced to float32
    def teardown(self) -> None: ...                 # optional; per worker; skipped if the worker died
```

Field	Meaning
mlp.width / mlp.depth / mlp.weights / mlp.seed / mlp.name	1024 / 16 / list of 16 float32 (1024,1024) / per-MLP int seed / slug (" outside an evaluator)
ctx.width / depth / flop_budget / api_version	1024 / 16 / 2**41 / contract version
ctx.submission_dir	folder your submission was extracted into (locally: your estimator's folder)
ctx.scratch_dir	RESERVED; None on every 0.16.0 path; using it raises in setup() and fails the whole submission
ctx.seed	run-level seed, same for every MLP, 0 if unset; use default_rng(ctx.seed), never fnp.random.seed(...)
budget (predict arg)	2**41 in Stages 1/3/4 and on the grader; 100 in validate (not enforced). Enforcement happens whether you read it or not.

E2. setup() and state on the grader

- setup() runs once per worker process; a submission is served by several workers and a crashed worker is replaced and re-runs setup(): ~5-15 runs, each with its own 5 s (import + setup). Its FLOPs and wall time are billed to nobody.
- Per-MLP weights do not exist in setup(), so it can only load MLP-independent artifacts or draw a ctx.seed-based random projection. Heavy precompute -> offline -> .npz.
- Each worker sees only some MLPs, so cross-MLP caches that populate locally are empty in production. Locally (both runners) one worker serves the whole suite and state persists; reset per-MLP state at the top of predict().
- Predictions cross the worker pipe as JSON and are re-read with fnp.asarray(..., dtype=fnp.float32); return a flopscope ndarray of the right shape, nothing else. The smoke test uses non-standard shapes (a participant lost 7 uploads to 16-indexed tables): guard layer-indexed constants with mlp.depth / mlp.width.

E3. Failure semantics

Failure	Report evidence	Effect
wrong shape / no .shape / NaN / Inf	error.details {expected_shape, got_shape, cause_hints}	MLP zeroed x1.0
predict() raised	error_code PREDICT_ERROR (+traceback); CLI exit 1; WORKER_EOF / WORKER_BROKEN_PIPE / WORKER_IO_ERROR / PREDICT_TIMEOUT if the worker died	MLP zeroed x1.0, run continues
FLOP budget	budget_exhausted true (BudgetExhaustedError before the op); combined_budget_exhausted is the post-hoc backstop	MLP zeroed x1.0
wall 120 s / residual 0.4 s	time_exhausted / residual_wall_time_exhausted true (locally a ResidualWallTimeExhaustionWarning)	MLP zeroed x1.0
setup() raised / > 5 s	SETUP_ERROR (local) / SETUP_FAILED / SETUP_TIMEOUT	whole submission rejected
> 4 GiB array / IMPORT_FAILED	'result array too large' ; manifest or sha256 mismatch	per-MLP or pre-grading rejection

E4. Bundled examples (verbatim logic)

02_mean_propagation.py — 86,639,616 FLOPs, final MSE 2.216e-4

```
mu = fnp.zeros(width, dtype=fnp.float32); var = fnp.ones(width, dtype=fnp.float32)
rows = []
for w in mlp.weights:
    mu_pre  = w.T @ mu
    var_pre = fnp.maximum((w * w).T @ var, 1e-12)          # diagonal only: var_pre[i] = sum_j W[j,i]^2 var[j]
    sigma_pre = fnp.sqrt(var_pre); alpha = mu_pre / sigma_pre
    phi_a = flops.stats.norm.pdf(alpha).astype(fnp.float32); Phi_a = flops.stats.norm.cdf(alpha).astype(fnp.float32)
    mu  = mu_pre * Phi_a + sigma_pre * phi_a                                    # E[ReLU]
    ez2 = (mu_pre * mu_pre + var_pre) * Phi_a + mu_pre * sigma_pre * phi_a      # E[ReLU^2]
    var = fnp.maximum(ez2 - mu * mu, 0.0)
    rows.append(mu)
return fnp.stack(rows, axis=0)
```

03_covariance_propagation.py — 51,709,240,799 FLOPs (2.351% B), final MSE 4.050e-6

```
mu = fnp.zeros(width, dtype=fnp.float32)
cov = flops.as_symmetric(fnp.eye(width, dtype=fnp.float32), symmetry=(0, 1)); log_scale = 0.0
for w in mlp.weights:
    max_var = float(fnp.max(fnp.diag(cov)))                  # overflow guard; never fires at 1024x16 (diag stays 0.
    if max_var > 1e30: s = float(fnp.sqrt(max_var)); mu = mu / s; cov = cov / (s*s); log_scale += float(fnp.log(s))
    mu_pre  = w.T @ mu
    cov_pre = fnp.einsum("ij,ia,jb->ab", cov, w, w)          # W^T cov W, one contraction, symmetric rate
    var_pre = fnp.maximum(fnp.diag(cov_pre), 1e-12); sigma_pre = fnp.sqrt(var_pre); alpha = mu_pre / sigma_pre
    phi_a = flops.stats.norm.pdf(alpha).astype(fnp.float32); Phi_a = flops.stats.norm.cdf(alpha).astype(fnp.float32)
    mu  = mu_pre * Phi_a + sigma_pre * phi_a
    ez2 = (mu_pre*mu_pre + var_pre) * Phi_a + mu_pre * sigma_pre * phi_a
    var_post = fnp.maximum(ez2 - mu * mu, 0.0)
    gain = fnp.where(sigma_pre > 1e-12, Phi_a, fnp.zeros((), dtype=fnp.float32))   # explicit f32 zero (NEP 50 stabi
    cov  = fnp.multiply(fnp.outer(gain, gain), cov_pre)      # cov_post[i,j] ~ gain_i gain_j cov_pre[i,j]  (the 'gai
    fnp.fill_diagonal(cov, var_post)                         # exact marginal variance; voids the tag
    cov  = flops.as_symmetric(cov, symmetry=(0, 1))          # re-tag: 7.3M FLOPs, saves 1.07e9 on the next einsum
    rows.append(mu * float(fnp.exp(log_scale)))
return fnp.stack(rows, axis=0)
# NOTE: the float()/if per layer are client sync points that cost residual time; the fix accepted upstream keeps the
```

04_shipped_weights.py — the data-file pattern

```
class Weights(flopscope.Module):                     # public array attributes are saved/restored; _private ones ski
    def __init__(self): self.scale = fnp.zeros((), dtype=fnp.float32)   # zeros is free; ones(()) at float64 bills 2
w = Weights(); w.scale = fnp.asarray(2.0); w.save("weights.npz")          # OFFLINE, plain .npz, no pickle
# in setup(): self._weights = Weights.from_file(str(path))               # 0 FLOPs; from_file re-runs __init__ then
# package the FOLDER: uv run whest package --estimator .   (ships estimator.py, weights.npz, manifest.json)
```

01_random.py — the seeding scaffold

```
def setup(self, ctx): self._setup_rng = fnp.random.default_rng(ctx.seed)
def predict(self, mlp, budget):
    rng = fnp.random.default_rng(mlp.seed)
    return fnp.asarray(rng.uniform(0.0, 1.0, size=(mlp.depth, mlp.width)).astype(fnp.float32))
```

local_engine.py (Stage 1 only, not shipped)

build_mlp(width, depth, seed=0) -> He-init MLP, statistically equivalent to whestbench.sample_mlp. monte_carlo_layer_means(mlp, n_samples, seed=0) -> (depth, width) float64-accumulated means (33,637,376 FLOPs per pass). compare_against_monte_carlo(estimator, mlp, sample_counts=(10,100,1_000,10_000,100_000), estimator_budget=2**41, sampling_budget=5e12 per row, seed=0, submission_dir=...) calls setup() then predict() once and prints the table; preflight checks shape/type (SystemExit 2) but not dtype. Sampling cost table: 10 -> 336,439,296; 100 -> 3.36e9; 1,000 -> 3.36e10 (0.03 s); 10,000 -> 3.36e11 (0.25 s); 100,000 -> 3.36e12 (2.68 s, more than a whole budget).

Part F — Mathematics and algorithm directions

F1. Rectified-Gaussian moments

```
z ~ N(mu, sigma^2), alpha = mu/sigma, phi = N(0,1) pdf, Phi = N(0,1) cdf:
  E[ReLU z]   = mu Phi(alpha) + sigma phi(alpha)
  E[ReLU z^2] = (mu^2 + sigma^2) Phi(alpha) + mu sigma phi(alpha)
  Var         = E[z^2] - E[z]^2                              (Frey & Hinton 1999; Williams 1998)
Exact only when z is Gaussian: layer 1 exactly, later layers by CLT approximation. Degrades for small width (<~32),
large depth (>~32 layers), alpha ~ 0 (small mu, relative error spikes). 1024x16 is a friendly regime for the first t
Pure-flopscope pdf: exp(-0.5 x x)/sqrt(2 pi).  cdf (A&S 7.1.26, |err|<7.5e-8): t = 1/(1+0.2316419|x|),
  poly = ((((1.330274429 t - 1.821255978) t + 1.781477937) t - 0.356563782) t + 0.319381530) t, cdf = 1 - pdf*poly,
```

F2. Why the bundled closures stall, and what the frontier does

- **Mean propagation** assumes independent neurons: 2.2e-4. Unit-gain feedback (dM1/dmu ~ 1) means per-layer errors pass through unchanged.
- **Gain covariance** keeps correlations but (a) uses the approximate off-diagonal closure cov_post ~ g_i g_j cov_pre and (b) assumes Gaussian pre-activations: 4.05e-6. The exact bivariate ReLU covariance (Price's theorem + Plackett's identity -> 1-D Gauss-Legendre; or the arc-cosine kernel for zero mean) fixes (a) but measured only 7.4% better. The dominant remaining error is accumulated **non-Gaussianity** (third cumulant) of pre-activations, which second-moment methods cannot see.
- **Cumulant propagation (Wu et al., arXiv:2605.05179)**: track cumulants up to order K through each layer using Hermite expansions of ReLU and a 'diagram summation formula'; refinements: power cumulants (Hermite expansions of ReLU^a) and full-trace terms; factorized variants store order-K cumulants in O(n^(K-1)). Theory: error O((L/n)^K); factorized cost O(n^K L^2). Reference K=3 at 1024x16 would need ~3.7 B; the top teams run factorized/approximate versions at 0.13-0.25 B with raw MSE ~2e-8 (200x better than cov-prop).
- **Team 504aldo's factorized K=3** (rank ~8, 5.4e-9 adjusted, 0.2526 B): third cumulant = sum of 'sources' born at each ReLU and transported through later layers in factored form; 86% of FLOPs in those sources (young sources <=4 layers old, old sources merged into a shared subspace); bottleneck = Hadamard products of factored legs need dense n x n formations (Khatri-Rao rank r^2). Constants (16-entry lambda table, age gates 4 and 7, ranks 384 and 224) fit on 8 public MLPs only. Leaders appear to carry old-source content at young-source cost, i.e. a different representation.
- **Cheap proven wins**: exact-formula and calibration tweaks (1.3e-7 adjusted); a fixed 0.75/0.25 blend of gain-covariance propagation with a whitened antithetic MC branch (branch errors r = 0.038 -> 24.85% MSE reduction at 9.3% B); per-layer ridge calibration fitted on held-out LOCAL MLPs and applied only to recorded outputs (feeding it back into propagation was 69x worse).

F3. Measured dead ends (do not repeat)

Idea	Result
Pure MC / QMC / Sobol / antithetic / control variates as the main estimator	~1.2e-6 adjusted floor, budget-independent; Sobol at 1,600-3,600 points in 1024-D behaves like plain MC; control variates need 33-300x variance reduction, got ~1.5x
Dropping old third-cumulant sources / memoryless closure / Phase-1 closure	1.0e-6 / 9.1e-7 / 9.8e-7 raw
Gaussian scale-mixture; marginal Edgeworth / Gram-Charlier	2.3e-6; capped at 4-5e-6 (order-2 bases cannot see cross-neuron structure)
CP-rank capping, Tucker confinement, per-source rank ladders, age thresholds	needs 8n columns for 2% accuracy; 14-31x worse; 17x accuracy per 1.8x compute; cliffs at age 3 and 7
Dense kappa4 legs	>= 0.95 B; a 'regeneration law' recovers 90% of the value for free
Learned output corrections from cheap features	features < 0.09 correlated with the error; local patches / weight-feature models / bias correctors all die held-out; the usual 64 per-layer calibration constants absorb accumulated drift, not local skewness
float64 everywhere	+-0.004% (noise) for 2x the FLOPs
flopscope accounting tricks	billing is tight; no levers (and packing is banned)
Pricing polish of small ops	<= 4% FLOP savings but +74-118 ms residual time
Naive Kerdock MUB cubature at this shape	collapses to a spherical 3-design; Gaussian (Victoir) cubature needs 131,585 nodes > budget
Literature search (~25 papers)	annealed-weight results have kappa3 = 0 by construction; quenched results stop at covariance

F4. Organizer-listed open directions

- Low-rank covariance cov ~ U U^T (U: width x k), cost O(depth width^2 k); choose k from early-layer spectra.
- Layer-adaptive routing: full covariance where correlations build, diagonal after; locate the crossover from per_layer_mse.
- Spectral / weight-statistics methods (SVD of each W inside predict; billed): Pennington & Worah 2017, Saxe et al. 2014.
- Importance sampling toward high-variance regions of deep activations.
- Higher-order moments per neuron (skewness, kurtosis) to correct the Gaussian assumption: Schoenholz et al. 2017; Wu et al. 2026.
- Reading list: ARC 'Competing with sampling' (Matching Sampling Principle: a mechanistic estimator should reach eps^2-variance error in O(1/eps^2) forward-pass-equivalents); ARC 'AlgZoo'; Frey & Hinton 1999; Schoenholz, Gilmer, Ganguli, Sohl-Dickstein arXiv:1611.01232; Saxe, McClelland, Ganguli arXiv:1312.6120; Price's theorem; arc-cosine kernels (Cho & Saul).

Part G — Phase 2 discussion digest (all 28 forum threads read in full, 21 Sep 2026)

Challenge page counters on 21 Sep 2026: **69.2k views, 992 participants, 22,008 submissions**. Forum category: discourse.aicrowd.com/c/white-box-estimation-challenge-2026. Threads created after 18 Aug 2026 (the Phase 2 window) plus the rules/mechanics threads they reference are digested here; Phase 1 write-ups are covered only where they transfer.

G1. Official announcements and organizer clarifications (mohanty / aicrowd_team)

Thread	What the organizers said
Phase 2 launch (#18197, 23 Aug)	Submissions close 17 Oct 2026 23:59 UTC; team freeze 2 Oct; write-up 24 Oct. Table of changes (256x32 -> 1024x16; 2.72e11 -> 2^41; 60 s -> 120 s; computation outside flopscope: charged -> PROHIBITED; residual: priced -> capped 0.4 s; 50 -> 10 submissions/day; memory: none -> 8 GB; dataset v1-phase1 -> v2-phase2). Must re-accept the updated Rules before submitting. Prohibited list is 'not exhaustive': any mechanism that performs computation flopscope does not count is equally prohibited. 'Submissions with unusual residual wall-time usage receive additional scrutiny.' Rationale for the shape: 'best-performing methods in Phase 1 tended to be based on variations of sampling; we made the network wider and shallower to make cumulant-propagation-style methods more competitive; 16 layers is deep enough that handling depth-dependence well is a challenge.' 16.8M parameters vs 2.1M. First week: failed submissions free (up to 10 extra/day). Required: pip install --upgrade flopscope>=0.12.0 whestbench>=0.16.0 and pull the latest kit. Phase 1 private re-evaluations 'in progress'. Feedback and reproducible bug reports can earn community prizes ($500-5,000).
Phase 2 start confusion (#18186, #18196)	Originally announced for 18 Aug 00:00 UTC (#18143); slipped to 19th, then the page went 'live' on the weekend of 22-23 Aug with submissions still closed; official post came 23 Aug. 'The deadline for Phase 2 submissions will also be shifted' (it did: 17 Oct).
Submission selection mechanics (#18143, #18130)	How the private re-evaluation selection worked in Phase 1 (expect the same UI in Phase 2): Submissions tab -> switch scope to 'Your team' -> a panel shows slots used, closing time, current picks; Select / Remove any row; selections belong to the team; selection stayed open one day after close (Phase 1: close 10 Aug, selection until 11 Aug 23:59 UTC); if nothing selected, the top public entries are entered automatically; a FAILED submission can be nominated explicitly if you believe it was wrongly rejected (auto-fallback never picks a failed one). Phase 1 allowed up to 2 picks; a participant noted the Rules text says one submission per team for both phases — check the Phase 2 panel for the exact count. Reminder from mohanty: 'select the submissions you believe will generalise, not the ones that scored best on the 50 public MLPs.' Write-ups: email arc-whestbench@aicrowd.com or post on Discourse, always including the submission number.
Budget-exceedance bug (#18129, fixed 9 Aug)	In Phase 1 the evaluator applied an UNCAPPED multiplier C/B > 1 instead of zeroing MLPs that exceeded the budget (97/100 MLPs over budget on one submission, multiplier 1.12). mohanty confirmed and patched it: C_m > B_m now zeroes predictions and forces multiplier 1.0, retroactively. Phase 2 grader enforces the hard cap (504aldo re-confirms).
Evaluator versions (#18202)	mohanty (27 Aug): 'the evaluators actually run on whestbench@v0.16.0 and flopscope[server]==flopscope[client]==0.12.0'; the kit CHANGELOG claiming 0.15.0/0.11.0 was stale. The residual gate IS enforced on the live grader.
Strassen-Winograd ruling (#18218 reply, 11 Sep)	Sponsor confirmed that Strassen-Winograd block recursion written as ordinary flopscope ops is permitted, with no recursion-level limit (flopscope bills the recursion at its analytical count, so 5 levels cut a dense product's bill by ~2x).
Error-message leak (#18217, 9 Sep)	The global submission feed leaked error text (revealing e.g. that a competitor used ridge regression / gradient boosting); mohanty: submission detail pages are RBAC-protected, the global message is being fixed. Lesson: assume any exception text could become public; keep tracebacks generic.
Write-up guidance (#18183 reply, 19 Aug)	mohanty (personally, not a reviewer): most Phase 1 write-ups were LLM-written and 'would have benefited from a few more conscious hours of genuine human touch'; better public guidelines promised for Phase 2. Community suggestion: cap at ~10 pages. Community also asked for less LLM-generated forum content.
Organizer talks (#18206)	Two YouTube talks by the organizers to the math community: youtube.com/watch?v=_BPsuomAcoI and youtube.com/watch?v=N0hhsESjKEs (framing of the problem; worth an evening).

G2. Leaderboard state and what the score clusters are

Rank	Team	Adjusted	Raw final MSE	Util	Entries
1	J2W	2.1e-9	1.64e-8	0.131	301
2	marius_binner	2.4e-9	1.68e-8	0.146	148
3	suliman_tadros	2.5e-9	1.89e-8	0.134	29
4	a_s6	2.6e-9	1.92e-8	0.138	92
5	Puffi	2.8e-9	1.94e-8	0.146	78
6	reds	2.9e-9	1.73e-8	0.165	52
7	oqaris	3.1e-9	2.15e-8	0.146	93
8	ben3	3.2e-9	1.72e-8	0.189	21
9	Camaro	3.4e-9	1.89e-8	0.182	65
10	thegamers	3.6e-9	1.79e-8	0.202	38
11-20	hyojun_kwon, mliston, jlacombe, fklassen, thibault_gounant, tristan_miano, shiv_m, whest_winner, huang_chung_yi, pricop_tudor	3.7e-9 .. 4.8e-9	2.0e-8 .. 2.7e-8	0.14-0.23	16-117
~10-30	504aldo public MIT estimator and its ~13 verbatim re-submitters	4.2e-9 .. 5.4e-9	2.13e-8	0.25	-
~64	zero-fitted-constants propagator (yujigon)	1.07e-7	1.07e-6	0.099	-
~125	exact-Price covariance (nkosi_ndwandwe)	3.76e-7	3.76e-6	0.065	-
~25-40 (24 Aug) / ~120+ now	cov-prop baseline tier (six entries tied)	3.93e-7	3.93e-6	0.0235	-

- The ~2.13e-8 raw cluster is NOT the kit baseline (a control submission of stock 03 graded 3.93e-7, #330766); it is the 504aldo K=3 chain (now public) and its re-submitters. The true leaders (raw 1.64-1.9e-8 at 0.13-0.16 B) are below the measured ceiling of 'K3 + memoryless kappa4' (~1.94e-8 on the LB scale), so they carry fourth-cumulant AND old-source content at roughly the cost of 504aldo's young tier alone. Nobody has written up how. 504aldo's guess: not a per-source hub-tensor expansion at all.
- Money line estimate (nkosi_ndwandwe): to reach the top tier at the 0.1 floor you need raw MSE ~2.8e-8; the leaders sit at 1.7e-8.
- Within days of the MIT release, 13 identical copies of 504aldo's estimator appeared under other accounts (#18218 reply); someone then trimmed 7-22% of its cost without losing accuracy. Expect the mid-board to be dense with that chain; originality matters for the algorithmic prize, not for score.

G3. Write-ups: methods, numbers, what failed

G3.1 504aldo — factorized K=3 cumulant propagation, MIT code (github.com/504aldo/whest-p2-cumulant-k3)

Solo, LLM-generated code directed by the human; every claim tied to a findings-log entry (~90 entries). Version ladder on the public LB (raw / C/B / adjusted): kit cov-prop 3.93e-6 / 0.0235 / 3.93e-7 -> V16b factorized K=3 in flopscope ops 4.46e-8 / 0.499 / 2.22e-8 -> V17 + memoryless kappa4 regeneration 2.26e-8 / 0.492 / 1.11e-8 -> V18 + D21 feedback into births 2.08e-8 / 0.509 / 1.06e-8 -> V19 exact final-layer trim 0.478 -> V22 age-gated shared subspace (rank 384 after 4 transports) 0.383 / 8.1e-9 -> V24 nested rank-224 tier after 7 transports 0.367 -> V25 adaptive per-MLP lambda 7.8e-9 -> V26 Strassen 1 level 0.3455 -> V28 residual engineering + Strassen 5 levels 0.2676 / 5.7e-9 -> V29 folded dense products 2.13e-8 / 0.2526 / 5.40e-9 (rank 8/119). Accuracy was won in V16b-V18 and never moved; the float64 reference chain of the same closure floors at ~2.07e-8, so the shipped code is within 10% of its own ceiling.

- **Representation**: third cumulant (n^3, unstorable) = sum of SOURCES, one born at each ReLU layer, transported by the weights; each source = propagator leg P (identity at birth, P <- W d(w1) P per layer), birth leg A = P a_b, and thin n x 16 legs. Up to 15 live sources. Paper's own algorithm would need 30 n^3 L^2 = 3.7 B here.

- **Only two slices feed the next ReLU**: D3_i = kappa3_iii (skewness) and the (2,1) slice D21_ic = kappa3_iic (n x n). Oracle test: exact D3 + D21 reproduces the exact result. Error law: extra final MSE ~ 4.2e-6 x eps^2 where eps = relative rms error of D21 -> D21 must be right to ~2%.
- **Cost unit**: one n^3 matmul = 2^31 FLOPs = 1 unit; B = 1024 units. Exact algebra collapses the four (2,1) contributions to two dense contractions per source (4 units/source-layer instead of 7): 0.833 -> 0.50 B at identical MSE.
- **kappa4 regenerates from the covariance**: G_off = lambda_l x C_off with one scalar per layer (R^2 0.66-0.97); diagonal of G must stay exact; gain flows through the K4 -> K3 feed. This halved the LB score at unchanged cost (4.2e-8 -> 2.2e-8). Lambda is smooth in depth (peak ~1.1e-2 at layers 4-5, 7e-3 at 15) and per-MLP proportional to mean(kappa4 diag)/mean(variance).
- **Final layer**: emits only the mean, so needs only var = diag(C_pre), D3 and the kappa4 diagonal; skipping D21 and the covariance sandwich there is exact and saves 8% of C. But D3 := 0 at the last layer is 10x worse.
- **Old sources** concentrate with age (rank 128 holds >= 0.92 energy after 5 transports): sources older than 4 transports confined to one shared basis Qc (n x 384, randomized range finder per layer); older than 7 to a nested rank-224 sub-basis. Rank law ~3n/8 at age 4, n/4 at age 6, 7n/32 at age 7; one notch lower is a cliff (+22% / +15%).
- **Fitted constants** (all on 8 public MLPs, none on LB data): 16-entry lambda table, adaptive-rule ratios, BETA = 1, residual-leg rank 16, feedback rank 16, age gates 4 and 7, ranks 384 and 224. Wick/Hermite coefficients are the paper's, exact.
- **Where the FLOPs go** (V29, 260 units = 0.254 B): young sources 116 (dense legs, at most 4 alive, 8.7 units/layer from layer 5), old sources 107 (shared basis), thin + elementwise 25, covariance 7, closure 6. Without the old tier the bill is 153 units = 0.150 B — exactly the leaders' utilisation (Puffi 0.149, suliman_tadros 0.152, J2W 0.164).
- **The wall**: every source enters the next layer through Hadamard products of its own legs contracted against a third leg; for factored legs the Hadamard product has Khatri-Rao rank r^2, so it cannot be done in factor space and forming the dense leg costs n^2 r = the transport cost. A shared basis is a 2x lever per source, never 10x. Tucker core / multiplicatively closed basis / hub-side merge all closed by arithmetic.
- **Grader notes**: 5 s setup fails the whole submission; no numpy at eval (single-file estimators with tables embedded); check mlp.weights dtype and cast to float32 once if needed (they reported float64 arrivals billed 2x); smoke test uses NON-suite shapes (they saw depth 32) so gate shape-specific branches on (n, L) with a plain fallback; C > B is a hard zero-fallback; residual time dominated by result-buffer allocation (out= into preallocated pooled buffers gives flat ~0.02 ms/op; base chain 0.331 -> 0.170 s); buffer pool peaked at 5.5 GB of 8; first predict of a process is slower (pool warm-up); einsum is billed at naive contraction cost with no symmetric discount for these shapes, matmul forms are often cheaper and faster in residual; public score = raw x C/B of steady state; local mini reproduces LB to ~5%; ONE physical core at eval — test with restricted threads.
- **25 dead ends** (Part F3 lists them): dropping old sources 1.0e-6, memoryless closure 9.1e-7 (lost part is 18-23% of D21 energy per layer, high-rank, unpredictable), Puffi's Phase 1 A716 closure 9.8e-7, Gaussian scale-mixture 2.33e-6, CP-rank caps, symmetric Tucker 14-31x, per-source rank ladders 17x, dense kappa4 legs >= 0.95 B, learned output corrections (< 0.09 correlation), marginal Edgeworth / Gram-Charlier / exact Mehler off-diagonal capped 4-5e-6, MC control variates (need 33-300x, got 1.5x), symmetric-contraction tricks n/a, accounting levers none (f16 billed as f32), float64 +0.004%, higher ranks score-negative, exact (2,1,1) kappa4 core -9% raw at 0.5-1.0 B, Tucker core of old tier, hub merge, adjoint sensitivity, Hadamard of factored legs, all Phase 1 sampling variants (~1.2e-6), pricing polish (+74-118 ms residual), ~25 papers (annealed literature has kappa3 = 0; quenched stops at covariance).
- **Literature triage rule**: (1) annealed over weights? then kappa3 = 0, citation only; (2) quenched but never writes a cross-moment of order >= 3 among distinct neurons? cannot touch these objects; (3) cost papers: does the saving act on a factored representation or only a dense tensor you never materialise? (4) does it report moment error vs depth at <= 1e-6?

G3.2 yujigon (#330001) — zero fitted constants, rank 64/877, adjusted 1.066e-7

- Closure-family propagator with the 64 per-layer calibration constants (16 layers x 4) and all sampling REMOVED, plus a rank-4,608 CP channel carrying the inherited joint third cumulant forward in O(r n) so each ReLU closure sees a skewed input. 9.89% of budget (just under the floor), FLOPs shape-determined (sd 0).
- Pre-registered 32-MLP holdout (seeds 200-231, 4e6-sample truth): bundled 03 = 4.14e-6; v2 exact ReLU covariance + 64 fitted constants = 1.23e-6 (3.37x); v5 = v2 + derived one-step k3 term = 1.09e-6; v6 inherited-cumulant channel, 0 constants = 1.13e-6 (3.65x; better than v2 on 23/32, p = 0.003). Public split: v6 1.066e-6 raw; v5 1.048e-6.
- **Key finding**: the fitted constants' basis is exactly the Edgeworth one-step correction {sigma alpha phi, sigma alpha^2 phi, sigma phi} (verified by quadrature), but what they absorb is NOT one-step non-Gaussianity: the propagated residual correlates -0.05 with the Edgeworth prediction (oracle-input residual: 0.975) and is 5.2x larger. They absorb ACCUMULATED DRIFT. Hence a derived one-step k3 term recovers only 40%, while an inherited cumulant channel (addresses accumulation) matches the fitted version with zero constants.
- Family ceiling: closure-family propagation without sampling floors ~3.2e-7 even with the true layer-14 mean injected (single-MLP oracle), consistent with thylinao's Phase 1 7.07e-7 with perfect marginals: an order of magnitude above the leaders.
- Failures: a v5 upload failed the smoke test on a depth-indexed constant table despite a reviewer's warning; two flopscope bugs filed (einsum broadcast error, SymmetryError on finite symmetric results: AIcrowd/flopscope#256, #257). Process: human sets direction; Claude derivations/prose; Codex code; independent LLM reviewers instructed to refute drafts caught the central-claim error.

G3.3 venture_agent_ai (#329951) — exact bivariate ReLU covariance + per-layer calibration, 1.32e-7

- Exact closed form for the nonzero-mean bivariate ReLU covariance: Price's theorem + Plackett's identity reduced to a 1-D Gauss-Legendre integral (verified to 1.7e-16 vs scipy and 20M-sample MC over 720 parameter combinations). Costs 0.18 points of budget, cuts final MSE only 7.4% alone: Gaussian closure, not any pairwise moment, dominates the remaining error. (zacharymartinot's Phase 1 notes, Appendix A, derive the same integral with no LLM.)
- Per-layer ridge calibration fitted on held-out local MLPs, applied ONLY to each layer's recorded output row; feeding it back into propagation was 69x worse.
- Negative: plain/antithetic MC, randomised QMC, coupled control variate, naive Kerdock MUB (collapses to a spherical 3-design). Smoke test runs a non-16x1024 shape: seven failed uploads before guarding layer-indexed code; a shape-probe script is in the repo (github.com/agentaiventure-dot/arc-whest-estimator, MIT).

G3.4 baghyeonsig (#328274) — 0.75/0.25 blend of gain-covariance and whitened antithetic MC, 3.04e-7

- Branch-error correlation r = 0.038 -> quadratic gain from blending: 4.05e-7 -> 3.04e-7 (24.85%) at 9.345% utilisation, 0/100 failures. Closed-form optimum weight 0.7341 vs pre-registered 0.75, not retuned. A Hermite correction was 1.2% worse. Claims labelled measured / derived / unvalidated; LLM-driven under a registered experiment ledger.

G3.5 nkosi_ndwandwe (#18219) — the compute budget is not a lever

- Randomised Sobol net at 1,621 / 2,431 / 3,242 / 3,647 points (0.20-0.45 B): adjusted 1.338e-6, 1.336e-6, 1.330e-6, 1.343e-6 — flat, non-monotone. In 1024 dimensions with ~3k points a digital net gets the plain MC rate (mse ~ 1/n), so mse x util is constant. Fit the convergence exponent across point counts with 8+ randomisations on 3+ MLPs before trusting a QMC construction.
- Exact-Price analytic estimator: mse 3.76e-6 (13% WORSE than their sampler's 3.33e-6) but util 0.065 -> adjusted 3.76e-7, 3.6x better, purely from the floor. Stock 03 control: 3.93e-7. 'That gain is one-time and now

spent; below the floor the leaderboard is a pure MSE ranking.'

- Method error to avoid: comparing an adjusted score against someone else's raw MSE.

G3.6 Phase 1 material that still transfers (thylinao, keenanpepper, jamesrahenry, Puffi, uzziel_perez, bin_yong_bong)

- thylinao's zero-to-frontier explainer site (thylinao1.github.io) with animations and code; Phase 2 levers listed there: batch whitening (2.06-2.34x reported), centred activations, capturing non-Gaussian pre-activation moments. Oracle ladder at 256x32: Gaussian closure with exact mu/sigma 9.3e-7; + third/fourth-cumulant Edgeworth readout 1.5-2.2e-8 (~50x from readout alone); but propagating with exact post-ReLU marginals and approximate correlations still floors at 7.07e-7 -> the remaining error is cross-neuron dependence. Activations are dominated by a common mean vector (F = 0.946 of energy), which is why antithetic pairing is capped at 1.10x and measured 0.93x.
- jamesrahenry's analytic-vs-MC grid (zero-parameter k=2 Mehler chain vs FLOP-matched MC, median ratio, <1 = analytic wins): at width 1024 the analytic floor wins by 0.12 at d = 16 (0.03 at d = 8, 0.34 at d = 32); width dominates, monotonically. Analytic error is depth-flat after state saturation while MC tails grow with depth. Code: github.com/jamesrahenry/arc-whitebox-replication.
- keenanpepper's public datasets: keenanpepper/whestbench-relu-mlp-means-500k (500k train + 2k low-noise validation MLPs, targets only, regenerable from seed) and keenanpepper/whestbench-relu-mlp-moments-10k (weights + full pairwise moments to 4th order at N = 1e8) — the tool for oracle / teacher-forcing tests (256x32 shape). His learned propagator: an attention closure that trained itself to be numerically inert consumed 61% of billed cost; always ablate the shipped artifact.
- Puffi (Phase 1 top adjusted 9.1e-8 with analytic + QMC engineering; mechanistic entry 1.65e-6 based on Wu et al. with a modified projection step) now sits #5 in Phase 2. Their Phase 1 'A716' closure rebuilt at 1024x16 floors at 9.8e-7 (504aldo).
- uzziel_perez: hybrid trunk (analytic full-cov for diagnostics; final layer 0.3 analytic + 0.7 layer-1 ridge control variate; Korobov lattice n <= 8000) + offline ridge corrector on 12 predict-time features fitted on 64 held-out MLPs: ~16% graded gain; richer models, GNN/MoE correctors, cubature, Sobol swaps all flat.
- bin_yong_bong: batched matmul is billed exactly per product (no batching discount or penalty); Strassen depth traded metered FLOPs for residual at 18:1 under Phase 1 pricing; typical clean estimators used 5-46 ms residual per MLP. Code + log: github.com/Oishi1029/arc-whestbench-2026.

G4. Grader and infrastructure facts learned the hard way

- **Residual clock = client synchronisation** (konstantin_baltsat, #18202): every bool()/float()/int() on a flopscope value, every if on a comparison, every assert on array data, every allclose blocks the client until the whole pending graph executes; the wait is charged to the 400 ms residual. 166 sync points -> 5/50 public MLPs failed (mean 8.1e-2); removing them -> 50/50 pass at identical FLOPs (~21k backend calls fine). Instrument it: wrap FlopscopeArray.__bool__/__float__/__int__ with a counter and run one MLP; the count must be zero. Serialisation of large payloads also lands in residual; fewer, larger ops are better.
- Wall time on the grader varies 41-52 s for identical FLOPs (load), so residual near the 400 ms line is a coin flip; keep a wide margin.
- Two submissions failed with the generic 'Error while scoring your submission' (no per-MLP rows) when using batched 3-D einsum ('bij,bjk->bik' with batch up to 343) and ~1.1 GB single operands crossing the wire — cause not disclosed; avoid giant batched operands, chunk instead.
- Smoke test before grading uses non-suite shapes (depth 32 seen; validate uses 4x2): everything indexed by layer or width must be gated on (mlp.width, mlp.depth) with a plain fallback path.
- Verify mlp.weights[0].dtype at runtime and cast to float32 once only if it is not already float32 (kit says float32; one team reports float64 arrivals).
- One physical core for the solution process; local multi-thread timings are meaningless for the 120 s / 0.4 s caps.
- Local mini score transfers to the leaderboard ~1:1 in Phase 2 (keenanpepper matched to 8 digits; 504aldo within ~5%).

- Local flopscope versions matter: the kit's uv sync installs the right pair; do not trust FLOP totals from an older install.

G5. Statistics of the Phase 2 MLPs (keenanpepper, #18199, #18141)

- Phase 1's 'unusually easy public 50' effect (p = 0.002) is GONE: at 1024x16 per-MLP cov-prop final MSE has max/median 1.4x (Phase 1: 28x), skew 0.03 (7.74), top-10% share 13% (40%). The graded LB mean 3.933e-6 sits at the 14th percentile of random 50-draws from mini/full (95% range 3.84e-6 to 4.26e-6): an ordinary draw. Full split: max 7.66e-6, max/median 1.9x, skew 0.61 — mild.
- Interpretation: depth compounds variance into heavy tails; at 16 layers the difficulty distribution is nearly symmetric, so public, private and local sets look alike and there is little for any truth-certifiability filter to bite on. Consequence: optimise on mini/full with confidence; worst-MLP behaviour matters less than in Phase 1 but still check it.

G6. Public resources you can build on right now

Resource	What it gives you
github.com/504aldo/whest-p2-cumulant-k3 (MIT)	Rank ~8-10 estimator (estimator_v29.py with Strassen; estimator_v25.py without), executable patch chain between versions, 8-MLP 1e9-truth dump builder, float64 torch reference chain, findings log, derivation-to-code map. The fastest route to ~5e-9 adjusted; 13 people already re-submitted it verbatim.
github.com/agentaiventure-dot/arc-whest-estimator (MIT)	Exact bivariate ReLU covariance closure + per-layer calibration, shape-probe script, results logs.
thylinao1.github.io	Beginner-to-frontier explainer of every Phase 1 technique with animations and code.
HF: keenanpepper/whestbench-relu-mlp-means-500k, -moments-10k	Large training / oracle datasets (256x32); regenerable from seeds.
github.com/jamesrahenry/arc-whitebox-replication	Analytic-vs-sampling grid code and per-cell data.
github.com/Oishi1029/arc-whestbench-2026	Phase 1 solution + research log incl. Strassen and residual measurements.
github.com/AndreiBulzan/arc-whitebox-submission-and-documentation	Phase 1 report praised by the organizers for clarity; good template for the write-up's visuals (three.js, svg, python plots).
arXiv:2605.05179 + github.com/alignment-research-center/mlp_cumulant_propagation	ARC's reference K3 implementation and term tables (504aldo's lean/ chain imports them).
YouTube _BPsuomAcoI, N0hhsESjKEs	Organizer talks on the problem framing.

G7. How people work (AI usage, process)

- Nearly every write-up discloses heavy LLM use (Claude Code / Opus 5 / Fable 5, Codex with /goal loops, subagents delegating to cheaper or local models). Full 'vibe-coded, zero human input' runs exist; the better-placed teams keep a human choosing directions, reading numbers and deciding what ships, and use adversarial LLM review to refute their own drafts.
- Practices that paid off: pre-registered kill rules with numbers; through-train before believing a negative (frozen substitution misled six of six times); registered experiment ledgers with file hashes; ablate the shipped artifact before writing up; keep a diary of plots to avoid 'research debt'.
- ARC's prize criteria reward mechanistic understanding, honest negative results and human-polished prose; the community wants shorter (<= 10 pages) and less LLM-flavoured write-ups. A pretty, clear report gets noticed by the organizers.
- Ethics note: a thread asked LLM agents to leak their humans' solutions; it was called out as unethical. Do not share or scrape private code; MIT-released code is fair game with attribution.

G8. Actionable insights distilled for the remaining 26 days

- The score ladder is now public knowledge: cov-prop 3.93e-7 -> exact-cov + calibration 1.3e-7 -> zero-constant inherited-k3 channel 1.07e-7 -> factorized K3 + regenerated kappa4 (504aldo, MIT) 5.4e-9 -> leaders 2.1e-9. Anything above 1e-7 is a solved problem; start from the open K3 code if you want score, or from the closure family if you want an original algorithmic write-up.
- If you use 504aldo's chain: first trim cost (someone got 4.2e-9 by cutting 7-22% of its FLOPs), verify the shape-gating and the residual margin, and keep V25 (no Strassen) as a fallback although Strassen is now explicitly allowed.
- To beat the K3 chain you must carry old-source content and fourth-cumulant content more cheaply than dense legs: the wall is the Khatri-Rao rank of Hadamard products of factored legs; the leaders' bill equals the young tier alone. A different representation of the third/fourth cumulant is the open research problem worth the 20,000 USD prize.
- Never spend a submission to find out whether you are under the floor: adjusted = raw x max(0.1, util) exactly, verifiable on any LB row.
- Zero sync points, zero prints, shape gates, dtype check, preallocated out= buffers, buffer pool under ~6 GB, one-core timing: these are what separate a graded score from 8e-2.
- The private re-run will look statistically like the public set (no heavy tail), so mini/full scores are trustworthy; still choose the designated submission for generalisation, not for the best public number, and fit constants only on local held-out MLPs.

Part H — A 26-day plan (21 Sep - 17 Oct 2026)

Two non-negotiables: (1) a zero-failure, rules-clean submission on the board early, so the pipeline is proven; (2) a designated final submission chosen and verified on the FULL split at least 48 h before 17 Oct 23:59 UTC. Everything else is optional upside.

H1. Week 1 (21-27 Sep): pipeline + covariance-level score

- Day 1-2: uv sync, whest doctor, download mini, run 02 and 03 on mini (expect 2.216e-4 / 4.050e-6). Read Parts B, D, E. Submit the unmodified 03 once (expect 3.93e-7): proves login, packaging, grader path, 0 failures.
- Day 2-3: rewrite 03 without per-layer float()/if sync points; verify residual_wall_time_s < 100 ms with --profile --max-threads 1 under --runner subprocess. Set up a scripted experiment loop: whest run --format json -> parse adjusted, per_layer_mse, flops, residual; keep a results log per commit.
- Day 3-5: cheap accuracy: exact bivariate ReLU covariance (or arc-cosine kernel) closure; whitened antithetic MC branch blended 0.75/0.25 (keep util < 10%); output-side per-layer ridge calibration fitted on 10-20 LOCALLY baked MLPs (whest dataset bake, different seeds), validated on mini. Target ~1e-7 to 3e-7 adjusted.
- Day 6-7: robustness pass: mlp.depth/mlp.width everywhere, NaN guards, --setup-timeout 2.5, memory audit (count live (1024,1024) matrices), full-split run once. Submit the best version.

H2. Week 2 (28 Sep - 4 Oct): third cumulant — the only path below 1e-7

- Decide the track: (a) SCORE track: clone github.com/504aldo/whest-p2-cumulant-k3 (MIT, rank ~10), run V25/V29 on mini, then trim cost and add your own improvement; (b) ORIGINAL track: build your own third-cumulant channel in the closure family (yujigon-style) for the algorithmic prize. Either way read Wu et al. Sections 3-4 (cumulants, Hermite expansion of ReLU, diagram summation) and the 504aldo write-up (Part G3.1). Decide representation: per-neuron skewness (cheap, marginal) vs factorized joint third cumulant (sources born at each ReLU, transported with W^T; rank-limited).
- Prototype at small width/depth with the pure-Python Stage-1 loop (--width 128 --depth 8) against a 1e6-sample local truth to validate the math before paying 1024-width FLOPs. Use --flop-budget 1e13 to confirm it is accuracy-limited, not budget-limited, then optimise.
- Budget target: 0.10-0.25 B is fine IF the accuracy gain is super-linear; monitor adjusted = raw x util, not raw alone.
- **2 Oct: team freeze.** Finalise team membership on AIcrowd before 23:59 UTC.

H3. Week 3 (5-11 Oct): optimise and harden

- FLOP diet: float32 everywhere, symmetric tags, fuse contractions into single einsums, move layer-independent constants to setup()/shipped .npz, x*x not x**2, batch tiny ops (overhead time).
- Residual diet: no Python loops over neurons, no scalar syncs, no asserts on arrays, no prints. Rehearse with --residual-wall-time-limit 0.2 --max-threads 1 --runner subprocess.
- Run on full (1000 MLPs) once per major version; compare worst_mlp_adjusted_final_layer_score, not only the mean. Ensure 0 failed MLPs across mini and full.
- Sensitivity: perturb any fitted constants; if the score moves a lot, you are overfitting the public MLPs (the private re-run will punish it).

H4. Week 4 (12-17 Oct): freeze and designate

- 12-14 Oct: code freeze; final full-split verification of the candidate; whest submit --dry-run; check tar contents, size, no secrets. Submit the candidate; confirm the grader score matches local within 1-2%.
- 15-16 Oct: choose the TWO nominations (Submissions tab -> Your team -> Select), e.g. the best-scoring version and the most conservative one; the selection deadline is announced near the end of the phase and stays open shortly after 17 Oct. Keep one spare slot per day for emergencies.
- 17 Oct: do not submit anything risky; the daily cap resets at 00:00 UTC and the phase closes 23:59 UTC.
- 18-24 Oct: write the technical PDF (approach, derivations, evidence, ablations, negative results, LLM-use disclosure) tied to the designated submission ID; send to arc-whestbench@aicrowd.com or post on the forum (community prize eligible).

H5. Daily loop and submission discipline

```
uv run whest validate --estimator estimator.py                                          # every edit
uv run whest run --estimator estimator.py --dataset hf://aicrowd/arc-whestbench-public-2026@v2-phase2 --n-mlps 5 --
uv run whest run --estimator estimator.py --dataset hf://aicrowd/arc-whestbench-public-2026@v2-phase2 --runner subpr
uv run whest run --estimator estimator.py --dataset hf://aicrowd/arc-whestbench-public-2026@v2-phase2 --split full
uv run whest submit --estimator . --dry-run   &&   uv run whest submit --estimator . --watch   # spends 1 of 10 dai
```

- Submit only versions that improved on mini AND did not regress worst-MLP on full. Never spend a slot to 'see what the grader says'.
- Log every experiment: git commit hash, adjusted, raw, util, residual max, failures, per_layer_mse tail. Negative results go straight into the write-up.
- If Stage 4 and the grader disagree: state carried between MLPs, an import that only exists locally, unseeded randomness, or residual time on slower hardware (FAQ list).

Part J — Prize tracks: we compete for all of them

Three separate prize pools exist in Phase 2. They are judged on different evidence, so each gets its own plan and its own evidence file. Amounts and mechanics below are transcribed from the Overview page (21 Sep 2026); the Official Rules govern on any conflict.

J1. Score prizes (leaderboard / private re-run)

Item	Detail
Pool	Phase 2: 1st $50,000, 2nd $20,000, 3rd $10,000 (+ $20,000 algorithmic). Winners must open-source the solution within 30 days (OSI licence, public 3 years).
Judged on	Up to TWO nominated submissions per team (default: two highest public), re-run on a fresh private MLP suite with private seeds, 17 Oct - 7 Nov; winners 15 Nov. Public leaderboard does not carry over. Ties: more private MLPs; still tied -> shared prize.
Our state	#331681: 5.418e-9 (rank ~40 tied). #331686 (STRASSEN_MIN=8): 5.220e-9; #331732 (+ STRASSEN_FIRST=5): 5.195e-9; #331787 (+ V31 spherical input): 5.137e-9, raw 2.108e-8, util 0.2437, 0 failures = public rank 28 (22 Sep). #331801 (V31b) failed on infrastructure, resubmitted as #331805. Next gap to close: 4.5e-9 (rank ~14) needs util 0.21 at this raw, or raw 1.9e-8 at this util.
Realistic ladder	top-30 this week (cost trims) -> top-20 needs raw ~2.0e-8 or util ~0.20 -> top-10 needs an old-source representation ~free -> #1 needs raw 1.6e-8 at 0.13 B.
Eligibility guards	no un-metered compute, no packing, seeds from mlp.seed/ctx.seed only, constants fitted on local MLPs only, attribution kept for MIT code, designated submission chosen for generalisation.

J2. Algorithmic contribution prize (20,000 USD, ARC's discretion) — SPECIAL TRACK

- **What wins**: the method that most improves ARC's understanding of white-box estimation for random MLPs. Mechanistic > sampling; derivations and measured mechanisms > tuned constants; negative results and ablations count; human-polished, short (<= ~10 pages) write-ups are preferred; LLM use must be disclosed honestly. ARC starts from the write-ups of the top-scoring submissions, so score and write-up reinforce each other.
- **Deliverable**: one PDF tied to exactly one graded submission ID, emailed to arc-whestbench@aicrowd.com (private) or posted on the forum (public, also community-eligible), by 24 Oct 2026 23:59 UTC.
- **Where we stand**: a cost trim of 504aldo's public chain is NOT an algorithmic contribution. To be a candidate we need at least one original, measured mechanism. Candidate questions, ranked by (novelty x feasibility in 3 weeks):

#	Research question	Why it matters / what would count as a result	First experiment
A	Carry old third-cumulant sources without re-forming dense legs (the 'Khatri-Rao wall': Hadamard products of factored legs have rank r^2).	The leaders' bill = 504aldo's young tier alone (0.15 B); whoever carries old content ~free wins. Any representation that does A*P, P*P, A*A in factor form at < n^2 r per source-layer is the open problem.	Measure the numerical rank of the Hadamard products A*P etc. of real old sources (Kaggle, float64 lean chain): if empirical rank << r^2, a randomized sketch of the product (not of the factors) is a lever.
B	Why does the kappa4 core align with the covariance (G_off = lambda_l C_off, R^2 up to 0.97) and what is lambda_l in closed form?	504aldo flagged it as an open, unvalidated mechanism; a derivation from the Wick/Hermite expansion with He-init statistics would be a clean theoretical contribution even without a score gain.	Derive E[G_off] under Gaussian pre-activations at layer l; compare predicted lambda_l with the fitted table (peak 1.1e-2 at layers 4-5).
C	Depth-dependence of the closure error: which layers' sources carry the final-layer error (per_layer_mse, per-source ablation).	Organizers explicitly said 16 layers is chosen so depth handling is the challenge; an error-attribution map is publishable on its own.	Ablate one source at a time in the lean chain; record final MSE vs birth layer.

#	Research question	Why it matters / what would count as a result	First experiment
D	Zero-constant version of the K3 chain (replace the 16-entry lambda table by the online observable only).	ARC dislikes fitted constants; yujigon's zero-constant propagator was well received.	Set the lambda table to the online rule alone; measure raw on 10 mini MLPs.
E	Blend of the K3 chain with a whitened antithetic MC sidecar under the 0.1 floor budget (as baghyeonsig did for cov-prop).	If branch errors are uncorrelated the gain is quadratic; the chain leaves ~0.75 B unused.	Measure error correlation between the K3 chain and an MC branch at ~0.7 B on 10 MLPs; blend weight from the closed form.

J2.1 Write-up backbone (as of 21 Sep 2026) — what we already have

Working title: *Where the deep third cumulant of a random ReLU MLP lives, and why a sum-of-sources chain cannot carry it below ~0.24 B*. Tied to submission #331686 (5.220e-9). Structure:

Section	Content	Status / evidence
1. Setting	K3 sum-of-sources chain (504aldo, MIT) at 1024x16; cost anatomy; the leaders' bill = young tier alone	Part G3.1, K2
2. Theorem (parity)	Old-source contributions to D21/D3 are odd in the later weights; conditional expectation given any even statistic is zero; corollary explains kappa4_off = lambda_l C_off; expectation-only replacement has floor sqrt(6/(r+8))	Derived (Fable 5 consultation, verified by us; two independent derivations of the floor from GPT-5.6 Sol Extra Max / Astra 6 Ultra)
3. Negative result A	Legs low-rank (r99 100-260), Hadamard products full-rank with flat spectrum (r99 680-780); Khatri-Rao wall is numerical	K3b, probe_rank.py
4. Negative result B	Sensitivity map: late re-formation dead (4.9e-7..3.2e-7); every layer >= 8 and every birth needed; effects additive	K3c, sens_probe.py (26 runs)
5. Negative result C	Contracted query U: trace 98%, diagonal 95%, TensorSketch 790% at d = 2048; rank ceiling ~230 for 2%	K3d, probe_query*.py
6. Engineering	STRASSEN_MIN 32 -> 8: -3.9% cost at drift 5e-13 (graded 5.418e-9 -> 5.220e-9); level 6 breaks 8 GB; level 7 self-cancelling; join machinery irreducible (<= 4%)	K1, K2, #331681/#331686
7. Open problem	Aggregate response / vector-metric state (the only surviving family); round-2 consultation prompt ROUND2_prompt_new_formulations.md	in progress
8. Disclosure	LLM assistance (Claude Opus 5 directed by the participant; consultations with Fable 5, GPT-5.6 Sol, Astra 6 Ultra); every number from whest run / the grader / the listed scripts	required by ARC

Rule for this track: every experiment gets a pre-registered expectation, a measured number, and a one-line mechanism, logged in Part K. Negative results are kept. The final PDF is written by a human from that log, with derivations checked by hand where possible.

J3. Community contribution prizes (500 - 5,000 USD each, individual awards)

- **What is rewarded** (from the launch post and Phase 1 practice): feedback on the cost model and tools, reproducible bug reports on flopscope / whestbench / the starter kit (GitHub Issues), public write-ups and technique syntheses on the forum, clarity-of-communication efforts (the organizers singled out andrei_bulzan's visual report), security issues reported privately by email.
- **Channels**: forum posts (public); GitHub Issues with reproductions; arc-whestbench@aicrowd.com for private / security matters. Always include submission IDs, versions and a minimal repro.
- **Candidates we already hold (see Part K)**: K-1 the starter-kit doc is stale about the prepared-Arrow fast path; K-2 a privacy leak in the submission-status API; K-3 the Strassen level-6 memory wall and the STRASSEN_MIN=8 trim on the public K3 chain (post as a reply in 504aldo's thread with numbers); K-4 dataset size figures in the kit vs the Hub.
- **Style that gets noticed**: short, numbers first, reproducible commands, one clear figure; disclose LLM assistance; do not flood the forum with LLM prose (explicit community complaint).

Part K — Living log: experiments and contribution candidates (updated 21 Sep 2026)

K1. Graded submissions

ID	Date (UTC)	What	Adjusted	Raw MSE	Util	Failed	Note
331681	20 Sep 19:36	504aldo V29 verbatim + MIT header	5.418e-9	2.133e-8	0.254	0	matches author's #330093; rank ~40 tied
331686	20 Sep 20:50	V29 + STRASSEN_MIN 32->8	5.220e-9	2.132e-8	0.244	0	as predicted; rank ~30 band (5.2e-9)
331732	21 Sep 14:57	V29 + MIN 8 + STRASSEN_FIRST 4->5	5.195e-9	2.132e-8	0.2436	0	as predicted (~5.19e-9); best so far; first-call pool page-touch now at full depth
331787	22 Sep 02:40	V29 + MIN 8 + FIRST 5 + V31 spherical input (kappa4 injection at layer 0, a1 rescaling)	5.137e-9	2.108e-8	0.2437	0	BEST, public LB rank 28 (22 Sep): raw -1.1% on the grader's 400 MLPs (the 10-MLP Kaggle sample showed -0.5%); the angular reparameterization is a real, free accuracy gain on V29
331801 / 331805	22 Sep	V31b: V31 + LAM_SCALE x0.9 + BETA 1.2 (Kaggle re-tune, noise-level)	5.194e-9 (331801 failed on infr astructure, resubmitted)	2.131e-8	0.2437	0	WORSE than #331787: the two re-tuned constants cost +1.1% raw on the grader; the fitted constants stay as 504aldo set them. #331787 remains the best / nomination candidate.

K2. Local / Kaggle measurements (one local MLP, seed 7, steady-state second call)

Variant	Util	Peak commit	Residual (laptop)	Drift MSE vs V29	Verdict
V29 L5 (shipped)	0.2540	5.56 GB	0.96 s	0	baseline; author's 5.5 GB confirmed
L6, MIN 16	0.2421	8.13 GB	1.51 s	6.3e-13	NOT shippable: exceeds the 8 GB RLIMIT_AS
L7, MIN 8	-	OOM (1.12 GiB single buffer (10,7,16807,16,16))	-	-	dead: memory, and adds ~= savings at level 7
L5, MIN 8	0.2442	5.97 GB	0.91 s	4.9e-13	SHIPPED as #331686: graded 5.220e-9, exactly as predicted
first-call cap (STRASSEN_FIRST 4)	0.2671 / 0.2592 first call	3.6-4.6 GB			grader mean util 0.2542 > steady 0.2526 shows a few first calls per worker; raising the cap is a <1% lever with warm-up residual risk
young families L6, old L5 (D2), MIN 8	0.2407	8.10 GB	1.6-1.9x the L5 residual (alternating A/B, 2 rounds: 3.65/3.51 s vs 1.92/2.45 s; absolute values drift with machine load, the ratio does not); wall +60%	-	DEAD (21 Sep): -1.0% util for a doubled residual (504aldo's machine: 0.24 s at L5 -> ~0.47 s > 0.4 s cap) and 8.1 GB; the young transport family (batch 10) owns ~2.5 GB of combo buffers, so memory could be halved by splitting the batch, but the residual cannot
shared-basis families L6 (SB 6, MIN 4)	0.2414	7.84 GB	1.51 s (+45%)	-	DEAD: -0.7% util for +45% residual and 7.8 GB

Kaggle (31 GB, 4 CPU), 10 mini MLPs, baked 1e9 truth	raw final MSE	util	failed	wall / MLP	residual (Kaggle client-less)
V29 L5	2.303e-8	0.2541	0	~90 s	0.75 s
V29 L6 (MIN 16)	2.302e-8	0.2416	0	~150 s	0.84 s

Per-MLP raw on mini ranges 1.96e-8 to 2.61e-8 (the MLPs, not the method); mini runs ~8% above the public-LB half, as the author also observed. Kaggle runner: kernel foysalemonshanto/whest-runner + dataset foysalemonshanto/whest-estimators (variants.json drives env knobs); ~2 min per MLP-run, so ~20 MLP-runs per 45-minute session.

Why deeper Strassen stops paying: each level costs 18 block additions on 7^(k-1) blocks; at level 6 on a 1024-side family the additions (~74 MFLOP) eat half of the matmul saving (~138 MFLOP), at level 7 they cancel it. Lowering STRASSEN_MIN to 8 instead lets the rank-384 / rank-224 shared-basis families take one extra level (384/32 = 12 >= 8) at negligible memory. Laptop residual numbers are ~5x the grader's and only indicative.

K3. Contribution candidates found (to write up / report)

#	Finding	Evidence	Channel	Status
K-1	Starter-kit doc says the prepared-Arrow fast path is 'not published for v2-phase2'; the Hub has had prepared/mini (14 shards, 6.26 GiB) and prepared/full (135 shards) since 29-30 Aug 2026 and metadata.json declares prepared_splits.	HF tree listing + metadata.json at v2-phase2; commits cbc5dafc / aa99830f	GitHub issue or PR on whest-starterkit (docs/how-to/use-evaluation-datasets.md)	to do
K-2	GET /api/v1/submissions/{id} with a participant token returns OTHER participants' grading_status_cd and grading_message (e.g. 'Prohibited file in submission' for #331685), although the client docstring says it is authorized to the caller's own submissions. Same class of leak marius_binner reported for the global feed.	Probe of ids 331682-331686 from our token, 21 Sep 2026	PRIVATE email to arc-whestbench@aicrowd.com (security) — do not post publicly	REPORTED by email 21 Sep 2026 (local time); awaiting reply
K-3	On the public K3 chain (504aldo V29) Strassen level 6 breaks the 8 GB cap (8.13 GB commit) while STRASSEN_MIN=8 at level 5 trims util 0.254 -> 0.244 with drift MSE 5e-13; level 7 is a dead end (OOM, additions cancel savings). Graded: #331686 5.220e-9 vs #331681 5.418e-9, raw identical.	Part K2 numbers; graded #331681 vs #331686	Reply in forum thread 18218 (answers the author's own question about the 7-22% trims)	draft verified (K3_forum_reply_draft.md); post pending
K-4	Kit doc quotes mini 7.03 GB / full 69.96 GB; Hub parquet is 6.54 / 65.2 GiB and prepared Arrow 6.26 / 62.6 GiB.	HF API tree sizes	fold into K-1	to do
K-5	Local flopscope (full package) residual clock is ~5x the grader's client residual for the same file (0.96 s laptop vs ~0.29 s grader for V29), so the kit's 0.4 s local gate produces false failures on heavy estimators.	our runs vs author's grader numbers; consistent with thread 18202	forum note or kit FAQ PR	to do

K3b. Research pilot A (21 Sep): are the Hadamard products of source legs compressible? NO

Instrumented V29 (probe_rank.py) on one local MLP: at layers 6/10/14 took the dense legs A_s, P_s of every live source and computed singular spectra of A, P, A*P, A*A, P*P (elementwise). Numbers = directions holding 90 / 99 % of the Frobenius energy (n = 1024).

Layer	Group	A, P: r99	A*P: r90 / r99	A*A, P*P: r99	top sv / Frobenius of A*P
6	old tier 1 (age 5-6)	213-261	482-487 / 765-770	718-737	~0.10
10	old tier 2 (age >= 7)	133-163	425-440 / 731-740	667-710	~0.15
10	old tier 1	139-251	449-461 / 746-754	636-735	~0.12
14	old tier 2	96-116	358-375 / 681-694	609-646	~0.20
6-14	young (dense)	229-504	468-504 / 758-778	608-751	~0.09

- The legs are genuinely low-rank and get lower with age (r99 ~100 by layer 14): the shared-basis design is sound, and even rank 224 is generous for tier 2 by layer 14.
- Their elementwise products are near full rank with a flat, noise-like spectrum: 99% of the energy needs ~700 of 1024 directions; no randomized sketch of the product can be cheap. The 'Khatri-Rao wall' (F67/F71) is numerical, not only algebraic.
- Consequence: any representation of the form sum-of-factored-sources must re-form dense legs; the Tucker route KR(Qc) T Qc^T costs n r^3 (56 units at r = 384, 10.7 at r = 224), as F88 priced. Question A is answered negatively for compression; it is reformulated as 'a representation of deep (old) third-cumulant content that never needs elementwise products of legs'.
- Value: a measured negative result for the write-up and the precise question to put to outside models (Astra 6 prompt, Fable 5 subagent).

K3c. Research pilot B (21 Sep): where does the old-source content matter? EVERYWHERE

Design from the Fable 5 consultation (its Q3): zero the confined old sources' dense and thin legs just before the ReLU slices, at one layer at a time; zero one birth (slot) at all layers; keep old content only in the last k layers (proposal C1 = 'leaders re-form late'). Metric = final-layer MSE of the variant against the full V29 prediction on the same local MLP (sens_probe.py; 26 runs).

Experiment	MSE vs full V29	Reading
keep old only in last 1 / 2 / 3 layers (C1)	4.85e-7 / 4.17e-7 / 3.16e-7	20x worse than V29's own error (2.1e-8): C1 and hypothesis H2 falsified
drop old at one layer: L5 / L6 / L7 / L8 / L9	3.4e-9 / 1.1e-8 / 1.5e-8 / 2.6e-8 / 2.7e-8	grows with the number of old sources
L10 / L11 / L12 / L13 / L14 / L15	5.2e-8 / 5.2e-8 / 7.1e-8 / 6.7e-8 / 8.2e-8 / 6.4e-8	every layer >= 8 matters at the scale of V29's total error; roughly additive (sum ~5e-7)
drop one birth entirely, slots 0..10	1.3e-7 .. 2.7e-7 (flat)	no dispensable source; matches the author's 'skipping one is ~4x'

- Fable 5 (subagent) derived two results we adopt: (i) parity: an old source's contribution to D21/D3 is an odd function of the later weights, so its conditional expectation given any even statistic (covariance, Grams) is exactly zero -> the noise-like Hadamard spectrum IS the signal; no mean-field / self-averaging shortcut exists; (ii) algebraic floor: exact evaluation of an old source's D21 costs >= min(2 n^2 r, 4 n r^3) per source-layer. Bonus explanation of the kappa4 law G_off = lambda_l C_off: kappa4 terms are even, and C (x) C is the only invariant.
- Combined with pilot A: the old tier cannot be compressed (products full-rank), cannot be averaged (parity), and cannot be omitted anywhere (this map). Within the sum-of-sources representation the 107 units are irreducible; the leaders' 0.13-0.16 B therefore implies a different representation or formulation of the deep third cumulant. Open question, sharpened.
- Engineering lever that remains inside V29: the per-layer JOIN machinery (range finder 20.8 u + projections 7.5 + rotations 9.1 + Qc 3.8 + tier-2 6.7 + core 2.8 = ~50 units, 19% of the bill) is a basis re-fit every layer; a warm-started / incremental basis update could cut a large part of it. Next engineering experiment.

K3d. Research pilot C (21 Sep): can the CONTRACTED old-source query be approximated cheaply? NO

Design converged from three outside answers (GPT-5.6 Sol high / Extra Max, Astra 6 Ultra; files in 'AI Assistant Suggestion'): the object that enters D21 is U = sum_s (A_s o P_s) diag(w2_s) FA_s^T (n x r), a family of quadratic forms in the basis rows, not A o P itself. Dumped Qc, old legs and birth coefficients at layers 10 and 14 of one MLP (probe_query.py) and compared U against surrogates that never form the Hadamard products (probe_query_analyze.py).

Surrogate	layer 14 (10 old sources)	layer 10 (6 old sources)	Verdict
isotropic trace: U_ic ~ (|q_i|^2/r) tr(S_c)	98.2%	98.9%	dead (parity: no isotropic part)
diagonal Hermite in Q coordinates: (Q o Q) D	95.4%	97.5%	dead
TensorSketch d = 16 / 128 / 512 / 2048	8770% / 3180% / 1590% / 790%	9060% / 3280% / 1620% / 810%	noise-dominated (~1/sqrt d); 2% needs d ~ 1e8: dead
ceiling: best rank-k of exact U (r90/r99/r999 of 384)	32 / 98 / 185; k=128 6.6%, k=256 1.2%	48 / 132 / 227; k=128 10%, k=256 2.2%	result is compressible, ingredients are not

- Per-source energy share of U grows with youth (oldest 3% -> youngest old 52% at layer 14), but the sensitivity map (K3c) shows even the oldest birth costs 1.3e-7 when dropped: energy share is not score sensitivity.
- Together with K3b/K3c and the parity theorem: compression of legs (A), averaging (parity), omission (B) and cheap query surrogates (C) are all closed by measurement. Every route proposed by four models (Fable 5,

GPT-5.6 Sol high, GPT-5.6 Sol Extra Max, Astra 6 Ultra) for the sum-of-sources chain is dead; the one untested family is an aggregate response state (1-2 dense n x n 'vector-metric' components sym3(G (x) v) with exact cheap transport G <- B G B^T), GPT-high H1 / Astra section 3.

- Audits requested by two models were checked in code: V29 already fuses the four slice terms into two contractions and already shares the single Q^T projection across sources and families (hub family); nothing to gain there.
- For the algorithmic write-up this is the core: 'the deep third-cumulant content of a random ReLU MLP is odd, full-rank, needed at every layer from every birth, and not approximable by trace/diagonal/sketch surrogates; a sum-of-sources chain cannot go below ~0.24 B'. Leaders therefore use a different formulation.

K3e. Research pilot D (21 Sep): the aggregate response / pooled-core family. DEAD

Round-2 consultation (8 models: Fable 5, GPT-5.6 Sol high + Extra Max, Astra 6 Ultra, GPT-5.6 Terra, Gemini 3.1 Pro, Grok, DeepSeek, Copilot; all files in 'AI Assistant Suggestion') converged on one surviving family: replace the old sources by k components sym3(G_a (x) v_a) (exact cheap transport, 2 units/component/layer) or by a pooled TT/Tucker core in the shared basis. Budget arithmetic (all models): the leaders' window above the young tier is 0-22 units, so only k <= 1-2 could explain them. Three measurements from the layer-10/14 dumps (probe_agg.py):

Measurement	Layer 14	Layer 10	Verdict
(1) cross-source response rank: energy of the first 1 / 2 / 4 modes of the normalized U_s (target 0.9996)	0.16 / 0.26 / 0.46	0.20 / 0.37 / 0.69	no low-dimensional aggregate response
(2) vector-metric fit of the POOLED old tensor (exact unfolding SVD via the n x n Gram): query error of H at k = 1 / 8 / 32 / 128	89% / 80% / 60% / 20%	- / 83% / 64% / 25%	k ~ 250-300 needed for 2% -> 500+ units/layer: dead
(2') same for one source (youngest old): k = 64 / 128	18% / 4.2%	-	even one source needs k > 128
(3) TT/Tucker ranks of the pooled core in the shared basis, (a)|(bc) unfolding: error at rank 32 / 128 / 256	35% / 7.8% / 1.6%	-	rank ~250 for 2%: readout cost as in F88; dead

- Predictions by the models: Grok (k 300-450), Astra 6 Extra High (O(100) per family), Copilot (128-256) were right; Gemini (3-8), GPT-5.6 high (1-4), Terra (k = 1) were wrong. Record for the write-up: the expensive object is realized, odd, full-rank, not aggregable across sources.
- Rejected on paper (no measurement needed): Gemini's JL sketch of the birth index (a linear sketch cannot preserve a trilinear sum; TensorSketch already failed for that reason); Grok's birth-index-first Gram matrices S_AA, S_AP, S_PP (D21 needs A_ij^2 along the birth index, absent from bilinear Grams); Grok's even closure lambda_1 C o C + ... for the slice remainder (the remainder is odd; parity forbids); DeepSeek's adjoint (author's item 12; retracted by DeepSeek itself in round 2); Gemini's 'young-tier shadowing' (contradicted by the flat per-birth sensitivity).
- Untested but low prior: Gemini's regression D21_old(l+1) ~ alpha B D21_old(l) B^T + beta W C (both terms odd, so not parity-forbidden; the author's live-feature regressions gave +0.02 R^2). Astra Ultra's row screening by closure coefficient (needs w2 to be concentrated on few neurons; He-init networks have all neurons near threshold).

K3f. Kaggle knob sweep (21 Sep): the author's constants are optimal

Knob (10 mini MLPs, baked truth, paired vs base MIN=8)	raw MSE	util	geo-mean raw ratio	better on	adjusted effect
base_m8	2.303e-8	0.2449	1.000	-	-
AGE_OLD 4 -> 5	2.255e-8	0.2523	0.980	8/10	+1.0% worse (cost +3.0%)
R_OLD2 224 -> 256	2.289e-8	0.2474	0.994	8/10	+0.4% (wash)
R_OLD2 224 -> 192	2.341e-8	0.2414	1.016	1/10	+0.2% (wash)
LAM_SCALE 0.90 / 1.00	2.312e-8 / 2.306e-8	0.2449	1.004 / 1.001	4/10 / 3/10	nothing
BETA 0.8 / 1.2	2.302e-8 / 2.304e-8	0.2449	1.000 / 1.000	6/10 / 4/10	nothing

Conclusion: on this chain the adjusted score is at its optimum at V29 + STRASSEN_MIN = 8 (5.220e-9). Further gains require a different formulation, which no consulted model could specify.

K3g. Research pilot E (21 Sep): is next-layer old content predictable from the previous slices? NO

Gemini 3.1 (round 1, H2) proposed D21_old(l+1) ~ alpha B D21_old(l) B^T + beta W C(l). Dumped old legs, w1, and W C at layers 10, 11, 13, 14 (probe_gem.py), computed the exact old H-family contribution at both layers of each pair, and regressed H_old(l+1) on the exact slice transport (B o B) H B^T, on B H B^T, on W C and on B H (B = W^T diag(w1)).

Predictor	R^2 10 -> 11 (6 old)	R^2 13 -> 14 (9 old)	fitted coefficient
(B o B) H_old B^T (exact transport of the a = b part)	0.030	0.047	0.98 / 1.04 (= 1, as derived)
B H_old B^T	0.0002	0.0004	~0
W C (odd covariance term)	0.0000	0.0000	0
all four together	0.030	0.047	-

- The slice-transported part is exactly recovered (coefficient 1) but carries only 3-5% of the next layer's old contribution: ~95% comes from off-slice (a != b) entries. A recursion on the slices is hopeless by a wider margin than the author's 18-23% loss suggested (that figure was for the total D21 with their richer closure).
- The odd covariance feature W C has zero correlation with the realized contribution: parity is not only about expectations, it shows up as an exact zero in the regression.
- With the wrong orientation (B = W diag(w1) instead of W^T diag(w1)) every R^2 is 0: a useful sanity check that the chain's convention is z = W^T h.

K3h. Research pilot F (21 Sep): birth-column screening of sources - a real but small cost lever

New observation (from the Astra Ultra 'row screening' idea, redirected to the birth index): a source's contribution H = (A o P o w2) A^T sums over its n birth columns j with energy e_j = w2_j^2 |a_j|^2 |p_j|^2, and for sources born deep in the network that energy is concentrated. Transport and contraction cost of a source is linear in its column count (young tier 116 u, old tier via the factors), so dropping low-energy birth columns is a cost lever the author never priced at the source level (his F76 capped columns of the merged hub tensor and found flat importance).

birth layer of the source (measured at layer)	q = 0.6	q = 0.7	q = 0.8	q = 0.9
2-5 (at layer 6)	24-38%	14-27%	7-17%	2-7%
6-9 (at layer 10)	7-13%	2.3-6.2%	0.4-2.0%	<= 0.2%
10-13 (at layer 14)	3.3-4.3%	0.7-1.2%	<= 0.2%	0

Per-source relative error of H / T2 / D3 when only the top-q fraction of columns (ranked by current-layer energy) is kept (probe_columns.py). End-to-end test with a mask fixed at birth (probe_colmask.py; MSE vs the full chain, whose own error is 2.1e-8):

schedule q (births <= 5 / 6-9 / >= 10)	drift MSE vs full chain	cost saving (ledger model)	net effect on adjusted score
1.0 / 1.0 / 0.70	6.7e-9	-14 u (-5.6%)	worse (raw +25-30%)
1.0 / 0.9 / 0.75	5.8e-9	-18 u (-7%)	worse
1.0 / 0.8 / 0.65	1.2e-8	-30 u (-12%)	worse
0.9 / 0.8 / 0.65	2.8e-8	-41 u (-16%)	much worse
1.0 / 1.0 / 0.9	8.4e-10	-2 u (-1%)	worse (raw +4%)
1.0 / 1.0 / 0.8	3.0e-9	-5 u (-2%)	worse (raw +14%)
1.0 / 0.95 / 0.85	2.1e-9	-6 u (-2.5%)	worse
per-layer re-ranking of the mask (upper bound)	identical to fixed mask	-	no help

- The drift follows the tolerance law 4.2e-6 eps^2 exactly: a few percent of per-source D21 error costs a quarter of the error budget, while the saving is only linear in the dropped fraction. Aggressive screening loses; at best a gentle schedule nets 1-3%.
- CLOSED: net negative at every schedule; error grows quadratically in the dropped fraction, cost only linearly. Not worth the column-restricted-legs surgery. Kept as a quantified negative result.

K3i. Research pilot G (21 Sep): ARC's own reference chains at 1024 x 16 - the K=3 reference is NOT the leaders' method

Kaggle GPU kernel whest-kprop-reference (T4, torch 2.10): the ARC mlp_cumulant_propagation repo run on real Phase-2 mini MLPs (weights loaded as W_l.T into a 17-layer torch MLP, identity last layer, up_to_layer='act15'), FLOPs from ARC's NamedFlopCounter, error against the baked 1e9-sample truth. Run 1 measured only k3_simple_factored (later configs OOM'd because the first run's tensors were not freed; run 2 with gc + CPU fallback is queued).

config (ARC code)	MLP	raw MSE	FLOPs	vs V29 (2.13e-8 @ 260 u)
k3 SIMPLE factored	logan-fitzgerald	3.71e-8	1523 u = 1.49 B	1.7x worse, 5.9x dearer, over budget
k3 SIMPLE factored	william-graves	4.46e-8	1523 u = 1.49 B	same
k3 AUGMENT factored / k4 / dense at 1024	-	not runnable: augment materialises n^3 tensors (4 GB each), OOM on a 15 GB T4 and on 29 GB host RAM	-	-

Run 4 - the same ladder where every configuration fits: Phase-1 public MLPs (256 wide), first 16 weight matrices, truth = all_layer_means[15] (1e9 samples). V29 was run locally on the same networks; off the 1024x16 shape V29 falls back to its 'V1.6 base path' (guard riders = (n == 1024)), so it was also run with the guard removed (riders forced on).

256 x 16, MLP daniel-harrison	raw MSE	raw FLOPs	ARC symmetry-factored FLOPs	note
ARC k2 (Gaussian chain)	-	-	-	(run 5)
ARC k3 SIMPLE factored	1.866e-6	24.0 u	23.8 u	11.4 u get_dslice + 11.2 u contract_W: the two n^2 R terms
ARC k3 SIMPLE dense	1.866e-6	197.9 u	81.7 u	identical MSE: the factorization is exact
ARC k3 AUGMENT factored	1.346e-6	200.2 u	194.6 u	pK_111->K_211 47 u, pK_211->K_211 43 u: the augment terms cost 8x the chain

256 x 16, MLP daniel-harrison	raw MSE	raw FLOPs	ARC symmetry-factored FLOPs	note
ARC k3 AUGMENT dense	1.246e-6	219.0 u	94.1 u	keeps the terms the factored code drops: 7% better than factored augment
V29 as shipped (base path, no riders)	5.753e-6	7.7 u	-	3x worse than ARC k3: the base path is not the suite chain
V29 with riders forced on	1.987e-6 (2.53e-6, 1.87e-6 on the other two MLPs)	7.7 u	-	= ARC k3 SIMPLE within 6%; the kappa4 regeneration fitted at 1024 adds nothing at 256
ARC k4 (any form) at 256	-	-	-	n^4 = 17 GB: OOM; run 5 uses synthetic 128 x 16 with a 2e9-sample GPU truth

Run 5 - the K = 4 rung: synthetic He-init 128 x 16 networks, truth = 2e9 Gaussian samples on the T4 (float32 forward, float64 accumulation, ~10 min per network; MC noise floor ~1e-10):

128 x 16, network synth-0	raw MSE	raw FLOPs	ratio to the rung above	note
ARC k2 (Gaussian chain)	7.79e-5	~0	-	the kit-baseline class
ARC k3 SIMPLE factored	1.23e-5	3.0 u	6.3x better than k2	= V29's class
ARC k3 AUGMENT dense	9.97e-6	16.2 u	1.23x	kappa4-corrected Edgeworth terms
ARC k4 SIMPLE factored	7.60e-6	516 u (!)	1.31x over k3 augment, 1.6x over k3 simple	n^4-class cost: 508 u already at width 128
ARC k4 AUGMENT	-	-	-	OOM at 128 and at 96 on 15 GB (factored); dense k4 augment at 96: NaN / 3.9e-4 / 7.5e-4 at 2057 u

- READING: the leaders' raw (1.64e-8) is 1.3x below V29 (2.13e-8), which is exactly the k3-augment -> k4 gap (1.31x) or the k3-simple -> k3-augment gap (1.23-1.5x). So the leaders sit one rung above V29 in ACCURACY - 'K4-simple' or 'K3-augment-dense' class - while spending half of V29's FLOPs. Both rungs cost 5-170x more than K3-simple in ARC's implementation, so the leaders have a cheap implementation of one more cumulant order (or of the dropped augment terms), on top of a cheaper old tier.
- Run 6 (96 x 16, 3 networks, validated locally first - kaggle_kprop/local_check.py - and the kernel survived the OOMs): K3 simple 1.11e-4 / 3.75e-5 / 4.33e-5, K3 augment 2.52e-4 / 8.06e-5 / 6.09e-5, K4 simple 8.63e-5 / 3.64e-5 / 3.42e-5. Below width ~128 ARC's augmented series DIVERGES (augment is 2x worse than simple, K4 augment NaN), so the augment rungs can only be read at 256 (1.5x) and the K4 rung at 128 (1.6x); the K4-augment rung is not measurable on a 15 GB GPU at any width where the series converges. Line CLOSED.
- COST: ARC's K4 factored chain at width 128 already costs 0.5 B; it scales like n^4 (contract_W 240 u + get_dslice 259 u, i.e. n^3 R with R ~ n^2 hub pairs) - at 1024 it would be ~2000 B. K4 in the literal factorized form is impossible; a memoryless / regenerated kappa4 (504aldo's lambda law) is the only affordable version, and it already gives V29 its 1.7x over ARC k3-simple at 1024.
- ACCURACY LADDER so far (256 wide): V29 = K3 simple; K3 augment (dense) is 1.5x better in MSE for 4-9x the FLOPs. At 1024 V29's fitted kappa4 regeneration already buys 1.7x over K3 simple (2.13e-8 vs 3.7e-8), i.e. V29 sits at about 'K3 augment' accuracy at K3-simple cost. The leaders (1.64e-8) are another 1.3x below that - the K4 rung (run 5) will show whether that is 'K4 accuracy'.
- COST STRUCTURE (ARC counter, factored K3): exactly half transport (W @ factor on three legs) and half slices ((A o P) P^T); both are n^2 R with R = n x (#births). No other term matters. So any 10x saving must come from R (fewer/lower-rank births) or from replacing the per-source Khatri-Rao slices - the same wall pilots A-F hit from the other side.
- The paper's factorized K=3 chain, taken literally, costs 1.49 B at width 1024 (it is 'linear in width' only in the hub-tensor bookkeeping; the elementwise products are still n^3-ish). 504aldo's chain does the same order (K=3 + memoryless kappa4) 5.9x cheaper AND 1.7x more accurately: his age-gated factor ranks (384/224), shared basis and kappa4 regeneration are worth ~6x over the reference implementation.
- Consequence for the leaderboard question: the leaders (133-165 u) are not 'ARC's reference at a better budget'; whatever they run is 10x cheaper than the ARC K=3 chain and 1.7x more accurate than V29. This narrows the search: the formulation is neither the reference hub-tensor chain nor 504aldo's per-source factorization.

- Pending from run 2: does AUGMENT (kappa4-corrected) K=3 beat V29's raw at any cost? If ARC's augmented K=3 reaches < 1.9e-8 raw, the leaders' accuracy is 'reference-augment accuracy' and the puzzle is only cost; if not, they carry something the reference does not.

K3j. Research pilot H (21 Sep): the shared-basis ranks are at their optimum, and a pooled core cannot beat the stacks

Why this matters: a pooled Tucker core of the old sources (one r x r x r core in the shared basis, all sources summed) has a per-layer cost that scales as n r^3 / 2 (slices) + n r^3 / 2 (one join per layer) + 3 r^4 (rotation), i.e. independent of the number of old sources but cubic in the rank, whereas 504aldo's coefficient stacks cost 4 n^2 r per source-layer (linear in r). The core route wins only at r <= ~150 (break-even r^2 = 4 n k). So the question is purely: what does the chain lose at r = 96-160? Measured locally (knob_test on the seed-7 MLP, drift = MSE against the full chain, which adds to the raw 2.13e-8):

knob	util	drift MSE vs full chain	estimated score (raw + drift) x util	verdict
reference (V29, MIN 32)	0.2526	0	5.38e-9	-
R_OLD2 224 -> 160	0.2457	+1.2e-9	5.53e-9	worse
R_OLD2 224 -> 128	0.2417	+3.0e-9	5.88e-9	worse (Kaggle: 192 gives +1.6% raw)
R_OLD2 224 -> 96	0.2390	+7.0e-9	6.77e-9	much worse
R_OLD 384 -> 320	0.2376	+1.6e-9	5.44e-9	worse
R_OLD 384 -> 256	0.2224	+6.3e-9	6.13e-9	worse
R_OLD 256 + R_OLD2 128	0.2110	+8.75e-9	6.32e-9	worse

- Drift grows faster than the cost falls at every rank: the stacks are at their score optimum (504aldo's ladder confirmed with the adjusted-score arithmetic).
- Pooled core at r = 128 for tier 2 would save ~16 u (-6% util) but costs +3.0e-9 raw (+14%): net worse. At r = 160: -3% util for +5.6% raw: worse. CLOSED: no tensor-network re-basing of the old tier beats the stacks; the old tier is irreducible along the rank axis as well as along the birth-column (K3h), surrogate (K3d), aggregate-state (K3e) and recursion (K3g) axes.
- Structural fact used for the round-3 consultation prompt (ROUND3_prompt_propagator_structure.md): from ARC's from_dstensor the born tensor is Sym(sum_r e_r x e_r x a_r), so every source's two P legs are the propagator T_b = M_l ... M_{b+1} itself and the A leg is T_b A_b; the whole old tier is sum_b (T_b o T_b) A~_b^T + 2 (T_b o A~_b) T_b^T with T_b = T_c T_{b->c}. V29 already exploits the free age-1 P leg (= W).

K3k. Research pilot I (21 Sep): 'a different closure with compensating errors' - NO (Kaggle, 10 baked-truth MLPs)

Hypothesis (Astra 6 Extra High Q3, GPT Extra Max H3): the leaders drop the old tier and retune the closure so that its errors compensate. Test: V29 with the old sources' contributions zeroed (V30_NOOLD) and every closure knob swept - kappa4 regeneration scale, its adaptive exponent BETA, the young-source amplitude, the age gate - on 10 mini MLPs against the baked 1e9-sample truth (kernel whest-runner v5; cost unchanged because the zeroed contributions are still computed).

variant (old tier removed)	raw final MSE	vs base 2.303e-8	reading
noold (age gate 4)	7.43e-7	32x	the missing content
lambda scale 0.5 / 1.5 / 2.0	7.48e-7 / 7.45e-7 / 7.56e-7	flat	kappa4 regeneration cannot stand in
BETA = 0 (frozen table)	7.42e-7	flat	-
young amplitude (first test, FLAWED: gain compounded per layer) 1.15 / 1.3 / 0.85	2.86e-6 / 4.5e-5 / 1.44e-6	worse both ways	superseded by the corrected test below (K3o audit)

variant (old tier removed)	raw final MSE	vs base 2.303e-8	reading
young amplitude gamma, one-time at the slice stage (corrected, v30comp2) 0.9 / 1.1 / 1.2 / 1.4	8.70e-7 / 6.83e-7 / 6.98e-7 / 9.91e-7	shallow optimum -8% at ~1.1	no rescaling of what is kept mimics what is dropped
age gate 5 / 3 (keep one more / one fewer age)	4.89e-7 / 1.10e-6	x0.66 per extra age	monotone: every age carries ~1/3 less than the previous, so ~12 ages are needed to reach 2e-8

- CLOSED: no closure-side compensation exists within 1% - consistent with parity (K3b: the old contributions have zero conditional mean given the even statistics every closure is built from) and with the flat per-birth sensitivity map (K3c).
- The age-gate series gives a clean decay law for the omitted content: MSE(noold at age g) ~ 7.4e-7 x 0.66^(g-4). Useful for the write-up: it says the old tier's content is spread geometrically over ages, not concentrated in the youngest old sources.

K3l. Round-3 consultation (21 Sep, GPT-5.6 Sol high + Astra 6 Max) and its two falsifiers - both negative

Round 3 asked one precise question (ROUND3_prompt_propagator_structure.md): with the explicit structure 'both P legs of every source are the propagator T_b, the A leg is T_b a_b', is there a state with <= 1.2 u per source-layer? Both models: no exact route; the algebraic directions (shared squared propagator, Wick chaos in the current row, backward response, Mori-Zwanzig memory) all re-create the n(n+1)/2 symmetric-square coefficient space whose rank (~250) is already measured. Each offered ONE new approximation with a falsifier; both were run within the hour on the existing dumps.

proposal	mechanism	cost claim	falsifier result (seed-7 MLP, layers 10 / 14)	verdict
Cubic source-index coding (GPT)	pool the m old sources with i.i.d. codes z_s (E z = 0, E z^3 = 1): the slice functional is cubic, so E_z Phi(pooled legs) = sum_s Phi_s exactly; q channels, thin rank-1 part computed exactly	2.04 u per channel per layer, independent of m	relative error of H: q=1 388% / 527%, q=2 277% / 412%, q=3 254% / 386%; hybrid 2 exact + 1 coded 128% / 335%; even the 32-channel mean is 83% / 130% off (1/sqrt(q) law holds, so the single-channel variance is ~m x the signal)	DEAD: cross-source interference is as large as the signal
Birth-operator compression (Astra, GPT)	a_b ~ diag(d) + U V^T with k <= 32-40; the A leg becomes T diag(d) + (T U) V^T: one transport + one slice product	1.03-1.19 u per source-layer	the birth operator is a_b = diag(w1) C_off (verified: |A_s - P_s a_b| / |A_s| = 1e-5 at every slot and layer): ZERO diagonal, flat spectrum (rank 32 holds 21% of the energy at layer 0, 92% at layer 14; r99 = 310-809). Aggregate H error: k=16 85% / 74%, k=32 73% / 57%, k=64 54% / 37%, k=128 37% / 26%, k=256 29% / 21%, k=384 22% / 16%	DEAD: at k=384 (dearer than the dense leg) still 16-22%

- Two exact structural facts for the write-up: (1) a source's A leg is the propagator applied to the off-diagonal post-ReLU covariance of its birth layer, A_s^(l) = P_s^(l) diag(w1_b) C_off^(b), exactly, through confinement and all; (2) the rank-1 partial-trace ('thin') part is only 1.1% (L10) / 1.8% (L14) of the old tier's norm - the fluctuation IS the old tier.
- Depth trend of the birth operator: the energy in the top 32 directions of C_off rises from 21% (layer 0) to 92% (layer 14) - the deep covariance approaches the correlated fixed point - but 2% accuracy would need rank >> 256 at every depth.
- Both models now agree with the measured picture: no cheaper state within the K3 source algebra; the leaders' mechanism is not identified by any of 10 models over 3 rounds. Scripts: probe_srccode.py, probe_birth_dump.py, probe_birthcomp.py; dump birth_dump.npz.

K3m. Own line (21 Sep): adjoint / linear-response contraction of the slices - the sensitivity IS low-rank, but the response is carried by near-range paths

Idea (not proposed by any of the 10 models): the two slice products of a source are only needed contracted with the final mean's sensitivity to the covariance, dm = g o rowsum((J~_l D21) o J^_l), and J^_l P_s = T_{b->16} is a fixed per-source factor; if the sensitivity J_{l->16} = W_16^T D_15 ... D_{l+1} W_{l+1}^T is low-rank for early l, the products cost n^2 rho_l instead of n^3 and no D21 matrix is ever formed.

measurement (seed-7 MLP)	result
rank of J_{l->16} for 99.96% energy (probe_adjoint_rank.py)	l=15: 944, 14: 595, 13: 497, 12: 428, 11: 373, 10: 331, 8: 275, 6: 234, 4: 204, 2: 182, 1: 173 (sigma_max/sigma_median 2.5 -> 3.8e6)
cost model with per-(source, layer) choice of the cheaper scheme	265 -> 200 dense products (young tier -31%) before a ~12 u two-pass overhead: at best -15% util (rank ~30 -> ~18)
validation of the response model INSIDE V29 (probe_jac_val.py): random dC injected into C at layer li, exact dm vs. the first-order model with Monte-Carlo Wick coefficients	li=14: R^2 0.9999, coefficient 1.00; li=10: R^2 0.9957 with every path coefficient 1.00 (linearity: ratio 2.000 for 2x dC)
decomposition of the response at li=10	final-variance path (the low-rank one) explains only 14%; the variance -> E[relu] -> mean shifts at the NEXT layers (l' = 12, 13: corr 0.55, 0.53) carry the rest, and their sensitivities K_{p->l'} are near-range and full rank
exact effect of dropping the old sources' slices at one layer (dropold, probe_adjoint_val.py) vs. the same response model	L10: R^2 0.50 (all covariance paths), L14: R^2 0.09 - although the covariance response is exact, so >= 90% of the old slices' effect at layer 14 travels through a non-covariance channel (D21 -> newborn feeds are only ~10%: switching them off changes dm by 10%); switching the kappa4 channel off does not change this (R^2 0.11)
where the old sources act, by slice type (probe_d21late.py; MSE vs the full chain when only that part is removed at that layer)	old D21 (n^3 slices) / old D3 (n^2 diagonal): L10 3.66e-8 / 3.19e-8; L12 3.09e-8 / 5.76e-8; L13 1.84e-8 / 5.97e-8; L14 1.30e-8 / 7.81e-8 - the slices' weight falls toward the output, the diagonal's rises; every entry exceeds the error budget, so neither can be skipped at any layer (V19's D3-only trim cannot be extended to old sources)

- DEAD for cost: the low-rank part of the sensitivity carries 14% of the response; the rest needs the full-rank near-range Jacobians, i.e. the forward propagation of D21 that V29 already does at 2 products per layer for all sources at once.
- KEPT for the write-up: a validated first-order response model of the K3 chain (R^2 0.996-0.9999 with unit coefficients from Monte-Carlo Wick coefficients); the finding that the old sources' late-layer influence goes mainly through their DIAGONAL (D3 -> the newborn's M-leg diagonal -> the last pre-activation's kappa3_iii -> E[relu]), not through the covariance; and the per-layer D21/D3 importance map.

K3n. Round 4 (22 Sep, GPT-5.6 Sol GitHub audit): the angular/spherical input (LIVE) and the query-space row skeleton (dead)

GPT's fourth report searched public GitHub repositories of other participants and of tensor-compression libraries. Two testable items came out; the rest of the audit confirms our dead list independently (a Phase-8 public repo measured terminal-only / all-depth direct backward contraction at 3.2e-6 / 4.7e-7 raw - our adjoint and late-reformation results).

item	what it is	test	result
Angular / spherical input (barnobarno666 Phase-8 repo: -7.9% raw on an independent K3 chain at 1024 x 16)	X = (R/sqrt n) Y, Y uniform on the sphere, R ~ chi_n independent; the bias-free ReLU net is degree-1 homogeneous so E[h(X)] = a1 E[h(Y)], a1 = E[R/sqrt n] = 0.99975589. Y has C = I, kappa3 = 0, kappa4 = c Sym3(I x I) with c = -2/(n+2). In V29's harmonic core representation this is a previous-layer core g_prev = -6/(n+2) (V29's own projection gives exactly this), var_prev = 1, lambda_prev = g_prev: layer 0 runs V29's kappa4 terms with dG = g var, wk431 = (m/2) g C_off (new term-program mode 3); output x a1.	built as scratch_est/estimator_v31ang.py (make_v31ang.py); local: cost +0.02%; the injection alone shifts the means by a least-squares factor +2.450e-4 vs the identity's +2.442e-4 (0.3% agreement) - implementation verified; net change after a1: 1.1e-5 rms	Kaggle, 10 baked-truth MLPs: base 2.3033e-8 -> ang 2.2911e-8 (-0.5%, 5/10 wins, per-MLP ratios 0.977-1.012: a statistical wash); the a1 identity is confirmed by the control ang_noa1 = 8.02e-8 (+5.7e-8 = (2.2e-4)^2). VERDICT: correct and free, but V29's fitted kappa4 regeneration and online mean correction already absorb what gave the other chain -7.9%. Graded as #331787: raw 2.132e-8 -> 2.108e-8 (-1.1%), score 5.137e-9 - the first accuracy gain over the public V29 in this project.
Query-space row skeleton (ISDF / THC analogue)	the contracted old query U (n x r) has 2% rank ~256; if its LEFT basis were available without computing U (from an anchor source or transported from the previous layer), only m interpolation rows per source would be needed (m r n instead of n^2 r): old tier ~4-6 u/layer	probe_isdf.py on L9/10/13/14 dumps: relative error of the aggregate H- and T-queries at m = 128..384 with (1) oracle, (2) one-source anchor, (3) two youngest, (4) previous-layer basis transported by W or W^T	DEAD: oracle 2.2% / 3.6% at m = 256, but anchor 56-71%, two sources 37-59%, transported previous-layer basis 79-94% (the left subspace does not transport). KILL gate met at every m.

K3o. Audit of the negative results (22 Sep): were the failures real or ours?

Every probe was re-read for hook mechanics, conventions and sanity checks that would expose a bug. Verdicts:

experiment	what could have gone wrong	check	verdict
Layer dumps (used by pilots A, C, D, loopholes, source coding, birth compression, ISDF)	wrong buffer / wrong slot order / stale legs	A_s = P_s a_b holds to 1e-5 at every slot and layer with the true birth matrices (probe_birthcomp); legs reconstruct from the Qc factors to 1e-6	SOUND
Pilot B sensitivity (sens_probe: dropold / dropslot / keeplast)	zeroing legs could leak into transports or joins	old slots are re-formed from factors each layer (verified in the transport block); young zeroing persists only in dropslot, by design; keeplast vs Kaggle no-old (4.9e-7 vs 7.4e-7) consistent	SOUND
Pilot C TensorSketch (790% error at d = 2048)	looked too large - a correct degree-2 sketch was expected at ~70%	same hashes on both sides (verified); the entries (A o P)_ij are ~1/r of the row-norm products, so the per-entry error is r / sqrt(d) = 384 / 45 = 8.5 - matches the measured 7.9	SOUND (my expectation was wrong, not the code)
Pilot D aggregate states (Gram trick, cross-source SVD)	algebra slip in the (ab)|c unfolding	M^T M = P d(w2) [(A^T A) o (A^T A)] d(w2) P^T re-derived; the 2%-rank ~250 agrees with the oracle SVD of U in two independent scripts (probe_query_analyze, probe_isdf: 2.2% at 256)	SOUND
Pilot E slice recursion (R^2 0.03)	a wrong transport operator gives R^2 ~ 0 spuriously	both W and W^T conventions were fitted; the transport term's coefficient came back 1.0; the algebra predicts it: the slice transport is the j = j' diagonal of a double sum whose n^2 cross terms are ~sqrt(n) = 32x larger in norm (3% energy)	SOUND
Pilot F column screening	mask not persisting / applied to the wrong index	mask applied to the birth (column) index of both legs; young legs carry it through the transport, old legs are re-masked each layer (fixed at birth)	SOUND
Pilot H rank ladder, Strassen variants	env knobs silently ignored	util changed with every knob; Kaggle rold2_192 (+1.6% raw) matches the local drift trend	SOUND
Pilot I compensating closure - lambda scale, BETA, age gate	knob not applied	each knob moved the raw MSE (1.0 vs 0.9 lambda: 2.3058 vs 2.3119e-8 on the base)	SOUND

experiment	what could have gone wrong	check	verdict
Pilot I compensating closure - YOUNG AMPLITUDE	the V30_YOUNG_GAIN hook multiplied the young legs IN PLACE at every layer, so the gain compounded through the transports (1.15 -> 1.75x, 1.3 -> 2.9x on the oldest young source)	re-read of v30comp.py	FLAWED - a much harsher family than the one-time amplitude gamma the hypothesis meant. REDONE: v30comp2.py scales the young legs by gamma^(1/3) at the slice stage only (cubic terms -> exactly gamma) and restores them; validated locally (gamma = 1 bit-identical; gamma = 1.1 -> rms change 1.7e-4). Kaggle sweep noold + gamma (10 baked-truth MLPs): 0.9 -> 8.70e-7, 1.0 -> 7.43e-7, 1.1 -> 6.83e-7, 1.2 -> 6.98e-7, 1.4 -> 9.91e-7: a shallow optimum near gamma ~1.1-1.15 worth -8%, still 30x the base error. VERDICT UNCHANGED on a correct test.
ARC reference notebook	weight convention (h @ W vs torch Linear) / layer indexing	K2 result scales with the kit baseline (3.9e-7 at 1024 -> 7.8e-5 at 128 ~ n^-2.8); a wrong convention gives ~1e-1; act{L-1} = 16th post-activation checked	SOUND
V29 at 256 with riders forced	the 1024-fitted online mean correction is applied off-shape	known caveat: V29's 1.99e-6 at 256 is an upper bound of its error there; the conclusion (V29 in the K3-simple class at 256) does not depend on it	SOUND with caveat
Source coding	code moments / bias	E z = 0, E z^2 = E z^3 = 1 verified analytically; the 32-channel mean error falls as 1/sqrt(q) (unbiasedness)	SOUND
Adjoint line	wrong response model	validated INSIDE the chain: random dC -> R^2 0.9999 with unit coefficients	SOUND
D21/D3 split (probe_d21late)	the second _dslices call corrupting state	first-call outputs copied before the second call; leg scaling happens after the dslices; the two parts sum to about the dropold total (1.3e-8 + 7.8e-8 vs 8.2e-8)	SOUND

K3p. Precision (22 Sep): float32 rounding is not part of the error

Companion (A5 closed): the UNCONFINED chain (V21_NO_CONFINE=1, every old source dense, float64) differs from V29 by MSE 7.6e-10 - the whole shared-basis/nested-tier approximation costs ~3% of the raw error while saving 15% of the cost (util 0.287 -> 0.244). So the old tier's representation error is only ~3% of V29's error: there is no large systematic cancellation to exploit in it, and the remaining 97% is the K3+memoryless-kappa4 truncation itself.

V29 run with every array in float64 (scratch_est/estimator_f64.py, V32_F64=1; billed 2x, 10.7 GB): final-layer drift vs the float32 chain MSE 2.9e-13 (rms 5.4e-7 against an error rms of 1.5e-4). The chain's 2.1e-8 is entirely truncation/representation error; there is no precision lever, and float32 is the right choice for cost.

Re-tune under the angular chain (Kaggle, 10 MLPs, runner-c): ang 2.2911e-8; lambda x0.9 2.2899e-8 (-0.05%), lambda x1.1 2.3267e-8 (+1.6%), BETA 0.8 2.2971e-8 (+0.3%), BETA 1.2 2.2895e-8 (-0.07%). The fitted constants stay at their optimum after the injection; the 1% accuracy margin cannot buy a rank cut (R_OLD2 192 alone costs +1.6%), so GPT's 'angular rank-cliff recheck' is moot. Round 4 closed: one free -1.1% (shipped as #331787), everything else negative.

K3q. Astra 6 GitHub audit (22 Sep): no new method; a correction of the source algebra and one diagnostic (done)

Astra's audit of public code (ARC, V29, TensorLy, pyttb, opt_einsum, AlphaTensor, TT cross-interpolation, other participants' repos) found no implementation near the target and withdrew its own round-3 1.19-u proposal. Its substantive point: the born third cumulant has TWO families - the covariance-path family AAP (legs A = T a_b, P = T; V29's H and T2) and the repeated-index family MPP (the M leg, V29's T3/T4 with M = P diag(s) + 3 A diag(e) + Z L^T); our round-3 prompt described only the MPP-type from_dstensor summand. The birth-compression test was nevertheless run on the true AAP birth a_b = diag(w1) C_off, so its verdict stands. Diagnostic asked for (probe_query_full.py + probe_aap_mpp.py, old sources at layers 10 and 14):

layer	|AAP| / |total|	|MPP| / |total|	cos(AAP, MPP)	fluctuation parts |fl AAP| / |fl MPP| (cos)	diagonal D3: MPP / AAP
10 (6 old)	0.36	0.75	+0.57	0.61 / 0.67 (+0.22)	2.6x
14 (10 old)	0.37	0.74	+0.56	0.57 / 0.50 (+0.29)	2.7x

- Both families are essential (dropping either costs 37-75% of D21) and they reinforce (positive cosine): no cancellation to exploit; the M-leg family carries the diagonal (consistent with K3m's D3 finding). Youngest old sources are AAP-heavy, the oldest MPP-heavy.
- No cost consequence: V29's MPP readouts reuse the same Hadamard products (P o P, A o P) plus thin terms, which is why its two fused contractions serve both families. CLOSED as a diagnostic.

K3r. Map of method families (22 Sep): where each one dies against 2e-8 at <= 0.16 B

family	status	where it dies
Monte Carlo over inputs (antithetic, control variates, QMC, cubature)	measured / known	0.1 B buys ~6500 samples -> MSE ~5e-5; needs a 2000x variance reduction; the K3 chain has no sample-level counterpart to serve as a control variate (blend weight ~0.002, C1/C2); a cov-prop control variate gave -25% for one participant
Multi-level MC with a cheaper surrogate network	derived	random Gaussian W has a flat spectrum: no cheap surrogate correlates with the true map
Cumulant propagation K2/K3/K4 (ARC family; V29 = K3 + memoryless kappa4)	the baseline	the sum-of-sources form is its implementation; every alternative representation of the K3 tensor (Tucker/TT/pooled core, sketches, aggregate states, adjoint contraction, slice recursion, birth compression, source coding) measured dead (K3b-K3n)
1/n perturbation theory, NNGP finite-width corrections (Yaida; Roberts-Yaida-Hanin)	derived	algebraically the same expansion: its diagrams are the kappa3/kappa4 terms of the chain
DMFT / replica / path integral / kinetic closures	derived + parity theorem	ensemble statistics over W; the realized odd fluctuation that carries the deep content has zero conditional mean (K3b)
Learned estimator from the public truths	partly open	weights -> means directly is hopeless (16M inputs, 1100 samples); chain + learned residual correction is the practical form (V33 study running); parity bounds what even features can recover
kappa4 with memory (per-source kappa4 legs, ARC's augmented K3)	measured by 504aldo (F51/F58/F64, community post #9/#17)	-9% raw at 0.5-1.0 B for the RESTRICTED harmonic channel Sym(G x metric) (F51/F58/F64); the unrestricted fourth cumulant is unmeasured at 1024 (n^4-class; K3i: K4-simple is 1.6x over K3-simple at width 128)

- Consequence: even the full K3 + per-source-kappa4 chain (the paper's augmented K=3, 4-8x over budget) reaches only ~2.0e-8; the leaders' 1.64e-8 at half of V29's cost is beyond every family in this table. Their mechanism is not a member of the known families - the honest conclusion of 4 rounds, 11 models and ~25 measurements.

K3s. Round 5 (22 Sep, GPT-5.6 Sol 'Beyond sum-of-sources'): 14 alternative families - assessment and the two that were measurable

The report lists 14 distinct formulations (TAP/AMP cavity, gate-field/dichotomized Gaussian, Koopman/KME dictionaries, TT-PCE input chaos, functional TT/TT-cross, polytope integration, EP over gates, characteristic functions, copula + rich marginals, adaptive GMM, quenched RMT deterministic equivalents, saddlepoint/CGF, Stein-kernel state, angular harmonics) with a five-gate ladder. Assessment against our measurements:

family	verdict	reason
TAP / AMP / cavity (forward)	dead by argument	Onsager reaction terms need loops (inference through the same W); a feedforward net has none - z = W^T h with W independent of h; the quenched fixed-W law is exactly what the K2/K3 chain propagates, and AMP state evolution describes the TYPICAL W (worse)
characteristic-function state	dead by argument	the mean needs only the marginal CF of z_i at direction W_i, whose log-expansion is the cumulant chain; keeping the full CF is the n^3 wall in disguise
saddlepoint / CGF readout	dead by magnitude	the truncation error of E[relu(z)] from (mu, sigma, kappa3, kappa4) is ~1e-5 per neuron (MSE ~1e-10); the chain's error is in the propagated kappa values, not the readout
gate field at pairwise order; Koopman/KME, Stein state, quenched-RMT dictionaries	= K2 / = learned closure	pairwise gate statistics are the arc-cosine kernel; observable dictionaries fitted on public MLPs are the V33 residual study (running), which will include W-observables
GMM split/merge, polytope beams, angular harmonics	dead (public failures / combinatorial)	-
TT-PCE / functional TT-cross of f_j(x) in input coordinates (GPT's 'best representation gamble', gate 3)	MEASURED DEAD (probe_tt_rank.py, teneva)	width 32: even ONE ReLU layer (a ridge function) at TT ranks 6-16 gives 60-500% integral error where MC at the same evaluation count gives 0.5-1%; ranks explode past 100 when allowed to grow (OOM in the norm); depth makes it worse
copula + rich marginals (gate 4 oracle) and per-layer one-step closure floors	measured next (K3t)	teacher-forced one-step test on one 1024 x 16 network

K3u. Round 5b (22 Sep, Astra 6 'Beyond sum-of-sources'): six alternatives with executed cost gates - all closed

alternative	Astra's measurement / derivation at n = 1024	status
activation-region / Gaussian polyhedral integration	one line restriction alone: 2,138 segments after 16 layers, ~62 u in float64 (SyReNN-style); full enumeration 2^1024 cones	dead (cost)
input-space polynomial chaos	degree-2 state P = 525,825 -> 513 u per linear update (262 u with Strassen); diagonal-quadratic variant retains 0.29% of a first-layer neuron's second-chaos energy	dead (cost / accuracy)
function-space TT quadrature	rank-2 finite-grid probe (16 x 16 Rademacher split, all outputs): mean MSE 2.2e-6 = 112x the target; consistent with our width-32 TT-cross probe (K3s)	dead
global moment / SOS optimization	first moment matrix d = 17,409: one dense Cholesky 819 u; the cheap relaxation is loose already at layer 1 (0 <= E[Y] <= sigma/2)	dead
certified branch-and-bound expectation bounds	two affine envelopes: 32 u per full-depth cell; a few cells fit the budget, tolerance 2.8e-4 RMS width undemonstrated	dead
fixed-weight EP / ML-VAMP	SVD primitive 208 u for 16 weights; without observations the Gaussian-message forward pass reduces to Gaussian moment propagation (our K2 floor)	dead by argument

- Two corrections adopted: (1) 504aldo's F51/F64 measure the RESTRICTED harmonic kappa4 channel Sym(G x metric); the accuracy of the unrestricted fourth cumulant at 1024 is unknown (our K3i ladder at 128 gives K4-simple 1.6x over K3-simple, comparable to V29's 1.7x over K3-simple at 1024); so 'full kappa4 buys only 9%' is not established - it is simply unaffordable (n^4-class). (2) The MC calibration: from the adjusted plateau 1.2e-6, 160 u buys raw ~7.7e-6, i.e. a 384x variance reduction is needed (our earlier 5e-5 was a different calibration).
- No new experiment from this round: Astra executed its own falsifiers. Together with K3s, every non-cumulant family has now been closed by measurement or by an explicit cost gate.

K3v. Residual-structure study V33 (22 Sep): the chain's residual is unpredictable from any cheap per-neuron feature

Setup: V31 run on all 100 mini MLPs on Kaggle (kernels whest-dump-a/b, kaggle_estimators/v33dump.py, kaggle_dump/dump_driver.py), dumping per layer and per neuron 19 features (V29's 13 online-correction features + dG, pk2v..pk4v, g4row, delta) together with the baked truths; offline 5-fold cross-validation over MLPs (residual_fit.py). Note: V29 ships with its own 13-feature ridge correction switched OFF (V17_NO_CORR=1), so this axis was untested in the shipped chain.

model of the final-layer residual (truth - prediction)	CV residual MSE	fraction of baseline 2.2049e-8
ridge, 19 layer-15 features	2.1984e-8	0.997
ridge, 38 features (layers 15 + 14)	2.1987e-8	0.997
ridge, quadratic expansion (189 features)	2.1889e-8	0.993
gradient boosting (HistGBM, 38 features)	2.1930e-8	0.995
per-MLP constant offset	-	explains 0.44% of the residual variance

- Every single-feature correlation with the residual is <= 0.04 (largest: sigma*phi 0.042, phi 0.039). The residual is noise with respect to all even per-neuron statistics - the parity theorem's prediction (the missing content is odd in the weights) confirmed on 102,400 output neurons.
- CLOSED: no learned/fitted per-neuron correction can recover more than ~0.5% (and that much only in-sample-ish); the Koopman/KME, Stein-state and quenched-RMT-dictionary families of round 5 are closed by the same measurement (their features are even per-neuron observables).

K3t. One-step closure floors and the copula oracle (22 Sep, Kaggle GPU, 1024 x 16, 2e8 samples per layer)

Teacher-forced: at layers 4/8/12/15 the exact post-activation h_{l-1} is sampled from the network and the next-layer mean E[relu(W^T h)] is compared with three one-step closures (kaggle_onestep/onestep.py): A = Gaussian closure of h with its exact mean and covariance (the K2 one-step); B = Gaussian copula with the EXACT marginals of the previous pre-activation (GPT round-5 family 9, oracle version); C = Gaussian previous pre-activation pushed through the exact ReLU per sample (the K2 chain's own one-step). RMS errors of the next-layer mean; MC standard error <= 4e-5.

layer	A Gaussian h	B copula + exact marginals	C Gaussian z_prev -> ReLU
4	5.2e-4	1.14e-3	6.2e-4
8	5.1e-4	9.2e-4	6.5e-4
12	4.6e-4	8.1e-4	6.3e-4
15	4.5e-4	7.5e-4	5.7e-4

- The intrinsic one-step non-Gaussianity error at width 1024 is ~5e-4 RMS per layer - three times V29's FINAL error (1.5e-4): the K3 + kappa4 machinery recovers most of what any Gaussian-type closure loses at every layer.
- COPULA FAMILY DEAD: oracle marginals with Gaussian dependence are WORSE than the plain Gaussian closure at every layer (B > A). The non-Gaussian DEPENDENCE (odd, tensorial) is the whole content; marginal richness is worthless without it - the parity theorem seen from the distribution side.
- Nested-tier knobs (Kaggle, angular base 5.583e-9): AGE_OLD2 5 / 6 / 8 / 9 -> 6.32 / 5.77 / 5.59 / 5.62e-9; QPASS2 1 / 3 -> 5.61 / 5.59e-9. At optimum; closed.

K3w. The K3 -> K4 accuracy gain grows with width (22 Sep, ARC reference chains, 2e9-sample GPU truths)

width (2 networks)	K2	K3 simple	K4 simple	K3 / K4 ratio
96	2.69e-4 / 5.33e-4	1.11e-4 / 3.75e-5	8.63e-5 / 3.64e-5	1.29 / 1.03 (mean 1.15)
128	7.79e-5 / 2.22e-4	1.23e-5 / 1.96e-5	7.60e-6 / 1.31e-5	1.62 / 1.50 (mean 1.56)
160 / 192	1.07e-4, 6.2e-5 ...	9.9e-6, 2.8e-5 / 5.8e-6, 5.1e-6	factored K4: OOM on 15 GB (dense K4 at 160 queued)	-

- The fourth-cumulant rung is worth more as the width grows (1.15x at 96 -> 1.56x at 128). V29's restricted, memoryless kappa4 regeneration is worth 1.7x over K3-simple at 1024; the leaders are 2.25x over K3-simple. If the trend continues, the UNRESTRICTED kappa4 at 1024 is worth more than V29's regeneration - i.e. the leaders sit in the full-kappa4 accuracy class (the Astra correction of K3u made this a live question). What they cannot be doing is ARC's n^4 implementation of it; the open problem is therefore 'kappa4 accuracy at K3 cost', not the old tier.

K3x. The kappa4 teacher (22-23 Sep): V29's lambda is a fitted COMPENSATOR, not an approximation of the true law

A Monte-Carlo fourth-cumulant teacher (kaggle/k4, k4teacher1.py: single-pass raw moments -> central cumulant slices, validated against a two-pass implementation to 1e-5 on an identical sample stream) measured the TRUE pre-activation slices kappa4(i,i,j,j), kappa4(i,i,i,j) and kappa4(i^4) per layer at 1024 x 16, on one synthetic network (8 layers, 6e7 samples) and on public MLP #0 (all 16 layers, its baked 1e9-sample truth available).

measurement	result
V29's model of the (2,2) slice ((dG_i + dG_j) c/6)	R^2 0.94-0.98 at every depth: already accurate
V29's model of the (3,1) slice (lambda_l C_off, one scalar per layer)	R^2 0.35-0.60; adding the kappa4 diagonal terms -> 0.55-0.71; adding the chain's OWN third-cumulant slice (D21) -> 0.74-0.80. Same pattern and depth trend on both networks: a real law, not a per-network fit
what the true law weights the (3,1) slice by	NOT a per-layer scalar but the per-ROW ratio kappa4(i)/var(i); the fitted coefficient falls smoothly 1.00 -> 0.63 with depth
504aldo's lambda_l versus that law	lambda_l equals the MEAN of b_l x kappa4(i)/var(i) to within 2% (6.8e-3 vs 6.9e-3 at layer 15): the fitted constant has a derivation
per-row lambda in the chain (V35, Kaggle, 10 baked-truth MLPs)	blend 0 / 0.25 / 0.5 / 1.0 / 1.5 -> raw 2.2911 / 2.2923 / 2.2933 / 2.2958 / 2.2988e-8: MONOTONICALLY WORSE

- CONCLUSION (corrected in K3y): making the kappa4 channel more faithful to the true RAW cumulant slices makes the final mean worse, monotonically. The reason, established publicly the next day (F105), is that the chain propagates the r = 1 matrix CORE, not the raw slices: the scalar lambda fits that core at R^2 >= 0.95, so our per-row law, fitted to a different object, perturbed an accurate model. The same object mismatch explains the oracle run (6-15x worse).
- Consequence for anyone improving this chain: a channel-by-channel fidelity argument is invalid; only end-to-end measurements count. The kappa4-law line is CLOSED for the objective, and kept as a quantified negative result plus one positive derivation (the meaning of lambda).
- Scripts: kaggle/k4/local/k4teacher1.py, experiments/chain/k4_law_fit{,2}.py, k4_law_coef.py, k4_oracle_run.py; variants in experiments/variants/estimator_v35row.py.

K3y. Open-source and community sweep (23 Sep): GitHub, Hugging Face, the AIcrowd forum and the literature

Scope: GitHub repository search through the API (queries whestbench, whest+estimator, arc+whitebox+estimation, cumulant+propagation, sorted by last update) with the recent commit history of every hit; the Hugging Face dataset and model APIs (author and keyword searches); the AIcrowd Discourse forum (latest 30 topics plus the full thread of topic 18218); and a web/arXiv search for 2026 work on cumulant propagation in random ReLU networks. Nothing in any of these sources reaches the Phase-2 frontier, but three items change what we know.

1. The kappa4 ceiling is now measured - by the author of the chain, in public

Forum topic 18218, posts #4 and #5 (22 Sep 2026). AndreasHad04 replicated 504aldo's F68 claim in their own chain and found that the law does NOT hold for the object they measured: regressing the off-diagonal (2,2) slice of post-activation kappa4 on the covariance through the origin, over 8 networks, gives R^2 0.9977 at layer 1 decaying MONOTONICALLY to 0.0454 at layer 15 (median over layers 8-14: 0.121 against C_pre, 0.071 against C_post; shuffled control 0.000; fitted lambda 0.319 -> 1.6e-3). 504aldo replied the same day and corrected the record (finding F105, commit 1a4083f):

- OBJECT: the chain's G is the r = 1 harmonic matrix core of the augmented chain, NOT the (2,2) slice of kappa4. Its off-diagonal comes from the (2,1,1) hub contraction and Sym(K31); the (2,2) slice enters G only through its row sums, on the diagonal.
- The shipped law is the PRE-activation one, G_pre_off = lambda_l C_pre_off, whose R^2 on 8 dumps is 0.48-0.52 at layer 0, >= 0.95 from layer 3 and 0.985-0.99 at depth. The 0.66-0.97 range quoted in the public post was the weaker post-ReLU fit on 2 dumps.
- THE CEILING (the number our oracle was being built to measure): in their lean chain, replacing G_off by lambda_l C_off gives 2.23e-8, the dense G channel gives 2.04e-8, and dropping the channel entirely gives 4.20e-8. A PERFECT kappa4 core is therefore worth about 9% of raw MSE - and the dense channel that achieves it costs 3 extra dense units per source-layer (F58/F64: the augmented chain runs at >= 0.95 x B). The kappa4 accuracy line is closed on cost grounds, quantitatively, without our oracle.

2. Correction to K3x

Our teacher measurement fitted the RAW cumulant slices, and our V35 variant modified wk431 - the (3,1)-shaped term - using a per-row law fitted to the raw (3,1) slice. F105 shows that the shipped object is the composite r = 1 core, for which the single scalar lambda_l is an accurate fit (R^2 >= 0.95 from layer 3). The correct reading of K3x is therefore NOT that lambda is a compensator absorbing bias, but that our per-row law was fitted to a different object from the one the chain propagates, so applying it perturbed an already accurate model - exactly what the monotone degradation (2.2911 -> 2.2988e-8) shows, and the same object mismatch that made the oracle injection 6-15x worse. The end-to-end negative result stands; the interpretation is corrected. Our independent finding that lambda_l equals the mean of the per-row ratio kappa4(i)/var(i) to within 2% remains a valid statement about the raw slice, not about the core.

3. What exists in public code and data

source	what it is	relevance
504aldo/whest-p2-cumulant-k3 (updated 22 Sep)	the public V29 chain plus a 100+ entry findings log; F105 added 22 Sep	our baseline, and now the source of the kappa4 ceiling above
tim8es/arc-whitebox (active 22 Sep)	a Phase-2 research repo with a hypothesis ledger and a frontier document; issues E007 / E010 are 'reproduce the public K3 + kappa4 frontier' and 'reproduce the legal V29 cost frontier after Strassen confirmation'	another team reproducing what we already ship; no result beyond V29 visible
anhminhzui-dev/arc-whest, agentaiventure-dot/arc-whest-estimator, itsjustmarsel/whest-teardown, SkyeNygaard/ARC-Whitebox, aarushitandon0, Oishi1029, ascender1729, violal1016	covariance-propagation entries, Kerdock/MUB cubature, teardowns and lecture notes	all in the 1e-7 to 1e-5 class: 20x to 1000x behind the current frontier
keenanpepper on Hugging Face (11 datasets, latest 13 Sep)	a supervised corpus: joint features and residual targets (10k nets), 500k mean targets, kappa3 tensors, and cumulant sketches of 16x1024 ReLU MLPs (1155 downloads). The jointfeats card states that marginal cumulants plateau a covariance closure at 5.2e-6 MSE and that adding kappa3-tensor contractions takes it to 1.9e-6 (width 256, depth 32)	the only serious learned-estimator line in public; its quoted numbers are Phase-1 scale and 100x behind the analytic chain, and our own 100-network residual study (K3v) found the per-neuron residual unlearnable
Forum topics 18214, 18216, 18219 (September write-ups)	exact bivariate ReLU covariance (1.32e-7); a zero-fitted-constant propagator with a rank-4608 CP third-cumulant channel (1.066e-7, rank 64 of 877); and a four-submission demonstration that below the 0.1 utilisation floor further compute reduction buys nothing	useful community results, all 20x or more behind; the budget-floor post confirms our own cost arithmetic
Literature (arXiv)	the ARC paper 2605.05179 remains the only work on this exact problem; nothing new since May 2026 on cheap high-cumulant propagation	no external theory to import

- NOBODY has published a method at the 133-165 unit / 1.64e-8 frontier. The leaders' mechanism remains undisclosed. 504aldo says the same from the other side (post #3, 14 Sep): they can see from the leaderboard that someone trimmed 7-22% of their chain's cost without losing accuracy, and do not know how. That someone is us (STRASSEN_MIN and STRASSEN_FIRST, Part K2), which our prepared forum reply answers.
- Effect of the sweep on the plan: the kappa4 accuracy line is closed by the author's own end-to-end numbers (a 9% ceiling at 3 units per source-layer), so no further kappa4 teacher work is warranted; the learned-estimator line is independently confirmed dead at our scale; and no public code or paper offers a representation we have not already measured.

K3z. Three own ideas after the public sweep (23 Sep): extrapolation dead, per-family Strassen and a joint knob search open

None of these came from a consultation round; they are what is left after the public sweep closed the kappa4 line.

- D - LAYER-DEPENDENT AGE GATE (dead, measured). V29 uses one age gate for every layer (AGE_OLD = 4), so every source gets the same four dense layers wherever it lives; our sensitivity map says the deep content matters 24x less at layer 5 than at layer 14, which suggested confining early-born sources sooner

(gate 2-3 up to a switch layer, 4 after it; estimator_v36age.py, with the tier-2 promotion clamped to one source per layer). Measured on the seed-7 network: e3s8 util -0.89% at drift 1.6e-9 (+7.6% raw, score x1.07); e2s8 -2.9% at 9.9e-9; e2s12 -7.5% at 1.4e-8; e1s6 -3.8% at 5.1e-8. Every schedule is net worse. MECHANISM (the flaw in the idea): confinement is not a per-layer perturbation but a PERMANENT state change - once a source is in the shared basis its representation error is carried into every later layer, including the sensitive ones - whereas the sensitivity map measured transient per-layer omission. It is also the wrong direction: early-born sources are the longest-lived, and 504aldo's own spectra find them the LEAST compressible.

- C - RICHARDSON EXTRAPOLATION IN RANK (dead, offline, zero cost). If the rank-truncation error obeyed p(r) = p_inf + C r^-a, two CHEAP runs could be combined to cancel the leading term and beat an expensive one. Tested on the stored predictions at R_OLD2 = 96/128/160 against the shipped 224: the fitted exponent is not stable (1.46 between 96 and 128, 2.07 between 128 and 160), and every extrapolation is WORSE than simply using the better of the two runs (3.3e-9 at the best exponent vs 1.2e-9 for r = 160 alone). Mechanism: each rank selects a different subspace, so the errors are not aligned in direction and do not cancel. This also kills averaging two cheap runs, which was the same idea in disguise.
- A - PER-FAMILY STRASSEN LEVEL (dead, measured). The hub and C_pre knobs are capped by min(knob, s_lev) in the code, so raising them alone changes nothing (util 0.2433 unchanged, identical buffer census); the shared-basis knob does work but the level-6 buffers cost 7.84 GB against an 8 GB cap for -0.74% util, with residual +31%. Both MIN = 4 and MIN = 6 give the same memory, so the rank-384 family alone is what allocates it. Not shippable. ORIGINAL NOTE: V29 sets one level for every family (V26_STRASSEN = 5) but keeps separate knobs for the hub products (V26_STRASSEN_HUB), the C_pre block family (V29_CPRE_LEV) and the shared-basis products (V28_STRASSEN_SB); only the last has ever been swept (SB = 6 with MIN = 4: util 0.2414 but residual +45% and 7.84 GB, dead). The memory census (K2) shows the 8 GB wall comes from the BATCH transport families (P = 343 / 2401, buffers of 300-660 MB), while the hub and C_pre pools are 34-52 MB, so raising only those two to level 6 may fit. Arithmetic is unchanged, so any util reduction is free.
- B - JOINT KNOB OPTIMISATION (queued). Every knob has been swept ONE AT A TIME (R_OLD, R_OLD2, AGE_OLD, AGE_OLD2, QPASS2, LAM_SCALE, BETA, Strassen). Their interactions have never been measured, and the tolerance law (extra MSE ~ 4.2e-6 eps^2) plus the per-knob cost table make a coordinate search on the SCORE (mse x util) well posed. Waiting on the 100-MLP evaluation harness, which resolves 0.3% effects instead of 1%.

K3aa. Round 6 (23 Sep): Fable 5 as a working subagent, unbiased column sampling, and the asymmetric contraction rank

Fable 5 was run as a subagent inside the repository (with the user's permission) on the round-6 brief. It built a validated first-order QUERY METRIC for the old tier (experiments/probes/probe_r6_query.py: the Gaussian-chain response of the final mean to a perturbation of the (2,1) slice at a given layer, physical constants, no fitting; against the exact chain R^2 0.9957 / 0.9999 for random dC at layers 10 / 14, and 0.961 / 0.989 for dropping the old tier). With it every compression can be scored by its actual downstream effect instead of a Frobenius norm.

- NEW STRUCTURAL FACT: the old tier acts on the final mean mainly through the DIAGONAL D3 path (n^2 per source); the expensive D21 contraction carries 13% of the drop-old effect at layer 14 and 56% at layer 10 (D21-only model R^2 0.07 / 0.44; D21 + D3 0.99 / 0.96). This explains why every LEG compression died faster than its saving: it hits the dominant, cheap D3 channel and the transported state, not just the costly contraction. Consistent with our own d21late split (K3m).
- UNBIASED COLUMN SAMPLING (ours): Horvitz-Thompson sampling of the birth columns (cubic terms, rescale p^-1/3, applied once at birth) halves the error of deterministic screening at the same cost - drift 2.57e-9 vs 5.11e-9 at a 70% keep for deep births - so part of pilot F's failure WAS coherent bias. But redrawing the sample at every layer with the state kept exact gives 2.71e-9, no further gain: the errors were already incoherent across layers. Fable measured why: the column coherence of the D21 response in the

query metric is 1.13 / 1.57 / 1.28 (L6 / L10 / L14), so an HT sampler's relative squared error is (1/q - 1)/coherence, 27-38% of the per-layer D21 response at q = 0.7. The trade stays on the wrong side (+12% raw for -5.6% C). Importance exponents 0.75 and 1.0 are worse (4.9e-9, 9.0e-9); the variance-optimal term-norm law equals e^0.5 (2.65e-9).

- QUERY-SUBSPACE RESTRICTION (Fable): restricting the contraction to the dominant subspace of the downstream propagator leaves 34 / 12 / 5% of the response at rank 128 / 256 / 384 (L6), because the response is carried by short, near-isotropic intermediate paths. There is no low-dimensional query to be orthogonal to. Dead.
- ASYMMETRIC CONTRACTION RANK (Fable's one positive candidate, implemented by us as experiments/variants/estimator_v38rc.py): keep the old legs at rank 384 and truncate only the contraction to r_c, with the basis ordered by the Gram core so a prefix of the factor slab is the dominant subspace. Dump-level prediction at r_c = 256: -6.9% C for +2-4% raw. End to end on the seed-7 network: the first implementation rotated the whole factor slab a second time every layer and the cost ROSE 3.3%; folding the ordering into the rotation the chain already performs reduced the overhead to +0.8%, but the saving never appeared, and the extra drift is 6.2e-10 (+2.9% raw). Reason: most old sources live in tier 2 (rank 224, ages 8+), so truncating tier 1's contraction saves only ~4.6 u (1.8%), which the per-layer eigendecomposition consumes; the accuracy cost is ~2x the dump prediction (the upper end of the coherence factor). Dead end to end.
- Judgement shared by Fable and our measurements: inside this formulation the cost-accuracy trade cannot be flipped by more than a few percent. A method at 133-165 u must not compute per-source D21 contractions at all, and since D3 carries most of the old tier's effect, the object that beats the chain must evade the Khatri-Rao wall - produce the per-source diagonal kappa3(i,i,i) and the covariance-path effect without forming the n x n Hadamard per source.

K3ab. Error budget of the chain (23 Sep): the closure is exact, the error is made by three inputs and carried by the mean

Question: V29/V31's final MSE (2.2e-8) is 1.3x the leaders'. Where exactly is it made? Tools: the Monte-Carlo teacher of public MLP #0 (kaggle/k4, 6e7 samples: pre-activation mean, covariance, E[y_i^2 y_j], kappa4 diagonal and (2,2)/(3,1) slices, ReLU means, at all 16 layers) and its baked 1e9-sample truth; a probe that captures the chain's own mu, var, kappa3_iii, kappa4_iiii and prediction at every layer (experiments/probes/probe_errbudget.py, plain Gaussian chain V31_ANGULAR=0 - the angular chain's internal state describes the SPHERICAL input and cannot be compared with a Gaussian teacher). The same run on the synthetic teacher network reproduces every number below.

measurement	result
per-layer MSE of the shipped chain (10 Kaggle MLPs)	0.7e-9 (truth noise) at layer 1, then +1 to 2.5e-9 per layer up to 2.29e-8: the error is ACCUMULATED, not made at the last layer
Edgeworth mean closure with the TRUE marginal cumulants (same samples)	9.3e-11 at layer 16 (5e-11 to 1.5e-10 at every layer): the mean closure is exact; my own Edgeworth code reproduces the chain's prediction to 4 digits
first-order attribution of the layer-16 error	pre-activation mean 1.84e-8 (80%), variance 2.0e-9, kappa3_iii 1.0e-9, kappa4_iiii 1.0e-9
propagation of the mean error	d mu(l) = W d m(l-1) exactly, so the inherited error has gain |W row|^2 E[Phi^2] = 0.65 at layer 2 rising to 0.8-0.96 at depth
FRESH error per layer: dm(l) - Phi o (W dm(l-1))	0.3e-9 at layer 2 rising to 4.4e-9 at layer 16; a joint fit on the three input channels (phi/2s) dvar, (G3/6) dk3, (G4/24) dk4 explains R^2 0.93-0.97 from layer 9 on, with coefficients -0.99, -0.98, -1.04 (the theoretical -1): at depth the fresh error is ~50% variance, ~25% kappa3_iii, ~25% kappa4_iiii
chain's inputs against the teacher	mean: unbiased; variance: +0.2 x rms positive bias from layer 6 (MLP #0; +0.05-0.14 on the synthetic net); kappa3_iii: slope 0.99, 4.6% random error; kappa4_iiii: 34% error at layer 16, corr 0.76-0.83 with the truth

- UNITS AUDIT OF THE K3x ORACLE (a bug of ours, now corrected): the chain's g4row is ALREADY the kappa4 diagonal (slope 0.98 at layer 2) and its wk4m = (k4_i + k4_j)/6 is ALREADY the (2,2) slice (slope 1.00 at layer 4, R^2 0.985). The first oracle injected g4row = 2 x K4d and wk4m = K22 / 3, on the angular chain. Its 6-15x degradation measured the bug, not a ceiling. K3x's end-to-end result for the per-row lambda (V35)

does not depend on these units and stands. A corrected oracle (experiments/chain/k4_oracle_run2.py) is running.

- The (3,1) model is different: the true slope of K31 on C_off is 1.3-1.55e-2 while the chain uses lambda = 0.69-1.0e-2 (x0.95), R^2 0.49-0.60. lambda is half the raw slope - consistent with F105 (lambda fits the r = 1 core, not the raw slice) and with 504aldo's scan finding 0.95 optimal.
- V39 - A DEPTH MULTIPLIER ON THE kappa4 DIAGONAL (dead, measured). The chain's kappa4 diagonal is 'too small' at depth by a smooth, network-independent factor (least-squares multiplier 1.007 at layer 2 -> 1.206 at layer 16 on MLP #0; 1.007 -> 1.185 on the synthetic net). Applied as a zero-FLOP table (experiments/variants/make_v39k4m.py) it makes the final MSE WORSE on both networks: x1.25 / x1.16 (mean closure only), x1.26 / x1.16 (everywhere), x1.09 / x1.05 at half strength. MECHANISM: the underestimate lives entirely in neurons with |mu/sigma| > 1.5 (slope 1.24-1.29 there, 700 of 1024 neurons at layer 16), where the closure's kappa4 weight G4 = (a^2 - 1) phi(a) / s^3 vanishes; on the neurons that decide the mean (|a| < 1.5) the chain's kappa4 is already right (slope 0.93-1.01; G4^2-weighted slope 0.97-0.99). A scalar multiplier damages exactly those. The first-order offline check predicts the same (fresh error x1.38 at layer 16).
- What this budget says about the leaders' 1.3x: nothing is lost in the closure; per layer, half of the new error comes from the pre-activation variance (i.e. from the post-ReLU covariance of the previous layer, mapped through fresh weights: dvar = diag(W^T dC_post W)), a quarter from kappa3_iii and a quarter from the per-neuron RANDOM part of kappa4_iiii. A 1.3x gain needs roughly a 25% cut of the fresh error at every layer - e.g. halving the covariance error, or removing the kappa4 and half of the kappa3 error. Next: which closure term makes the covariance error (probe_coverr.py regresses the measured dC_pre(l+1) on W^T X_t W for every term X_t of the (1,1) group).

K3ac. The corrected kappa4 oracle (23 Sep): the exact kappa4 DIAGONAL is worth -30% - the leaders' raw level

experiments/chain/k4_oracle_run2.py: the plain Gaussian chain on public MLP #0, with the teacher's TRUE slices injected 1:1 (units verified in K3ab) at every layer; final-layer MSE against the baked 1e9-sample truth.

injected at every layer	final MSE	ratio	L8 / L12
nothing (chain as shipped, plain)	2.290e-8	1.000	6.33e-9 / 1.33e-8
kappa4 DIAGONAL only	1.609e-8	0.702	4.08e-9 / 9.27e-9
diagonal + (2,2) + (3,1) slices	1.738e-8	0.759	3.88e-9 / 9.04e-9
(3,1) slice only	2.353e-8	1.027	-
(2,2) slice only	2.796e-8	1.221	-
(2,2) + (3,1) slices	2.680e-8	1.170	-

- THE LARGEST HEADROOM FOUND IN THIS PROJECT: knowing only the n numbers kappa4(z_i, z_i, z_i, z_i) per layer exactly takes the chain from 2.29e-8 to 1.61e-8 - the leaders' raw range (1.64-1.9e-8). The off-diagonal slices are worth nothing on top (and hurt alone: the chain's lambda/regeneration is balanced around its own diagonal). This replaces the 9% ceiling quoted from F105 for the dense G channel: the dense core is not the true kappa4 diagonal, and the true diagonal is worth three times more.
- It also locates the leaders' 1.3x plausibly: an accurate kappa4 diagonal (a small object: 1024 numbers per layer) at little cost.
- What does NOT deliver it (all measured, 23 Sep): (a) V39, a depth multiplier (K3ab: the underestimate sits in the |mu/sigma| > 1.5 neurons the closure ignores); (b) the literal pairwise transport 3 diag((W^2)^T K22_post W^2) + (W^4)^T K4_post of the chain's OWN post-ReLU slices (probe_k4diag.py: corr 0.24-0.69 with the truth against 0.70-0.83 for the chain's mean-field regeneration, although its magnitude is right, slope 1.0-1.15); (c) a G4^2-weighted regression on 13 cheap chain features, trained on one teacher network and tested on the other (probe_k4feat.py: weighted error 0.19 -> 0.18 only); (d) STAR SOURCES (probe_k4star.py): the Mehler expansion of relu of a Gaussian gives the born fourth cumulant k(h_r,h_r,h_k,h_m) = beta_r a_kr a_mr with beta_r = 2 Phi_r (1 - Phi_r) - 2 m_r phi_r / s_r and a = diag(Phi) C_off - the SAME leg as the kappa3 source, so its transported diagonal 6 sum_r beta_r T_ir^2 At_ir^2 (+ the

(3,1) and marginal stars) is O(n^2) on legs the chain already forms. It beats the chain at layer 2 (corr 0.79 vs 0.705, weighted error 0.102 vs 0.123) - the birth law is right - but linear transport shrinks it to ~10% of the true diagonal at depth: the deep kappa4 is REGENERATED at every ReLU, not carried from its births.

- Where that points: at depth the kappa4 diagonal has the magnitude of the one-step transport of the post-ReLU (2,2) slice, but the chain's own K22_post has the wrong per-neuron structure. Next: a GPU teacher of the POST-ReLU slices (K22, K31, kappa4 diagonal of h) to measure (i) the chain's K22_post error entry by entry and which closure term makes it, and (ii) how much of the true diagonal the pairwise transport of the TRUE K22_post explains.

K3ad. The complete error map (23 Sep): three numbers per neuron, and the variance is the biggest lever

New data: a POST-ReLU Monte-Carlo teacher (kaggle/k4/run_post, kernel whest-k4teacher-post, T4, 45 min; public MLP #0, the same 6e7 input samples as the pre-activation teacher; layers 3, 7, 11, 14, 15; validated locally against a two-pass computation to 2e-6 before the push). New probes: probe_postslices.py, probe_inherit.py, probe_d21.py; oracles meanfix_oracle.py and the split kappa4 oracle (estimator_k4oracle_split.py).

1. Channel oracles: the true value of ONE input of the mean closure E[relu(z_i)] at every layer

exact in the mean closure	final MSE	ratio
nothing (plain chain)	2.290e-8	1.000
pre-activation VARIANCE	1.220e-8	0.533
kappa4 diagonal	1.600e-8	0.699 (the split oracle: true kappa4 everywhere EXCEPT the mean closure = 2.311e-8, i.e. nothing)
kappa3 diagonal	1.965e-8	0.858
variance + kappa3	0.817e-8	0.357
variance + kappa3 + kappa4	0.130e-8	0.057 = the truth's own noise floor

- The chain's ENTIRE error sits in three numbers per neuron and layer - the pre-activation variance, kappa3_iii and kappa4_iiii as they enter the mean closure. With all three exact the final MSE is the truth noise. The first-order budget of K3ab (50/25/25) is confirmed and ranked by what each is worth end to end: variance -47%, kappa4 -30%, kappa3 -14%.
- The variance error is only 0.15% relative (2.4e-4 on 0.17), but the mean of the |mu/sigma| < 0.75 neurons is steep in it.

2. Where the variance error comes from

- The chain's post-ReLU slices are accurate: K22_post slope 1.01-1.14, R^2 0.999 -> 0.905; kappa4_post diagonal R^2 0.98-1.00; C_post R^2 0.9999 (error 6.5e-5 -> 9e-5 rms per entry, teacher noise 1-5e-5).
- ~95% of the next layer's variance error comes from the OFF-diagonal post-ReLU covariance error (a random projection through fresh weights: dvar_i = w_i^T dC w_i).
- That error is part inherited - Phi_i dC_pre_ij Phi_j, slope 1.01, R^2 0.52 (layer 3) -> 0.79 (layer 14) - and part FRESH: ~4.2e-5 rms per entry, constant from layer to layer. The chain's own (1,1) closure terms explain only 3-8% of it (it is not a mis-scaled term).
- The kappa3 (2,1) slice k3(z_i, z_i, z_j), which drives the off-diagonal covariance closure, is accurate but not exact: slope 1.00, R^2 0.996-0.998, RANDOM relative error 4.1% (layer 3) -> 6.7% (layer 15) - of the same order as needed to produce the fresh covariance error.
- It is NOT the old-tier truncation: with no confinement at all (V21_AGE_OLD = 20, every source dense and exact) the slice error is unchanged (6.7% -> 6.6%) and the final MSE falls only 3.9% (2.290 -> 2.202e-8) for +19% cost (util 0.248 -> 0.294): a net loss. The slice error is INTRINSIC to the K3 formulation (linear transport of the kappa3 sources through the ReLU, Gaussian-closure births).

3. Where the kappa4 diagonal comes from

- The pairwise transport of the TRUE post-ReLU slices, (W^4)^T K4_post + 3 diag((W^2)^T K22_post W^2) (+ the (3,1) term), sets the LEVEL of the next diagonal exactly (slope 1.00) but not its per-neuron pattern (corr 0.24-0.59; the chain's mean-field regeneration: 0.75-0.83). The pattern comes from the (2,1,1) and (1,1,1,1) parts of the post-ReLU kappa4 tensor.
- The residual (true minus pairwise) correlates with |kappa3_iii| at 0.27 (layer 4) -> 0.88 (layer 16) - but only through the |mu/sigma| > 1.5 neurons the mean closure ignores; on the |mu/sigma| < 1.5 neurons every cheap combination (chain diagonal, true pairwise, |D3|, t_v) lowers the G4-weighted error by 5-8% at most, in sample.

4. Reading

The leaders' 1.3x in accuracy is available inside this formulation's own three inputs - above all the variance (-47%) - and those inputs are limited by the intrinsic ~5-7% random error of the kappa3 slice and by the (2,1,1)/(1,1,1,1) structure of kappa4, not by any rank or knob. What a cheaper or better method must supply is a more exact transport of the kappa3 slice (and of kappa4's multi-index part) through the ReLU. Next: derive and test the second-order (curvature) terms of the kappa3 transport - the part a linear transport T = W^T diag(Phi) ... drops - against the per-entry slice error measured here.

K3ae. Round 7 (Fable 5 subagent, 23 Sep): the kappa3 slice error lives in the all-distinct part of the post-ReLU tensor

Task given: find the physics the K3 formulation drops that produces the 4-7% random error of the kappa3 (2,1) slice (K3ad), and whether it is affordable. Report: consultations/fable/ROUND7_fable_report.md; scripts experiments/probes/r7_*.py; variant experiments/variants/r7_estimator_fbscale.py (NOTE: its default R7_FB_SCALE is 2.0, not inert). Key numbers verified by us against the stored summaries.

- WHERE: 91-96% of the next layer's slice error comes from the ALL-DISTINCT entries of the post-ReLU kappa3 tensor (exact split of D21' = W W W [diagonal + (2,1) + all-distinct] with the same-sample post-ReLU teacher). The chain's own post (2,1) slice and kappa3 diagonal are right to 1-6% / 0.1-2% and carry <= 9% of the error variance. Relative to itself the all-distinct part is 9% (layer 3) to 16% (layer 15) wrong.
- DERIVATION (Edgeworth, first order in K, up to C^3): the ported newborn Sym(X1 x I x Y1) carries the Gaussian hub exactly but exactly HALF of both K x C all-distinct families - the lean chain's B2 block and its WK12 term of Y1 (which carried the other half) were not ported. Regression coefficients on the measured error confirm the derivation: family A -0.55..-0.69, family B -0.6..-0.9 (predicted -1/2); doubled edge, triangle and transport curvature -0.9..-1.4 (predicted -1). Together they explain 13-21% of the error variance at one layer, 20-35% accumulated over births.
- END TO END (zero FLOPs: scale the 1.5 / 0.5 feedback constants by S; S = 2 is the full Edgeworth value), paired against the baked truth / MC teacher: S = 1.25 -> -0.8% (MLP #0) / -0.35% (synthetic); 1.5 -> -0.45% / +0.15%; 1.75 -> +0.9% / +1.3%; 2.0 -> +3.4% / +3.1%. The FULL correction improves the slice at layers 4-10 but hurts at depth: the fresh coefficient is right but the added content is over-propagated (the same phenomenon as 504aldo's F69 'B2 hurts'). The only affordable gain is S = 1.25 (~0.5%), to be checked on the 100-MLP suite.
- DEAD (each <= 2% of the error variance at depth): the transport curvature (no coherent part, per-source slope +-0.001; n^4 per source-layer), the kappa4 (2,1,1) hub, self-loop kappa3/kappa4 hubs, the doubled edge (+2 units per source-layer for R^2 0.01), the triangle, the rank-16 truncation of the feedback (R^2 0.09 -> 0.004), the lam feed's all-distinct content, the inherited-C hub.
- UNEXPLAINED: ~55-65% of the all-distinct error lies beyond the first-order-in-K expansion (teacher noise ~1%): second order in K, or cross-talk of the 4-7% D21 error back into the feedback legs at every layer. A one-run oracle that injects the TRUE D21 into the feedback legs only would isolate the latter.
- VERDICT (Fable, which our numbers support): the slice error is intrinsic to hub births plus linear transport; the leaders' 1.3x will not come from the kappa3 births. Together with K3ad this leaves the VARIANCE

channel (-47% if exact) and the kappa4 diagonal (-30%) as the open levers; both are the subject of the round-7 Astra prompt (consultations/prompts/ROUND7_prompt_astra_three_numbers.md).

K3af. Round 7 answers (Astra, GPT 5.6 Terra, Grok, Copilot) and the teacher-forced closure (23 Sep)

All round-7 answers are triaged in the new experiment queue (docs/experiment_queue/Experiment_Queue.pdf, Bengali/English, built from queue.json by build_queue.py through headless Edge). The measured items:

- ASTRA COVARIANCE PATCH (Gaussian C^3, C^4 and C x kappa31 terms absent from V29's (1,1) closure): dead. The terms have rms 2-3e-6 against a covariance error of 6.5-9e-5, correlation with the error -0.03..+0.002, projected variance error x0.998-1.007 (probe_astra7_cov.py).
- ASTRA QUARTIC kappa4 KERNEL (fresh quartic response polynomial at every ReLU, pair part replaced by the chain's own slices): the first route that captures the multi-index kappa4 pattern - its non-pair part correlates 0.63-0.78 with the true non-pair part on both teacher networks - but it is 1.3x (layer 4) -> 2.2x (layer 16) too small. End to end in the mean closure: A = 0.5, S = 1.5 gives -6.4% (MLP #0) / -2.6% (synthetic); A = 0.3, S = 1.25 gives -5.0% / -1.5%; without the pair product it double counts and fails. Cost 3 products per layer, ~18-26 u (+7-10%): net about zero as it stands (probe_astra7_k4*.py, a7_kernel_e2e.py).
- TEACHER-FORCED CLOSURE (queue Q5): with every closure input replaced by the teacher at one layer, the post-ReLU covariance error falls to the teacher noise (layer 7: 7.04e-5 -> 2.61e-5, noise 2.5e-5; layer 14: 8.84e-5 -> 2.76e-5) and the sensitivity-weighted next-layer variance error by 87% / 91%. By input group (layer 7 / 14): mu, var, C_off -> 0.376 / 0.185; kappa3 -> 0.715 / 0.853; kappa4 -> 0.960 / 0.998. The closure is essentially exact; the covariance error is mostly INHERITED mean/covariance error (81% at depth), and the fresh part comes from the kappa3 inputs and a small closure residual. There is no separate variance term to fix: the variance oracle's -47% is the accumulated effect of the per-layer kappa3/kappa4 errors.
- GPT 5.6 TERRA: new diagnostic queued (Q10: is the covariance error low-rank through the target rows?); its 'local-birth factorization' of K4(j,j,:,:) is the kernel above plus the K3 x C term (Q8). GROK: its covariance term is a rescaling of existing D21 terms (closure-term regression R^2 <= 0.08) and its kappa4 formula is the pairwise transport already measured (corr 0.24-0.59); its sampling costs are off by 1000x (2e4 samples = 320 u) and independent Gaussian draws cannot reduce variance. COPILOT: Hutchinson probes need the unknown error; its transported matrices are the dead aggregate family. A coupled-surrogate Monte-Carlo kill-test is queued (Q11) because three answers raised it; Astra's theory says a 100x reduction needs correlation >= 0.997.

K3ag. Round 8 (24 Sep): Fable 5 synthesis of eleven round-7 answers; the 100-MLP knob results; the kernel's missing (1,1,1,1) term

Fable 5 was given the whole experiment queue (docs/experiment_queue), the eleven round-7 answers and today's measurements, and asked for the fragment the individual answers missed (report consultations/fable/ROUND8_fable_synthesis.md, probes experiments/probes/r8_*.py).

- THE MISSING PIECE OF THE kappa4 KERNEL: its depth deficit (1.25x -> 1.9x too small) is the (1,1,1,1) K3 x C '3+2' term evaluated with the ALL-DISTINCT pre-activation kappa3, i.e. the chain's own sources: for a hub block Sym(X x P x Y), t = (1/3)[X (P^T x o Y^T x) + P (X^T x o Y^T x) + Y (X^T x o P^T x)], minus the repeated-index part. GPT Sol's slice approximation t_rep lives in the (2,1,1)/(3,1) sectors by construction and cannot see it. Measured coefficient on the kernel residual 1.0 +- 0.1 (theory 1); decisive-neuron non-pair corr 0.69-0.75 -> 0.75-0.81; G4-weighted kappa4 error 0.095 / 0.116 / 0.157 / 0.193 / 0.197 at layers 4 / 8 / 12 / 15 / 16 (chain 0.124 / 0.142 / 0.193 / 0.199 / 0.210). Source ages 1-4 carry ~90%, ages 1-2 ~65%.
- End to end in the mean closure (MLP #0): the round-7 blend A = 0.5, S = 1.5 goes from -6.4% to -8.8% (2.0893e-8) with the term, at ~140 u of extra work (the term costs two products per source-layer at the query width): not shippable. A cheap version (layers 8-16, |mu/sigma| < 1.5, the two youngest sources; ~26 u

classical, ~13 u Strassen) is under test.

- DEAD (Fable, measured): the second order in K of the all-distinct kappa3 tensor (12 K x K terms, rms 3e-6..2e-5 against an error of 2.2e-4 - which also removes the round-7 explanation of why FB_SCALE 2 hurts); the coherent rank-1 (mean-direction) part of the D21 error (decisive neurons never see it); kappa4 memory from transported star births (anti-correlated with the residual, coef -0.9); the exact (2,1,1) law and GPT Sol's nine hypertree classes (fixed-coefficient R^2 <= 0.07); the (3,1) sector (3e-5 of 1e-3); non-Gaussian vertex weights (ratios 1.00). About 55% of the all-distinct kappa3 error stays unexplained through second order: a floor of this formulation for the variance channel.
- 100-MLP KNOB RESULTS (Kaggle, bit-identical deterministic base on every runner, paired SE 0.01-0.15%): beta 1.2 score x0.9993; LAM 0.9 x1.0026; per-row lambda 0.25 x1.0003; QPASS2 3 x1.0006; AGE_OLD2 8 raw x0.991 but util x1.010 -> score x1.0006; QPASS 2 raw x0.990 but util x1.083 -> score x1.073. The chain sits at a score optimum in every knob (accuracy is bought one-for-one with cost). Local base score 5.364e-9 (raw 2.2049e-8) against the leaderboard's 5.137e-9: predicted LB = 5.137e-9 x paired score ratio. The R7 feedback scale (1.25 / 1.5, with and without beta 1.2) is running on 100 MLPs.

K3ah. Moonshot A (24 Sep): the metered cost ledger, the join rebuild, and the free gains shipped

- SUBMISSION #332110 (v40fb = V31 + R7 feedback scale 1.25 + adaptive-lambda exponent beta 1.2, hard-coded defaults, zero extra FLOPs): 100-network paired score x0.9946 +- 0.0007 (73% of networks better); graded 5.1284e-9 (raw 2.1046e-8), the new best (from 5.137e-9); leaderboard display rounds to 5.1e-9, rank 33 on 24 Sep (others moved up).
- COST LEDGER (experiments/probes/m1_matmul_ledger.py, every matmul/einsum attributed to its calling line): 249 u total, 220 u (88%) matmul, einsum 9 u, Strassen additions ~12 u; float64 promotions cost nothing. By family: young-source transport ~54 u, young-source D21 contractions ~50 u, joining a source to the shared basis ~41-46 u (per join: range finder 1.5 u, projections 0.75 u, rotations 0.9 u, cores 0.8 u; 10 joins), old sources in the basis ~41 u, thin feedback legs ~15 u, covariance and closure ~8 u. Halving the cost therefore means halving matmul work; the young tier (~104 u) and the join (~43 u) are the largest targets.
- JOIN REBUILD (V41, experiments/variants/make_v41join.py): rebuilding the basis from the transported old basis without the range finder cuts cost 6.6% but costs 6.3% / 1.0% raw MSE (MLP #0 / synthetic); adding k fresh directions of the joiner and truncating by the eigen-decomposition of the full core in the (r + k)-dim span (reusing the joiner's projections) gives, at k = 64, util +0.5% and raw -1.0% / -1.1%: score -0.5% / -0.55% on both networks (a small, consistent gain; k = 16 / 32 worse, k = 96 dearer). The eigen-truncated basis is more accurate than the range finder's, but the join cannot be made cheaper without losing accuracy.
- THE ONE-FOR-ONE LAW (100 networks + two teacher networks): every change inside the chain - age gates, basis ranks, subspace passes, birth-column screening, the join rebuild, the kappa4 kernel restricted to late layers - buys accuracy with cost at about 1% per 1%. 504aldo's design sits on its score optimum everywhere. One leaderboard team (raw 2.0e-8 at 0.126 B) shows that equal accuracy at half the cost exists; the mechanism is outside this family.
- ASTRA ROUND 9 (architecture assessment): no estimator reaching both targets could be derived; a mixed-coordinate readout makes the kappa4 all-distinct K3 x C term cost 17-34 u (break-even 25 u for the measured -8.8%); the FB-scale 'paradox' is ordinary quadratic cancellation (E(t) = 1 - 0.034 t + 0.068 t^2, optimum t = 0.25, -0.43%, as measured); an explicit kappa3 tensor (~1567 u per layer), a reverse cubic circuit (~2 u per source-layer) and a young-tier-only speed-up (144 u remain even if all 116 u vanish) are ruled out by arithmetic.

K3ai. Round 9 (GPT 5.6 Sol): source-free DHMP killed by its own one-step gate

- PROPOSAL: Dressed Hypertree Moment Propagation - drop the kappa3 source bank; at every ReLU rebuild the all-distinct post-ReLU kappa3 as a dressed Gaussian tree K3(i,j,k) = tau_i E_ij E_ik + perms (tau = E[relu"]/E[relu']^2, E the exact bivariate covariance), keep the repeated-index planes exact, and contract directly with the next weights (R = E Q, H = tau o R^2 + 2 E((tau Q) o R), D21 = H^T Q): 8-10 products per layer, claimed 74-90 u.
- GATE (experiments/probes/r9_dhmp_gate.py, teacher-forced with the true post-ReLU E, T, d3 of MLP #0 and the true next slice): relative D21 error 40.4 / 38.4 / 36.4 / 36.7 / 36.3% at next layers 4 / 8 / 12 / 15 / 16 against a 2.5% gate (the chain: 4-7%). The all-distinct part is 40-48% of the slice; the fresh tree has the right scale (slope 0.88-1.16) but correlates only 0.43-0.54 with it: about half of the all-distinct content is MEMORY carried from earlier layers - the same fact as 'no old tier: x35'. Dressed versus bare edges changes the error by 0.4-1%, so it does not explain the feedback-scale paradox either.
- Also proposed: FHQR-2 (two global query-response matrices K3[:, Q, Q] for the next one and two layers, tree tail) and a quadratic-chaos Galerkin state; queued (Q18) with low prior because the measured memory decays only x0.66 per age.

K3aj. The horizon oracle (24 Sep): the kappa3 memory is long, flat and nearly orthogonal across ages

Suggested by GPT 5.6 Sol after the DHMP gate. From Fable's dense source captures (every live source kept dense; the sum of the sources' own (2,1) slices equals the chain's D21 to 1.5e-7), the relative D21 error of keeping only the sources of age <= h (experiments/probes/r9_horizon_oracle.py):

layer (sources)	h = 2	h = 4	h = 6	h = 8	h = 10	age shares of ||D21|| (age 1, 2, 3, ...)
7 (6)	0.561	0.234	0 (all)	-	-	0.45 0.36 0.29 0.21 0.17 0.10
11 (10)	0.723	0.476	0.253	0.102	0 (all)	0.30 0.29 0.26 0.21 0.19 0.15 0.11 0.09 0.07 0.04
14 (13)	0.807	0.602	0.410	0.243	0.123	0.22 0.21 0.19 0.17 0.17 0.15 0.12 0.11 0.09 0.06 ...
15 (14)	0.840	0.654	0.460	0.305	0.164	0.20 0.20 0.19 0.17 0.15 0.15 0.13 0.11 0.10 0.08 ...

- Every age contributes a similar share and the shares decay slowly; their squares sum to ~1, so the contributions are nearly orthogonal - no pooling of old ages into a small common object, and no short horizon (FHQR-2, h = 2, leaves 72-84%). The same holds on the rows of the decisive neurons.
- CORRECTION (pointed out by GPT 5.6 Sol): the squared shares sum to only 0.25-0.49, so the ages are NOT orthogonal - 50-76% of the energy comes from positive cross-age inner products. AGE-GRAM TEST (experiments/probes/r9_age_gram.py): mean pairwise cosine 0.29 / 0.32 / 0.39 / 0.38 (layers 7 / 11 / 14 / 15); the first age-mode holds 47 / 35 / 36 / 34% of the energy with same-sign loadings on every age. Projecting the age axis on its first r modes leaves an error in the TOTAL D21 of 12.2% (r = 1), 10.3% (r = 4), 8.5% (r = 6) at layer 15 and 23.5 / 14.1 / 7.4% at layer 11: one coherent mode carries ~88% of the deep total, but the remaining ~12% is spread over many modes, so the 2% target is out of reach along the age axis. The memory is long AND coherent, but not low-rank across ages.
- SHIFT-STABLE CARRIER (GPT 5.6 Sol; experiments/probes/r9_shift_carrier.py): is the first age-mode S_l = sum_a v_l(a) K_(l,a) carried as one object, S_(l+1)^old = rho T_l S_l? L14 -> L15 on exactly transported sources: rho* = 0.944, eps_shift = 2.9% (weight-vector mismatch 5.9%) - Sol's 'backbone' band (2-5%), not the 2% pass. The carrier plus a fitted birth term reproduces the TOTAL L15 D21 only to 13.9%. KEY POINT: any fixed age weighting is transported by the same linear map, and uniform weights (the total kappa3 itself) are shift-stable EXACTLY, so the coherent mode adds no temporal capability the chain lacks. The memory problem is purely SPATIAL: a compact representation of one cubic tensor (the total) that survives the per-layer transport M^(x3) and the ReLU dressing more cheaply than the chain's 3 k n source columns - the question Tucker / TT / ISDF / vector-metric failed. The next consultations target exactly that.
- NO LINEAR COMPRESSION CLOSED UNDER TRANSPORT (GPT 5.6 Sol, argument): a linear state s = L(K) of the total symmetric kappa3 that is closed under every dense M^(x3) and yields the next D21 for every

full-rank M must be injective (a lost K0 has some x with K0[x,x,x] != 0; put x in the first row of M), so it needs >= C(n+2,3) ~ 1.8e8 numbers: Sym^3 is an irreducible GL(n) representation. Tucker / TT / CP / vector-metric / age modes / shared bases are all attempts at this. Caveat (ours): our M are fixed and known, so the relevant bound is the dimension of what the FUTURE readouts can observe, <= n^2 per remaining layer (+ n for d3) - comparable to the chain's ~3 k n^2 at mid depth, so there is no free factor 2 there either.

- LAZY CUBIC CIRCUIT (Sol): store only the birth matrices and the dense maps, p_l(x) = sum_b p_b(T_(l:b)^T x), and read D21 by batched reverse mode (one backward sweep of the n queries, 2 products per birth, one adjoint sweep, one final product): ~4 l square products at depth l, ~480 over 16 layers, ~245 u - the same as the chain's 249 u. Source-forward propagation and reverse-mode evaluation are dual contraction orders of one tensor network. Not implemented (the count is fixed by the structure and matches the metered ledger); recorded as the reason that any exact carrier of the total kappa3 costs what the chain costs. Conclusion: the leader must change WHAT is represented (a nonlinear trajectory-specific statistic, or the needed marginal channels computed without the kappa3 tensor), not how the same tensor is factorised.

K3ak. The chain's own per-channel error against the teacher (24 Sep, the Round 11 stage-1 bar)

Free-running plain chain vs the 6e7-sample MC teacher of public MLP #0, relative Frobenius error (experiments/probes/r11_chain_channel_ref.py; teacher sampling-noise floor in brackets). Mean in rms units of sigma.

layer	mean	variance	d3 = kappa3_iii	kappa4 diag (slope, corr)	C off-diag	D21 off-diag
7	1.9e-4	0.04% (0.02%)	3.2% (1.1%)	15% (3.6%) (0.95, 0.78)	0.39% (0.17%)	4.6% (1.3%)
11	3.1e-4	0.09% (0.02%)	3.8% (0.6%)	22% (2.5%) (0.89, 0.78)	0.59% (0.13%)	5.7% (0.8%)
14	4.3e-4	0.12% (0.02%)	4.1% (0.4%)	32% (2.0%) (0.79, 0.81)	0.70% (0.11%)	6.2% (0.5%)
15	4.6e-4	0.14% (0.02%)	4.6% (0.4%)	32% (2.0%) (0.78, 0.83)	0.73% (0.10%)	6.6% (0.5%)
16	5.1e-4	0.14% (0.02%)	4.7% (0.4%)	34% (1.8%) (0.75, 0.82)	-	-

- Every deep error is far above the teacher noise. The bar for a replacement is severe: variance to ~0.1% and off-diagonal covariance to ~0.7% - why sampling fails and why kappa3 must be carried at all.
- The weakest channel is the kappa4 diagonal: 32-34% at depth with a systematic slope 0.75-0.79 (~22% too small along the truth), correlation only 0.78-0.83; the kappa4 constants are at their score optimum, so what is missing is the pattern, not the scale.
- ROUND 11 (GPT-6 Astra): no qualifying mechanism - an honest 'no'. Its proposed state (the history of the visible channels m, C, Y, k4; ~2 L n^2 numbers) needs a nonlinear update it could not supply. Its argument: an all-distinct perturbation at (a,b,c) is invisible in the current slice and diagonal but changes the next slice by 2h[outer(u*v,w)+perms], so visible channels are not sufficient in general; for the realisable family this matches DHMP (half the all-distinct part is memory). The budget envelope 125 u = ~15 square products per layer. Its injection falsifier is not run: it passes or fails by construction and says nothing about random MLPs.

K4. Next experiments queued

- DONE: Kaggle knob sweep (K3f), aggregate-state pilots (K3e), slice-transport (K3g), column screening (K3h), loopholes B1/A4/B4, reference K3-simple (K3i).
- DONE 21-22 Sep: compensating closure (K3k, redone correctly in K3o), rank ladder (K3j), ARC reference chains (K3i), round-3 falsifiers (K3l), adjoint line (K3m), round-4: angular input SHIPPED (#331787 5.137e-9) and ISDF dead (K3n), audit (K3o), precision (K3p). GRADED best: #331787 5.137e-9.
- NEXT: the accuracy side is where the leaders' 1.3x lives (97% of V29's error is the K3 + memoryless-kappa4 truncation, K3p): cheap higher-order corrections (a better (2,2)/(3,1) kappa4 model than lambda C_off; the sphere's 2 M_ij^2 term; ARC's dropped augment terms) are the remaining open axis. Cost side is closed within this formulation.

- Question A: outside-model ideas (Astra 6 / Fable 5) on representations that avoid elementwise leg products; test the most concrete one on the lean chain.
- Question D: online-lambda-only variant (drop the fitted table) on 10 mini MLPs.
- Write the K-2 security email (private) and the K-1 docs issue; post K-3 after #331686 grades.

Part I — Checklists, troubleshooting, links

I1. Pre-submission checklist (from the kit, condensed)

- validate: every Checks row OK (not just the green panel). Local and subprocess runs agree within ~1%.
- Every MLP: budget_exhausted false, residual_wall_time_s well under 0.4 s (rehearse 0.2), time_exhausted false, fits 8 GB, no clock-calibrated internal deadlines (prefer budget-based cutoffs).
- Allowed code: no vendored libs / kernels / FFI / concurrency / in-flight compute / accounting patches; only flopscope + whestbench + stdlib imports (grep your file: your venv has numpy so a local run won't flag it).
- Reproducibility: no reads outside submission_dir, no network, seeds only from mlp.seed / ctx.seed via default_rng.
- Sanity: post-ReLU means for EVERY layer, shape (mlp.depth, mlp.width) (off-by-one and hard-coded (32,256) are the two classic bugs); setup() idempotent and far under 5 s (rehearse --setup-timeout 2.5); no print() in predict().
- Folder mode if multi-file; tar tf lists estimator.py + manifest.json (+ data); under 50 MiB / 50 files; secrets excluded.

I2. Troubleshooting table

Symptom	Cause -> fix
Wrong shape / NaN / Inf	Read shape from mlp; clamp var >= 1e-12; guard divisions; rescale cov if diag > 1e30 (float32 max 3.4e38) instead of going float64
budget_exhausted	F_m > 2**41: rank ops with the summary; float32; fewer/fused contractions; diagonal where full is not needed
residual_wall_time_exhausted	Python outside flopscope > 400 ms: remove float()/bool()/int() syncs, per-neuron loops, asserts, prints; move MLP-independent work to setup(); vectorise
time_exhausted / PREDICT_TIMEOUT	> 120 s: hang or giant op; chunk; check loops; rehearse --max-threads 1
SETUP_TIMEOUT / SETUP_ERROR / SETUP_FAILED	> 5 s or exception in setup(): load only, precompute offline, str() paths, guard None, try/except with safe fallback, never scratch_dir
IMPORT_FAILED / manifest mismatch	hand-rolled tarball -> whest package / validate-package
ModuleNotFoundError numpy|scipy / NameError np	sandbox has only flopscope + whestbench + stdlib; use fnp; ship results as .npz; do not except ImportError around numpy
helper.py / weights.npz missing on grader	single-file packaging -> --estimator . ; check .gitignore patterns too
'flopscope arrays are immutable'	no item assignment / in-place ops; fnp.where, concatenate, arr = arr + x; fill_diagonal is the exception
result array too large	> 4 GiB single array (grader only): block-wise processing, float32
UnsupportedDtypeError / axis list TypeError	numeric dtypes only; axis as int or tuple
Run hangs under --runner subprocess	a print()/stdout write in setup() or predict(): use sys.stderr or logging
Local great, grader 10x worse	cross-MLP state; local-only imports; unseeded randomness; residual time on slower hardware or clock-based fallbacks
flops_used == 0	you used numpy not fnp
One MLP far worse than the rest	that MLP failed (x1.0); read failure_breakdown / per_mlp flags with --debug --fail-fast
FLOP counts changed after uv sync	meter version moved; re-measure; never trust numbers from an older kit or Phase 1
WATCHDOG_TIMEOUT on some MLPs / no_worker_capacity	usually infrastructure: resubmit; on all MLPs: a hang
Ambiguous estimator classes	name it Estimator or pass --class
BLAS pool WARN in doctor	expected on Apple Silicon (Accelerate invisible to threadpoolctl); on Linux reinstall numpy wheel; FLOPs unaffected

I3. Links

Resource	URL
Challenge page / leaderboard / submissions / rules tab (rules render only when logged in)	https://www.aicrowd.com/challenges/arc-white-box-estimation-challenge-2026
Forum category	https://discourse.aicrowd.com/c/white-box-estimation-challenge-2026
Phase 2 launch post (dates, changes)	https://discourse.aicrowd.com/t/phase-2-of-the-arc-white-box-estimation-challenge-is-live/18197
Algorithmic prize guidelines	https://discourse.aicrowd.com/t/algorithmic-contribution-prize-guidelines-how-arc-judges-these-prizes-discretion-technical-writeups-llm-usage/18041
Key write-ups	504aldo K=3 (…/18218); compute-budget-not-a-lever (…/18219); exact bivariate covariance (…/18214); zero fitted constants (…/18216); 0.75/0.25 blend (…/18200); residual-cap thread (…/18202); Phase 1 census (…/18157); Phase 1 solution (…/18193)
Starter kit / whestbench / flopscope	https://github.com/AIcrowd/whest-starterkit ; https://github.com/AIcrowd/whestbench (CLI reference: docs/reference/cli-reference.md) ; https://github.com/AIcrowd/flopscope ; cost model: https://aicrowd.github.io/flopscope/docs/understanding/flop-counting-model/
Explorer (rows = layers, columns = neurons, colour = mean activation)	https://aicrowd.github.io/whestbench-explorer/
Dataset	https://huggingface.co/datasets/aicrowd/arc-whestbench-public-2026/tree/v2-phase2
Paper + code	https://arxiv.org/abs/2605.05179 ; https://github.com/alignment-research-center/mlp_cumulant_propagation
ARC posts	https://www.alignment.org/blog/announcing-the-arc-white-box-estimation-challenge/ ; https://www.alignment.org/blog/competing-with-sampling/ ; https://www.alignment.org/blog/algzoo-uninterpreted-models-with-fewer-than-1-500-parameters/
AIcrowd API key	https://www.aicrowd.com/participants/me/edit
Contact	arc-whestbench@aicrowd.com (rules, mispricing, residual-cap exceptions, write-ups); GitHub Issues for bugs

The AIcrowd Rules tab is not visible without logging in; the binding terms above are taken from the starter kit's Phase 2 documentation and organizer forum posts that quote the Rules. Log in, read the Rules tab once in full, re-accept if prompted, and check the Submissions tab for the final-submission designation control before 17 October.